SYSTEM OWNER'S HANDBOOK

Product and Architecture (Parts 1–3)

Multi-tenant clinic management platform

Status as of 2026-10-28 · code is the source of truth · written for a software-engineer owner

How to read this document

This is part of the **System Owner's Handbook** for a multi-tenant clinic management product, called "the Platform" throughout. It is written for the product owner, a software engineer, and every chapter can be read on its own.

Status labels. Every feature carries exactly one label, based on the code as of 2026-10-28:

- **[Implemented]**: built end to end.
- **[Partial]**: some of it exists; the text says exactly what is missing.
- **[Not implemented]**: planned, seeded, disabled or only mentioned. It is never described as working.
- **[Unverified]**: the code was read but the behaviour could not be confirmed; the text says what is unconfirmed.

Source of truth. The code is the source of truth. Where older documentation disagreed with the code, the handbook follows the code. Nothing was executed while writing: behaviour is described from reading the code.

Figures. Every figure has a number, a caption and a paragraph that describes it in words. Figures captioned "screenshot pending" are placeholders; the paragraph after them describes the screen from the code. The few real screenshots were taken on 2026-10-23 and show a former working name of the product.

Names. "Organization" is one clinic business (the tenant); "location" is one of its branches. File paths and code names appear in a short "Technical references" line at the end of each feature. Example people and places are fictional.

Table of contents

Part 1 — Product overview

Chapter 1 — What the Platform is 4

Chapter 2 — Roles: who uses the Platform 8

Chapter 3 — A day in the clinic 13

Part 2 — How the system is built

Chapter 4 — Repository and tech stack 18

Chapter 5 — Request lifecycle: from a click to the database and back 24

Chapter 6 — Processes, deployment and environments 33

Chapter 7 — Running the Platform locally 39

Part 3 — Organizations, locations, roles and permissions

Chapter 8 — The multi-tenant model 43

Chapter 9 — Organizations and locations 50

Chapter 10 — Features: entitlement, enablement and location overrides 60

Chapter 11 — Roles and permissions 68

Chapter 1 — What the Platform is

This chapter explains what the Platform is, who it serves and which parts are built today. It is the first chapter of the System Owner's Handbook and is written for the product owner, who is a software engineer and needs a clear picture before reading the detailed chapters. It ends with a map of the 18 module chapters, so a reader can find any topic.

1.1 One paragraph summary

The Platform is a **multi-tenant** web application that runs the daily work of clinics. "Multi-tenant" means one running system serves many separate clinic businesses, and each business sees only its own data. One clinic business is an **organization**. An organization has one or more **locations** (branches). Staff log in, and what they see depends on their **role** and on the **features** the organization has switched on. The backend is a NestJS API with a PostgreSQL database. The front end is a React web app. Chapter 4 describes the stack, and Chapter 8 describes how tenants are kept apart.

1.2 What a clinic gets

A clinic uses the Platform to do these jobs:

- keep a **patient** record, including clinical history, vitals, documents and signed consents (Chapter 28);
- book **appointments** and manage the calendar (Chapter 29);
- run a doctor-wise waiting line with **tokens** after the patient arrives (Chapter 30);
- let the doctor work from one screen, the **consultation workspace** (Chapter 31);
- write and print **prescriptions** (Chapter 32);
- plan treatment in steps with a **treatment plan** (Chapter 33);
- bill patients, take **payments**, issue **invoices** and close the cash counter (Chapter 34);
- calculate doctor **commission** (Chapter 35), run reports (Chapter 36), manage **employees** (Chapter 37), **payroll** (Chapter 38) and **inventory** (Chapter 39);
- send work to a dental **lab** and track it, including a separate **lab portal** for the lab (Chapters 40 to 42);
- manage subscription and sign-up (Chapter 45).

1.3 Specialties: general, dental and others

An organization carries one or more **specialty tags**. The code lists six: general, dental, skin, ortho, physio and eye. The sign-up form offers them as choices, for example "Skin / dermatology", "Orthopaedics", "Physiotherapy" and "Eye care". The tags describe which **features** are relevant to a clinic.

What is general. Patients, appointments, the queue, consultation, prescriptions, treatment plans, billing, commissions, reports, employees, payroll and inventory work for any clinic type. Their feature keys carry the tag "general".

What is dental-specific. Two parts are built for dental clinics:

- the **teeth chart** (FDI tooth numbering), whose feature `patients.teeth_chart` is tagged dental and ortho, and the "dental info" record of a patient;

- the **dental lab** ordering and production flow, whose feature is `dental.lab_orders`, tagged dental and ortho. The Platform ships dental lab workflows (for example "Try-in stage" and "Complete denture" stage lists) as seed data for the workflow engine.

Honest summary. Dental is the most developed specialty. A clinic of another type can use every general module today. The tags for skin, ortho, physio and eye exist as labels, but this chapter found no screens or rules written only for skin, physio or eye care **[Partial]**. The workflow engine (Chapter 43) is designed so a new specialty can be added as data, not code, but only dental data is seeded.

1.4 Market defaults visible in the code

The first customers are clinics in Pakistan. Several defaults show this. They are **defaults, not limits**.

Default	What the code does	Can it change?
Currency	An organization's currency column defaults to PKR. Billing screens show "Rs".	It is a column per organization (and one per location), so it can be set.
Timezone	Every location has its own timezone. Clinic dates and times (queue day, "today", no-show times) follow the location's timezone.	Yes, per location.
Urdu on prescriptions	A prescription PDF prints Urdu text in Nastaliq script, right to left.	It applies to typed text; printed labels stay English.
Tax	Sales tax is **exclusive**: it is added on top of the price.	Tax is configured in billing settings.
Roman-Urdu text	The "No access" page shows the headline "Aapke paas is page ki permission nahi", with English text under it.	It is fixed text in the web app.

The table lists the visible regional choices. They show the product is built first for Pakistani clinics, but nothing in them blocks another market. One point is **[Unverified]**: the location record has its own currency column whose database default is USD, while the organization default is PKR, and this chapter did not trace which one each billing screen reads.

1.5 What is and is not built

Every feature in this handbook carries one of four labels: **[Implemented]** (built end to end), **[Partial]** (some of it exists), **[Not implemented]** (planned or only mentioned) and **[Unverified]** (not confirmed from code). The table is the one-page summary of the largest items.

Area	Status	Note
Patients, appointments, queue, consultation, prescriptions	[Implemented]	Some sub-features are backend only; each module chapter lists them.
Billing, payments, invoices, cash close, refunds	[Implemented]	The invoice is issued when payment is collected.
Commissions, payroll, employees	[Implemented]	Payroll records are generated by a monthly job.
Dental lab, lab portal, lab finance	[Implemented]	A lab order is not linked to a consultation.
Reports and dashboards	[Implemented]	Reports can be exported.
Inventory	[Partial]	The screen exists but has no sidebar entry; the daily alert job only writes a server log.

Area	Status	Note
Patient portal	[Not implemented]	Patients cannot log in.
SMS, email and WhatsApp messages to patients	[Not implemented]	SMS and email feature keys are retired. A WhatsApp key exists in the feature catalog, but no patient messaging is built.
Online booking	[Not implemented]	Retired feature key.
Recurring appointments	[Not implemented]	Retired feature key.
Appointment reminders	[Not implemented]	Retired feature key.
Sign-in with Google or GitHub (OAuth)	[Not implemented]	The sign-in is disabled.
Notifications	[Partial]	In-app only.
Chairs	[Not implemented]	A database entity only; no screen or API.
Insurance claims and online payments	[Not implemented]	Retired feature keys.

The table shows that the core clinic workflow from booking to payment is built, while everything that reaches out to the patient (portal, messages, online booking) is not. Self-sign-up with a 14-day trial of the Professional plan exists in the backend and a register page exists in the web app; subscription payments are recorded by a super admin by hand (Chapter 45).

1.6 How a patient flows through the Platform

Figure 1.1 shows the main modules and the usual path of a patient.

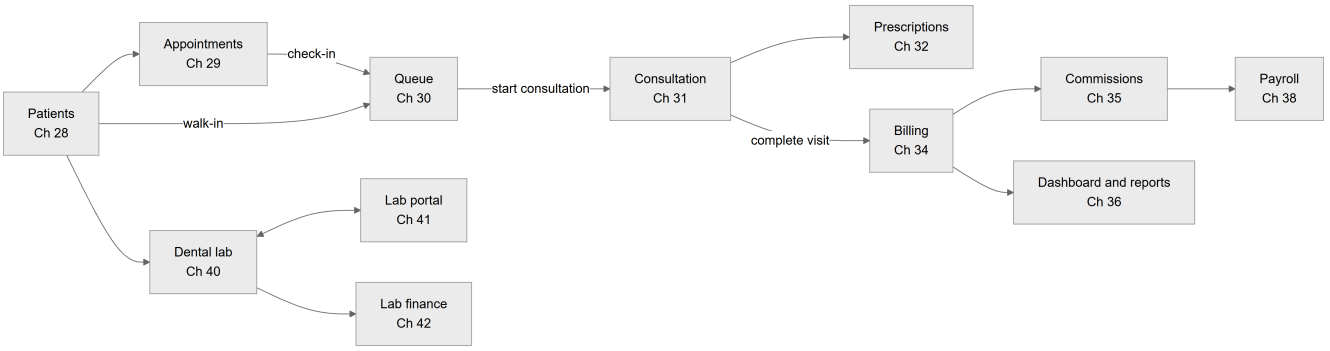


Figure 1.1 — Main modules and the usual flow of a patient, with chapter numbers.

The flowchart reads from left to right. The box "Patients (Ch 28)" has three arrows out of it. One goes to "Appointments (Ch 29)", and an arrow labelled "check-in" leads from Appointments to "Queue (Ch 30)". A second arrow, labelled "walk-in", goes straight from Patients to Queue, because a patient without a booking joins the queue directly. The third goes down to "Dental lab (Ch 40)". From Queue, an arrow labelled "start consultation" leads to "Consultation (Ch 31)". Consultation has two arrows out: one to "Prescriptions (Ch 32)", and one labelled "complete visit" to "Billing (Ch 34)". Billing has two arrows out: one to "Commissions (Ch 35)", which continues to "Payroll (Ch 38)", and one to "Dashboard and reports (Ch 36)". The Dental lab box has a two-way arrow with "Lab portal (Ch 41)", since the clinic and the lab both act on the same order, and a one-way arrow to "Lab finance (Ch 42)".

1.7 Map of the 18 module chapters

Chapter	Module	One line
28	Patients	The patient record and everything attached to it.
29	Appointments	Booking, rescheduling, check-in.
30	Queue	Doctor-wise live waiting line with tokens and a TV display.
31	Consultation	The doctor's one-screen workspace for a visit.
32	Prescriptions	Writing, signing and printing prescriptions, with Urdu.
33	Treatment plans	Planned work per patient and reusable plan templates.
34	Billing	Charges, payments, invoices, refunds, cash close.
35	Commissions	Doctor compensation plans and earnings.
36	Dashboard and reports	The home dashboard, finance overview and reports.
37	Employees	Staff records, assignments, schedules, leave.
38	Payroll	Monthly payroll records and payments.
39	Inventory	Stock, suppliers and purchase orders.
40	Dental lab	The clinic side of lab orders and production.
41	Lab portal	The lab's own login and screens.
42	Lab finance	Lab pricing, bills, statements and payables.
43	Workflow engine	Configurable forms and stages behind lab orders.
44	Notifications	In-app notifications and their rules.
45	Subscriptions and onboarding	Sign-up, trial, plans and subscription status.

The table gives the whole module list in chapter order. The roles that use these modules are described in Chapter 2, and a sample clinic day that touches most of them is told in Chapter 3. Chapters 4 to 27 explain the architecture, tenancy and data model that every module relies on.

1.8 Limitations and known gaps

- The product name is not final. The Platform is the name used in this handbook, and each clinic sets its own display name; the default is "Clinic Portal".
- Non-dental specialties have tags but no specialty-specific screens.
- Everything that reaches the patient directly is missing: no portal, no messages, no online booking.

Chapter 2 — Roles: who uses the Platform

This chapter lists every kind of person who touches the Platform, from the super admin who runs the whole system to the waiting-room TV that needs no login. It is written for the product owner and answers three questions for each audience: what they do day to day, which sidebar entries they use, and what they cannot do. The detailed permission model is in Chapter 11, and the access mechanics are in Chapter 57.

2.1 The audiences at a glance

The Platform checks **permissions**, never role names (Chapter 11). A **role** is only a named bundle of permissions. The audiences in this chapter are therefore described by what their default bundle allows.

Audience	Has a role?	Home page	Sidebar entries (labels)	Cannot do
Super admin	No, platform-wide	Dashboard	Organizations, Subscriptions, Lab Directory, Config Studio (Form schemas, Workflows, Stage templates), plus every clinic entry	Nothing is blocked inside a clinic.
Owner	No, owns one organization	Dashboard	Every clinic entry, including Locations, User Admin and the four "Clinic setup" pages	Platform-only pages: Organizations, Subscriptions, Lab Directory, Config Studio. Cannot enable a feature the super admin has not entitled.
Admin (system role)	Yes	Dashboard	Most clinic entries	Doctor acts: conduct consultations, write prescriptions, own queue and own earnings.
Doctor (system role)	Yes	My Queue	My Queue, Patients (own), Appointments (own), Prescriptions, Lab orders, My earnings, Clinic overview (the page refuses the Doctor role's own scope; see Chapter 36)	Billing, payments, employees, payroll, administration.
Receptionist (system role)	Yes	Appointments	Patients, Appointments, Billing desk, Invoices, Payments, Lab orders	Clinical records, consultation, refunds, voiding invoices, reports.
Assistant (system role)	Yes	Appointments	Patients, Appointments, Lab orders	Billing, prescriptions, consultation.
Accountant (system role)	Yes	Billing desk	Billing desk, Invoices, Payments, Expenses, Reports, Finance overview, Doctor earnings, Billing activity, Lab billing, Lab payables	Patient records beyond prices, queue, booking.
Custom role	Yes, built by the owner	Set per role	Whatever the permissions allow	Anything the owner did not tick; an owner cannot grant what they do not hold.
Lab admin	Lab membership	Lab portal	Lab portal	Anything inside a clinic.
Lab technician	Lab membership	Lab portal	Lab portal	Team management, accepting or rejecting orders.
Magic-link contact	No login	One order page	None	Anything except the one order the link opens.

Audience	Has a role?	Home page	Sidebar entries (labels)	Cannot do
TV display	No login	Display screen	None	Shows no patient names or phone numbers.
Patient	None	None	None	Cannot log in **[Not implemented]**.

The table puts every audience on one page. Doctor, Receptionist, Assistant and Accountant sidebars are deduced from each template's permissions and the sidebar registry, so a clinic that renames or hides entries (Chapter 58) sees a different sidebar. The sections that follow give the day-to-day view of each audience.

Sidebar labels. The sidebar items come from a registry in the web app. Each item appears only when the user holds its permission. The default layout has the groups Clinic, Finance (with Billing and Reports sub-groups), Team, Admin and Clinic setup. Settings is pinned at the bottom for everyone.

2.2 Super admin

The super admin is the person who operates the Platform itself, not a clinic. They are exempt from the tenant checks, the feature check and the permission check.

Day to day. They create and review **organizations**, set which **features** each organization is **entitled** to (optionally with a trial end date), record subscription payments by hand in **Subscriptions**, manage the shared **Lab Directory**, and edit forms, workflows and stage templates in **Config Studio**.

Sidebar. Organizations, Subscriptions, Lab Directory, and the Config Studio group with Form schemas, Workflows and Stage templates. These five entries are marked super admin only in the registry. They also see the clinic entries when they are working inside an organization.

Cannot do. Nothing is blocked by permission. In practice a super admin has no clinic role, so clinic-facing personal screens such as a doctor's own earnings have no staff record behind them.

2.3 Owner

The **owner** is the person who owns one organization. The owner has no role record: they get every permission at its widest scope, and permissions are still dropped when their feature is not active.

Day to day. Setting up the clinic and watching it: **Locations**, **User Admin** (staff logins), and the four setup pages **Roles & permissions**, **Features** (switch features on or off within what the super admin granted), **Sidebar** and **Theme**. They can also use every clinic module.

Cannot do. The owner cannot reach the five platform-only pages, cannot enable a feature that is not entitled, and cannot create permissions that do not exist in the catalog.

2.4 The five system roles

Every organization is seeded with five **system roles**: Admin, Doctor, Receptionist, Assistant and Accountant. They are read-only templates; to change one, the owner clones it into a custom role.

Admin. Runs the clinic day to day. It holds the Full bundle of every module except four clinical acts (conducting consultations, creating prescriptions, own queue, own earnings), but it can run and force-complete any doctor's queue. Home page: Dashboard. Uses: Dashboard, Clinic overview, Patients, Appointments, Billing, Reports, Employees, Payroll, Locations, User Admin and setup pages. Cannot: write prescriptions or conduct a consultation unless the owner clones Doctor for them.

Doctor. Consults patients. Home page: My Queue. A doctor sees only their own patients (scope `own`: people they have an appointment or queue entry with) and their own appointments. Day to day: start the queue, call the next patient, work in the consultation workspace, write and print prescriptions, create lab orders, check My earnings. Uses: My Queue, Patients, Appointments, Prescriptions, Lab orders, Order templates, My earnings, Clinic overview. Cannot: collect payments, see invoices, manage employees, change settings.

Receptionist. The front desk. Home page: Appointments. Day to day: register patients, book and reschedule, check patients in, add walk-ins on the reception queue board, run the TV display, collect payments, apply preset discounts. Uses: Patients, Appointments, Billing desk, Invoices, Payments, Lab orders. Cannot: read clinical notes, issue refunds, void invoices, give custom discounts, run reports.

Assistant. Chairside help. Home page: Appointments. Day to day: view patients at this location including clinical records, edit the teeth chart, book appointments, check patients in, view stock. Uses: Patients, Appointments, Lab orders. Cannot: take payments, write prescriptions, edit patient details.

Accountant. Money. Home page: Billing desk. Day to day: view and edit invoices, void them, issue refunds, close the cash counter, record expenses, see every doctor's earnings, read financial reports, manage lab billing and payables. Uses: Billing desk, Invoices, Payments, Expenses, Finance overview, Reports, Doctor earnings, Billing activity, Lab billing, Lab payables. Cannot: open patient clinical records, run the queue, book appointments.

The **home page** is where a user lands after login. If the user may not open it, the web app falls back to the Dashboard.

2.5 Custom roles

An owner (or a staff member with the roles permission) builds a **custom role** on the Roles & permissions screen. For each module the role gets None, Basic or Full, or an Advanced list of single permissions with a scope. A role also has a home page and members. Custom roles are the way to model a hygienist, a branch manager or a billing clerk. A holder of the roles permission cannot grant anything they do not hold themselves.

Figure 2.1 shows what a restricted custom role looks like on the dashboard.

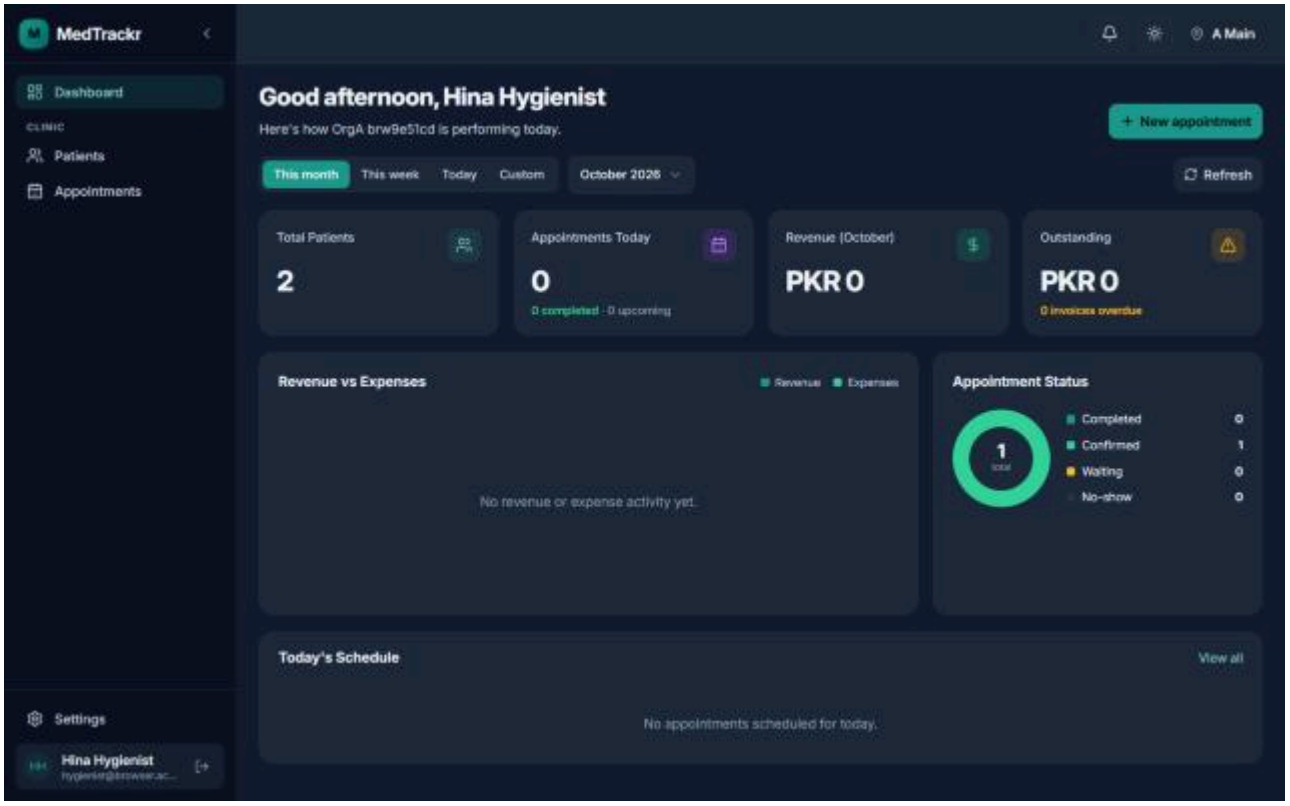


Figure 2.1 — The dashboard for a custom role with only Patients and Appointments.

The screenshot was captured on 2026-10-23 in dark theme and shows the former working name "MedTrackr" in the corner. The dashboard layout predates the redesign of that same date, so it is used only to show the sidebar. The sidebar lists Dashboard, then under the group Clinic only Patients and Appointments, with Settings and the user at the bottom. Entries for billing, reports, employees and administration are absent because the role holds none of their permissions. If this user types the address of a hidden page, the web app shows a "No access" page instead (Chapter 11).

2.6 Lab-side users

Labs are separate from clinics. A **lab** has its own members, who sign in with a normal login but hold a **lab membership** instead of a clinic assignment. They see the **Lab portal** entry in the sidebar (marked for users with at least one lab membership).

- **Lab admin.** Accepts or rejects a submitted order, asks the clinic a question (clarification), moves production steps, manages the lab team (add people, change roles, deactivate), and sees clinics, billing and profile.
- **Lab technician.** Works the production steps (for example finishing an order into quality check and passing it). Cannot accept or reject orders in the seeded dental workflow, and cannot manage the team.
- **Magic-link contact.** A person at the lab with no login. The clinic issues a **magic link** for one order. The link opens one page where the person can read the order, accept it, reject it with a reason or ask a clarification, within the link's expiry and use limit. Chapter 41 covers the lab portal in full.

These users cannot see anything inside a clinic apart from what the order and the clinic-lab link show them. The clinic decides how much patient identity the lab sees: full name or a code only.

2.7 Public and unauthenticated audiences

- **The TV display.** A screen in the waiting room opens the display page for a location. It shows, per doctor, the token being served and the next few tokens. It needs no login and by design shows no patient names or phones. Managing the TV display from a screen is **[Not implemented]**: there is no admin page for it, and the receptionist role only holds the permission to manage it.
- **Visitors before login.** Login, register, forgot password, reset password and verify email pages are public.
- **Patients.** There is **no patient portal [Not implemented]**. Patients appear in the Platform only as records that staff create and as tokens on a ticket.

2.8 Limitations and known gaps

- Sidebar labels per role in the table were deduced from permissions and the registry, not observed in a running app.
- A staff member with no assignment at the active location gets no role permissions there, even if they hold a role elsewhere.
- Super admins have no staff record, so screens built on the employee record do not apply to them.

Chapter 3 — A day in the clinic

This chapter tells the story of one working day at a clinic that uses the Platform, from opening the queue in the morning to the jobs that run overnight. It is written for the product owner, so that the modules described one by one in later chapters can be seen working together. Every person in the story is fictional, and every step names the module, its status and the chapter that explains it.

The cast. Receptionist R1 works the front desk. Dr. Example One is the doctor. Sample Patient A is a patient. The Accountant closes the cash. Example Dental Lab is an outside lab. The clinic is Example Clinic, with a Main location.

Figure 3.1 summarises the whole day before the narrative starts.

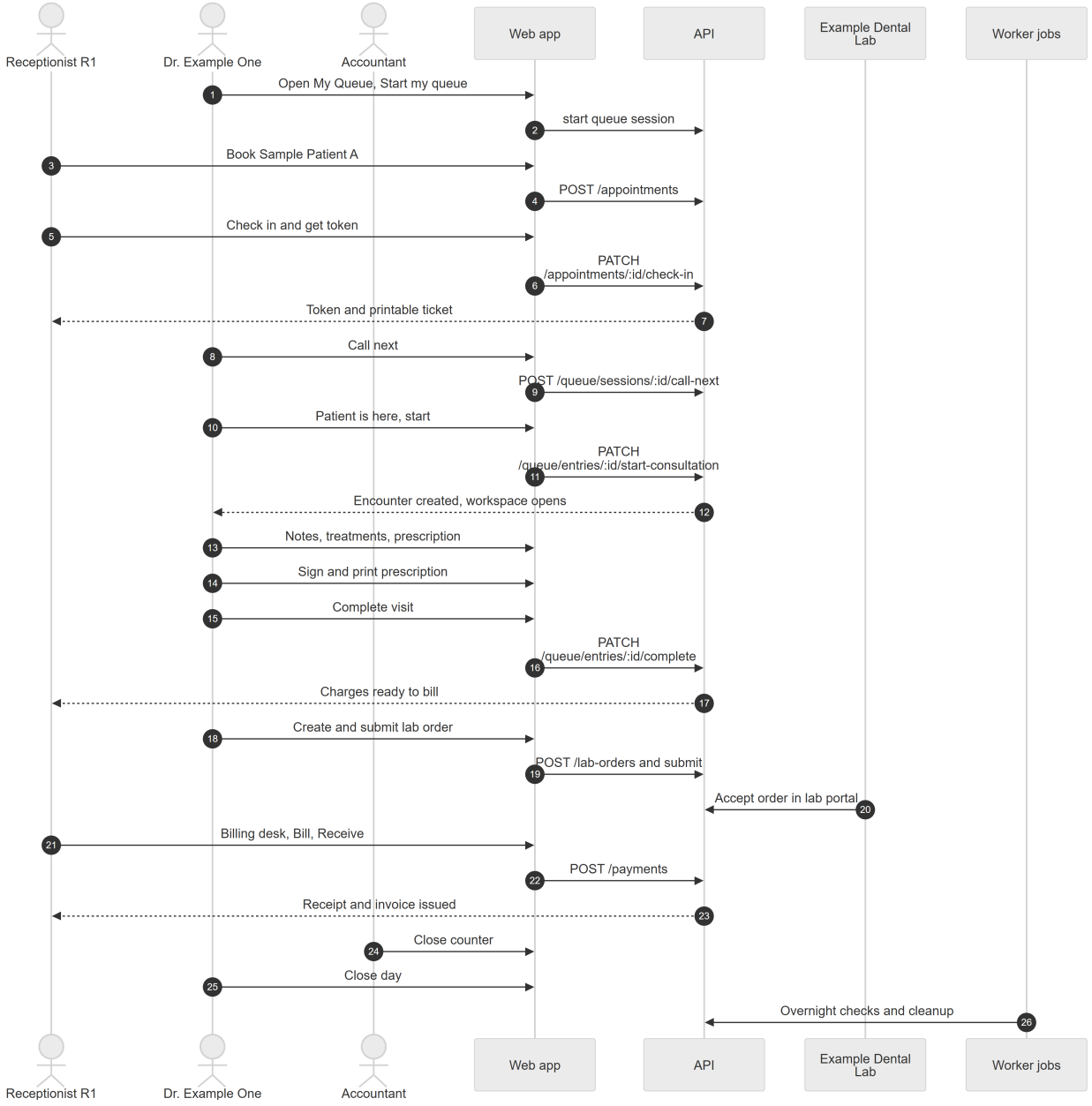


Figure 3.1 — One clinic day from opening the queue to the overnight jobs.

The sequence diagram has seven participants from left to right: Receptionist R1, Dr. Example One, the Accountant, the Web app, the API, Example Dental Lab and Worker jobs. It has 26 numbered messages. In messages 1 and 2 the doctor opens My Queue and starts the queue, and the web app calls the API. Messages 3

and 4 show R1 booking Sample Patient A with a call to create an appointment. Messages 5 to 7 show the check-in: R1 clicks Check in and get token, the web app sends the check-in call, and the API returns a token and a printable ticket to R1. Messages 8 and 9 are the doctor's Call next. Messages 10 to 12 are the doctor starting the consultation, after which the API creates the encounter and the workspace opens. Messages 13 and 14 are notes, treatments and the signed, printed prescription. Messages 15 to 17 are Complete visit, after which charges become ready to bill for R1. Messages 18 to 20 are the lab order: the doctor creates and submits it, and the lab accepts it in the lab portal. Messages 21 to 23 are R1 using the Billing desk to receive the payment, after which the API returns a receipt and issues the invoice. Message 24 is the Accountant closing the counter, message 25 is the doctor closing the day, and message 26 is the worker jobs running overnight checks and cleanup.

3.1 Opening: sign in and start the queue

Receptionist R1 signs in. If R1 may work at more than one location, the app first asks which location to use (Chapter 8, the multi-tenant model). R1's role is Receptionist, so the home page is **Appointments** (Chapter 11, roles and permissions).

Dr. Example One signs in and lands on **My Queue**, the Doctor home page. Opening that page creates the doctor's **queue session** for today if it does not exist yet. A queue session is one doctor's waiting line for one clinic day at one location, and the clinic day follows the Main location's timezone, not the server's. The doctor clicks **Start my queue**. A new session begins in the state "Doctor not started": reception can already add patients, but nobody can be called until the doctor starts. The Queue chapter (Chapter 30) describes the states Doctor not started, Open and Paused.

Status: **[Implemented]**. Module: Queue.

3.2 A booking is made

Earlier, R1 opened **Appointments** and booked Sample Patient A with Dr. Example One. The booking form checks the chosen time slot live and suggests another time when the slot is taken. The booking is saved as an **appointment**. Booking is only about reserving a time; it does not create a queue entry. The Appointments chapter (Chapter 29) covers booking, rescheduling and the status changes of a booking. Reminders to the patient and online booking are **[Not implemented]**.

Status: **[Implemented]**. Module: Appointments.

3.3 Arrival: check-in and token

Sample Patient A arrives. R1 finds the booking and clicks **Check in**, then **Check in & get token**. The Queue chapter (Chapter 30) explains that check-in is the moment the patient joins the doctor's line. The patient gets a **token** that belongs to Dr. Example One's queue for the day. A token is the short number printed on the ticket, such as "E-001": a letter prefix that comes from the doctor's name (or a prefix set on the doctor's assignment), then a three-digit counter. R1 clicks **Print ticket** in the success message, and the browser prints a small slip with the token, the doctor and roughly how many people are ahead.

If the patient has unpaid dues, a red "Rs … due" badge appears next to the patient in the queue, so R1 can ask about it (Billing, Chapter 34, feature "Show dues at check-in"). A person who arrives without a booking is a **walk-in**: R1 clicks **Walk-in** on the reception queue board, picks the doctor, and the patient gets a token the same way.

In the waiting room, the TV display shows the token being served and the next few tokens for each doctor. It needs no login and shows no patient names. The Platform pushes every change to the reception board, the doctor's screen and the TV live, through a realtime connection (Chapter 63).

Status: **[Implemented]**. Modules: Appointments (check-in) and Queue (token, board, TV).

3.4 The doctor's queue

Dr. Example One is on **My Queue**. The doctor clicks **Call next**. The button shows who will be called and why, for example "(E-001 · Appointment due)". The Queue chapter (Chapter 30) explains that a rule engine picks the next patient, either by smart rules or by first come, first served, depending on the clinic's setting (urgent patients go first in both modes). The called token flashes on the TV. When the patient walks in, the doctor clicks **Patient is here — start**.

If a called patient does not come, the receptionist or doctor can skip them, and they can rejoin later. A patient can also be sent for a test and return, or be transferred to another doctor. These actions are described in Chapter 30.

Status: **[Implemented]**. Module: Queue.

3.5 The consultation workspace

Starting the consultation creates the **encounter**, the clinical record of this one visit, and opens the **consultation workspace** at the consultation page. The Consultation chapter (Chapter 31) explains that the clinical record begins only when the doctor really starts, never at booking or check-in. In the workspace the doctor sees the patient at a glance, with allergy and medical alerts built from free-text fields (a structured allergy list is **[Partial]**), writes visit notes that auto-save, and records the **treatments** done today. The doctor can open the patient record without leaving the page.

The doctor does not see treatment prices unless the doctor's role holds the price permission; the default Doctor role does not.

Status: **[Implemented]**. Module: Consultation.

3.6 Prescription and printing

In the same workspace the **Prescription** panel lets the doctor search a medicine, set dose, frequency and duration, or choose a preset. The Prescriptions chapter (Chapter 32) explains the full module. The doctor clicks **Sign & print**. The Platform signs the prescription and shows a PDF preview on the clinic's print layout, with a **Print** button. Urdu text typed by the doctor is printed in Nastaliq script, right to left; the labels stay in English.

Status: **[Implemented]**. Module: Prescriptions.

3.7 Completing the visit

The doctor clicks **Complete**. The toast says "Visit completed" and the doctor returns to My Queue. At the same moment, in the same database transaction, the booked appointment becomes completed and the Platform creates **charges** for the visit: a consultation charge, plus one charge per treatment recorded in the encounter, priced from the clinic's price list. This happens only when the organization has the feature `billing.invoices` enabled. The Consultation chapter (Chapter 31) and the Billing chapter (Chapter 34) describe the automatic charges. No invoice exists yet: the charges wait on the Billing desk.

Status: **[Implemented]**. Modules: Queue (complete), Consultation, Billing (automatic charges).

3.8 A lab order

Sample Patient A needs a crown, so Dr. Example One orders it from Example Dental Lab. The doctor opens **Lab orders**, creates a **lab order** with an item (for example a zirconia crown), and submits it, so its status becomes Submitted. The Dental lab chapter (Chapter 40) covers the clinic side. In this version a lab order is **not** created from inside the consultation workspace and is not linked to an encounter **[Not implemented]**, so the doctor orders it from the separate Lab orders page.

The lab sees the order in its own **lab portal** (Chapter 41). A lab admin there can accept it with an expected date, reject it with a reason or ask a question. Instead of an account, the lab can also receive a **magic link** for the single order: a link that opens one page where the person can accept, reject or ask a clarification without logging in. The clinic can copy that link. The code can also queue the link for delivery through the notification outbox when the lab has a contact number or email (Chapter 44); whether a WhatsApp message actually leaves the Platform is **[Unverified]**, since notifications are in-app only. When the lab accepts, the order moves on, and its later stages are tracked through the workflow engine (Chapter 43).

Status: **[Implemented]**, except that a lab order has no link to the consultation. Modules: Dental lab, Lab portal.

3.9 Collecting payment and the invoice

Sample Patient A goes to the front desk. R1 opens the **Billing desk**. The first section, **Ready to bill**, lists Sample Patient A with the count of charges and the unbilled amount. R1 clicks **Bill**, and the **Collect payment** sheet slides in. R1 can give a discount (the Receptionist role holds the permission for preset discount types only), confirm the amount, choose the **Received in** payment account (for example cash) and click **Receive Rs ...**.

The Billing chapter (Chapter 34) explains that the **invoice is issued at this moment**: when payment is collected for charges that have no invoice yet, the Platform numbers and issues the invoice in the same transaction as the receipt. The result shows the receipt number and the invoice number, with **Print receipt** and **Print invoice** buttons. Exclusive tax, if configured, is added on top of the price. If the patient pays more than the charges, the extra money is held as **advance**. The same transaction records the doctor's **commission entries** (Chapter 35), which the doctor later sees under My earnings.

Status: **[Implemented]**. Module: Billing.

3.10 Cash close

At the end of the shift the cash is closed by someone who holds the cash-close permission. The default Receptionist role does not hold it; the Accountant and Admin roles do. In this story the Accountant opens **Payments** and clicks **Close counter**. The Platform shows the expected amount (payments minus refunds since the previous close). For a cash account the Accountant types the counted amount, adds a note if there is a difference, and clicks the close button. The close is an immutable record with a printable slip, and a difference above the alert amount shows a "Flagged" badge. Each close starts where the previous one ended, so no payment is counted twice. Closing never blocks taking payments (Billing, Chapter 34).

Status: **[Implemented]**. Module: Billing.

3.11 Closing the day

The doctor clicks **Close day** on My Queue. The button is disabled while a patient is still with the doctor. A dialog lists every patient still waiting, skipped or at a test, and for each one the doctor chooses to cancel them or transfer them to another doctor. When the list is settled the session closes and no more tokens are issued

(Chapter 30). A session that is left open with patients still waiting is not cancelled automatically; the system logs an alert for an admin.

Status: **[Implemented]**. Module: Queue.

3.12 Overnight: jobs that run by themselves

When the clinic is closed, scheduled jobs run in the one **worker** process (Appendix E lists them and Chapter 6 explains the process roles). Their schedules are in server time, not in the clinic's timezone. The ones that matter to this day:

Job	When	What it does for the clinic
Queue session day-end	Every 5 minutes	Closes the previous day's queue sessions that are empty. Sessions with waiting patients stay open and log an alert.
Patient credit expiry	Every day at 01:00	Marks prepaid patient credits past their expiry date as expired.
Billing invariant check	Every day at 02:30	Recomputes billing totals, checks the money rules and logs any break. It only logs.
Queue cleanup	Every day at 03:15	Deletes old idempotency keys and published realtime events.
Inventory alerts	Every day at 08:00	Counts low-stock and soon-expiring items and writes to the server log. Nobody is notified **[Partial]**.
Month-end payroll	05:00 on the 1st of each month	Generates payroll records for the month that ended (Chapter 38).
Queue no-show marker	Every 60 seconds, all day	Marks a booked patient as a no-show once they are too late to check in.

The table shows that the Platform does its housekeeping without any user clicking anything. Realtime delivery of queue changes and in-app notifications also runs continuously in the worker, as loops rather than scheduled jobs (Chapter 63).

Status: **[Implemented]**, except inventory alerts, which are **[Partial]**.

3.13 What the day did not include

Some things a reader may expect are not built. The patient received no reminder message, and cannot book online or see anything in a portal. Notifications reach only logged-in staff, in the app. Insurance claims and online payments are not built. These gaps are listed in Chapter 1.

3.14 Limitations and known gaps

- This chapter follows the default flow. A clinic with fewer features enabled, for example without `billing.invoices`, would not get the automatic charges at the end of the visit.
- The ticket prefix example "E-001" is deduced from the token rule (first letter of the doctor's name unless a prefix is set); the actual prefix depends on clinic data.
- The lab order stages after acceptance are described in Chapters 40 to 43, not here.

Chapter 4 — Repository and tech stack

This chapter explains how the source code of the Platform is organised and which technologies it is built on. It is written for the product owner, who is a software engineer and needs a map of the code before reading any module chapter. Everything here was read from the repository itself: the folder tree, the two `package.json` files and the module registrations in the backend.

4.1 One repository, two applications

Status: [Implemented]

The Platform lives in a single Git repository (a "monorepo"). It holds two applications that are built and run separately:

- **`backend/`** — the API server. It is a NestJS application written in TypeScript. It talks to a PostgreSQL database through TypeORM, and to Redis for cross-process messages.
- **`web-app/`** — the web application that clinic staff, owners and super admins use in the browser. It is a React 19 application built with TanStack Start and Vite.

Next to them, `docs/` holds feature documents, test plans and this handbook. The backend also has its own `backend/docs/` folder with the generated entity catalog (`backend/docs/data-model/ENTITY_CATALOG.md`) and one generated module chapter.

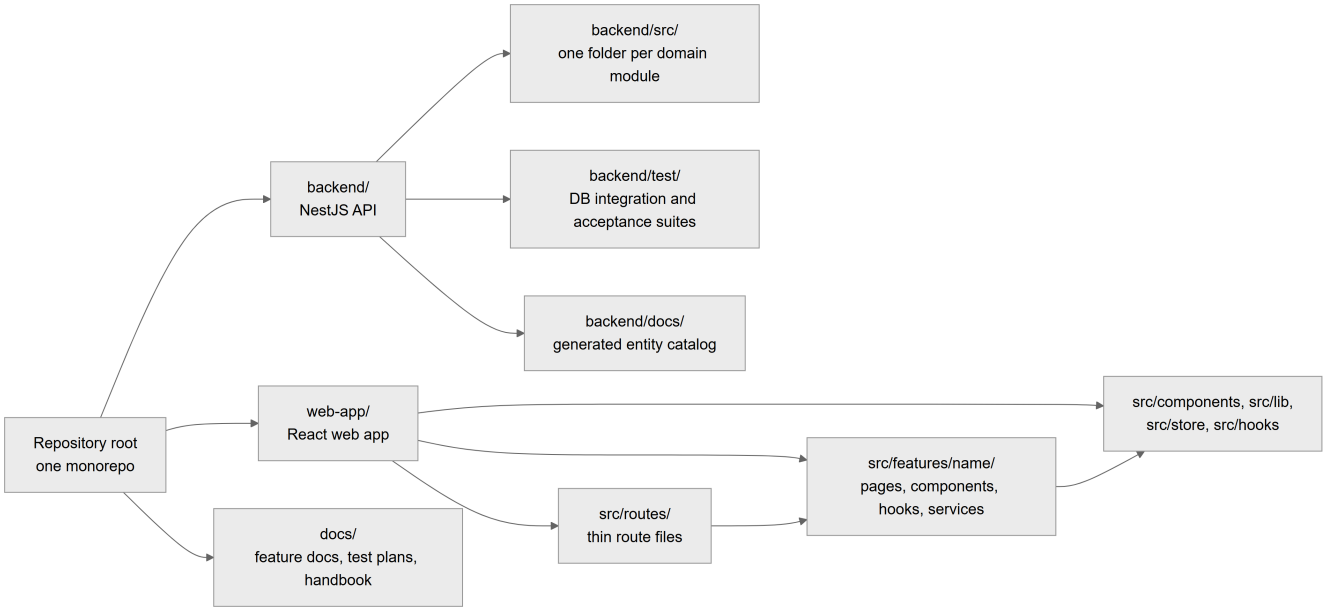


Figure 4.1 — The repository layout at the top level, and the main folders inside each application.

The figure reads from left to right. A single box, "Repository root, one monorepo", has three arrows to its three main folders: `backend/` (the NestJS API), `web-app/` (the React web app) and `docs/` (feature docs, test plans and the handbook). The backend box branches into three folders: `backend/src/` with one folder per domain module, `backend/test/` with the database integration and acceptance suites, and `backend/docs/` with the generated entity catalog. The web-app box branches into `src/routes/` (thin route files), `src/features/<name>/` (pages, components, hooks and services) and the shared folders `src/components` , `src/lib` , `src/store` and `src/hooks` . Two further arrows show the direction of dependencies in the web app: route files point to feature folders, and feature folders point to the shared folders.

There is no workspace tool joining the two applications (no root `package.json`). Each one has its own `package.json`, its own `node_modules` and its own commands.

Technical references: `backend/package.json` · `web-app/package.json` · `README.md` (root)

4.2 The web application's real folder layout

Status: [Implemented]

The web app keeps business logic out of route files. A route file only declares the URL, an optional permission requirement and which feature page to render. The real layout under `web-app/src/` is:

Folder	What it holds
`routes/`	92 file-based route files for TanStack Router. Each one wraps a feature page in `DashboardLayout` and may declare `staticData: { can: … }`, the permission needed to open it (54 routes do). `__root.tsx` is the root route: it restores the session, redirects, and mounts the access providers.
`features/<name>/`	One folder per feature (about 30, for example `patients`, `appointments`, `queue`, `billing`, `roles`, `access`). Inside: `pages/`, `components/`, `hooks/` (TanStack Query and form hooks), `services/` (axios calls), and often `schemas/` (Zod), `types`, `lib/` and `utils/`. A feature's `index.ts` is its public barrel.
`components/`	Shared UI: `ui/` (shadcn/ui primitives), `layout/` (`DashboardLayout`, `Sidebar`, `Topbar`, `LocationSwitcher`), `shared/` (`DataTable`, `ConfirmDialog`, `TooltipButton`, `BackButton`, `DatePicker` and more), `feedback/` (`EmptyState`, `ErrorState`, `LoadingSkeleton`, `NoResultsState`), `forms/`, `data-display/`, `feature/`.
`lib/`	`api.ts` (the configured axios instance), `query-client.ts` (TanStack Query defaults), `currency.ts`, `formatters.ts`, `utils.ts`.
`store/`	Zustand stores: `useAuthStore.ts` (token, active organization and location, kept in memory only), plus small UI stores for patients, appointments, employees, navigation history and general UI.
`hooks/`	Cross-feature hooks such as `useBackNavigation`, `useDebounce`, `useTheme`.
`context/`	`FeatureFlagsContext.tsx` (the active features and the organization's specialty tags).
`test/`	Vitest setup.

The table lists each top-level folder of `web-app/src/` and its role. The pattern to remember is: a route file is thin, a feature folder owns its screens and its API calls, and anything used by many features lives in `components/`, `lib/`, `store/` or `hooks/`.

Where the CLAUDE.md conventions file differs. The root `CLAUDE.md` describes a different, planned layout: route files under `src/app/`, shared code under `src/shared/`, and an `api/` folder in each feature. None of those folders exist. The code uses `src/routes/`, `src/components/`, `src/lib/`, `src/store/` and feature `services/` folders, as the table shows. Some feature folders also deviate from the planned shape (for example `organizations/` and `locations/` keep their views at the folder root, and a few components call the axios instance directly). This handbook always describes the real layout.

Technical references: `web-app/src/routes/__root.tsx` · `web-app/src/features/` · `web-app/src/lib/api.ts` · `web-app/src/store/useAuthStore.ts`

4.3 Backend module map

Status: [Implemented]

The backend is a "modular monolith": one NestJS application whose code is split into modules, one folder per business area under `backend/src/`. All modules are loaded by `AppModule` (`backend/src/app.module.ts`). Every folder is listed here with one line.

Folder	One-line purpose
`access`	Permission catalog (70 permissions), bundles, the 5 system role templates, `AccessResolver`, `AccessGuard`, `GET /me/access`, sidebar layouts (`/admin/navigation`).
`appointments`	Booking: scheduling, validation, booking rules, doctor double-booking protection, appointment status changes.
`auth`	Sign-in, tokens, refresh, sessions endpoints, password reset, email verification, rate limiting, the JWT and tenant guards, `AuthorizationService`. OAuth code exists but is not registered.
`billing`	Charges, invoices, payments, allocations, refunds, discounts, expenses, cash close, ledgers, credit packages, doctor commissions.
`bootstrap`	`http.ts`: starts an HTTP process (body limits, validation pipe, exception filter, CORS, cookies).
`chairs`	One entity file only. No module registers it, so it does nothing at runtime.
`common`	Shared building blocks: audit, clock (`ClockService`), database helpers and migrations runner, decorators, `ResponseWrapper`, exception filter, idempotency, Redis connection helper, number sequences, file storage, request context.
`employees`	The employee person record, assignments, employment roles (including the `/admin/roles` screen API), schedules, leave, doctor availability overrides.
`encounters`	Patient encounters: the clinical record of one consultation.
`engine`	Generic workflow engine: specialties, versioned form schemas, versioned workflows, stage templates (Config Studio).
`feature-flags`	Feature catalog, entitlement, owner switches, location overrides, `FeatureFlagGuard`.
`inventory`	Items, suppliers, purchase orders, stock movements, consumption, low-stock job.
`lab-access`	A lab's answers to orders through the lab portal, magic links or a clinic proxy; work-order QR.
`lab-billing`	Billing the patient for lab work.
`lab-catalog`	Lab services, price windows, clinic-to-lab service mappings, order templates.
`lab-finance`	Glue between lab billing, lab payables, commissions and the journals (no endpoints of its own).
`lab-orders`	Lab orders: drafts, submit, revisions, messages, attachments, work-order print.
`lab-payables`	Vendor bills from labs, lab payments, statements, payables reports.
`lab-production`	Production stages, technician assignments, shipments, remakes, warranties.
`labs`	Platform lab directory, lab staff, each clinic's lab list, lab portal foundation.
`locations`	Locations (branches) and location settings.
`migrations`	Three migration files; the first is a legacy baseline that never runs.
`notifications`	In-app notification rules, templates, outbox, quiet hours, the sending loop.

Folder	One-line purpose
`organizations`	Organizations (clinics), ownership, legacy organization settings.
`patients`	The patient record and its sub-records (vitals, dental info, teeth chart, documents, consents, recalls, referrals, insurance, feedback, plans), import and export, patient audit trail.
`payroll`	Monthly payroll records and the month-end payroll job.
`plan-templates`	Treatment plan templates.
`prescriptions`	Prescriptions, medicine directory, templates, presets, doctor signatures, print layouts.
`queue`	Doctor-wise queue: sessions, entries, tokens, `QueueOrchestrator`, selection engine, policies, Socket.IO gateway, outbox relay, queue jobs.
`reports`	Finance dashboard, clinic overview and 37 report definitions with Excel and PDF export (owns no tables).
`session`	User session records (the refresh-token sessions).
`settings-registry`	`config_values` cascade (platform, organization, location) used by the theme and newer settings.
`subscriptions`	Plans, trial, grace, read-only mode, manual subscription payments, self-signup and the setup wizard, `SubscriptionGuard`.
`treatments`	Treatments: clinical work performed.
`user`	User accounts (logins), user admin, staff logins, owner creation.

The table names every folder in `backend/src/` and what it is responsible for. Besides the folders, `backend/src/` holds the entry points `main.ts`, `main.api.ts` and `main.worker.ts` (one per process role, explained in Chapter 6), `app.module.ts`, a placeholder `app.controller.ts` / `app.service.ts` (a hello endpoint) and `data-sourse.ts`, the TypeORM command-line data source (the misspelling is the real file name).

Note on CLAUDE.md. `CLAUDE.md` describes inventory, invoices, reports and notifications as stubs. In the code all four are built. Only `chairs` is an entity with no module.

Technical references: `backend/src/app.module.ts` · `AppModule` · `backend/src/*/*.module.ts`

4.4 Backend technology

Status: [Implemented] (with the exceptions noted in the table)

Package (major version)	Purpose in the Platform
NestJS 11 (`@nestjs/core`, `common`, `platform-express`)	The application framework: modules, controllers, dependency injection, guards, pipes, interceptors, running on Express.
`@nestjs/typeorm` 11 + TypeORM 0.3 + `pg` 8	Database access to PostgreSQL: entities, repositories, transactions, schema synchronize, migrations. `pg` is also used directly for `LISTEN` connections.
`@nestjs/config` 4	Reads environment variables.
`@nestjs/jwt` 11, `@nestjs/passport` 11, `passport-jwt`, `passport-local`	Access tokens and the JWT strategy behind `JwtAuthGuard`.
`passport-google-oauth20`, `passport-github2`	OAuth strategies. Present but not registered, so OAuth sign-in is off [Not implemented].

Package (major version)	Purpose in the Platform
`bcrypt` 6	Password hashing.
`cookie-parser`	Reads the httpOnly refresh-token cookie.
`class-validator` , `class-transformer`	DTO validation and conversion in the global `ValidationPipe` .
`@nestjs/websockets` 11, `@nestjs/platform-socket.io` , `socket.io` 4	The realtime gateway at `/realtime` .
`ioredis` 5	Redis pub/sub for realtime events and cache invalidation across processes.
`@nestjs/bull` 11 + `bull` 4	A Bull queue named `submissions` is registered, but no code processes or adds jobs to it [Partial: registered, unused].
`@nestjs/schedule` 6	Scheduled jobs (`@Cron` , `@Interval`), loaded only in processes that run jobs.
`@nestjs/cache-manager` , `cache-manager`	Listed as dependencies; no source file imports them [Not implemented].
`@nestjs/mongoose` , `mongoose`	Listed as dependencies; no source file imports them [Not implemented].
`@nestjs/swagger`	Listed; no source file imports it, so there is no generated API documentation [Not implemented].
`cloudinary` 2, `multer` 2	File uploads (logos, documents, attachments).
`nodemailer` 7	Outgoing email (verification and password reset).
`exceljs` 4	Excel export of reports.
`pdf-lib` , `pdfmake` , `fontkit` , `@fontsource/noto-nastaliq-urdu` , `@fontsource/noto-naskh-arabic`	PDF generation, including Urdu text.
`axios` , `slugify` , `useragent` , `rxjs` , `reflect-metadata`	HTTP client, slug generation, user-agent parsing for sessions, and NestJS runtime needs.

The table lists the backend's runtime dependencies by name and what each is for. Development dependencies are the usual NestJS set: the Nest CLI, TypeScript 5, Jest 29 with `ts-jest` , Supertest, `socket.io-client` (for tests), ESLint 9 and Prettier 3.

Technical references: `backend/package.json` · `backend/src/app.module.ts`

4.5 Web application technology

Status: [Implemented]

Package (major version)	Purpose in the Platform
React 19, `react-dom` 19	The UI library.
TanStack Start 1 (`@tanstack/react-start`) with Vite 7 and the Nitro plugin	The application framework and build. Vite builds the app; the TanStack Start and Nitro plugins produce the server bundle that serves it.
TanStack Router 1 (`@tanstack/react-router` , `router-plugin`)	File-based routing from `src/routes/` ; the generated route tree is `routeTree.gen.ts` .
TanStack Query 5	Server-state cache: queries, mutations, invalidation.
TanStack Form 1	Forms.
TanStack Table 8	Tables in list views.

Package (major version)	Purpose in the Platform
Zustand 5	Client state: the auth store and small UI stores.
Zod 4	Form schemas.
axios 1	HTTP client with interceptors (`src/lib/api.ts`).
`socket.io-client` 4	Realtime connection to `/realtime` .
Tailwind CSS 4, shadcn/ui (Radix UI primitives), `class-variance-authority` , `clsx` , `tailwind-merge` , `lucide-react`	Styling, components and icons.
FullCalendar 6	Appointment calendar views.
Recharts 3	Charts.
`@dnd-kit`	Drag and drop (for example the sidebar builder).
`date-fns` 4, `react-day-picker`	Dates and the date picker.
`sonner`	Toast messages.

The table lists the web app's runtime dependencies and their roles. For tests the web app uses Vitest 3 with Testing Library and jsdom; formatting and linting use Prettier and ESLint.

Technical references: `web-app/package.json` · `web-app/vite.config.ts` · `web-app/vitest.config.ts`

4.6 What the repository does not contain

Status: [Not implemented]

There is no Dockerfile, no container compose file, no CI pipeline configuration (no `.github/` folder) and no hosting configuration in the repository. How the Platform is deployed is therefore not described by code. Chapter 6 explains what the code expects from a deployment.

Technical references: repository root listing

Chapter 5 — Request lifecycle: from a click to the database and back

This chapter follows one real request through the whole Platform: from a button in the web app, through the API's security checks, into a database transaction, and back to the screen. It is written for the product owner who wants to know exactly which layers every request passes and where an error can stop it. A second part follows a realtime event from a database commit to every open screen.

5.1 The example: registering a patient

Status: [Implemented]

The traced request is the one a receptionist makes when registering a new patient. On the Patients page the receptionist clicks **Create**, fills the multi-step **New Patient** form and clicks **Add Patient** on the last step. The web app sends `POST /patients`, the API stores the patient and an audit record, and the web app shows the toast "Patient created" and offers to book a first appointment.

This request is a good example because it touches every common layer: the token and tenant headers, all five global guards, DTO validation, a permission check (`patients.create`), a database transaction with an audit row, the response envelope and the cache refresh in the web app. It does not use realtime events; section 5.8 covers those with a queue action instead.

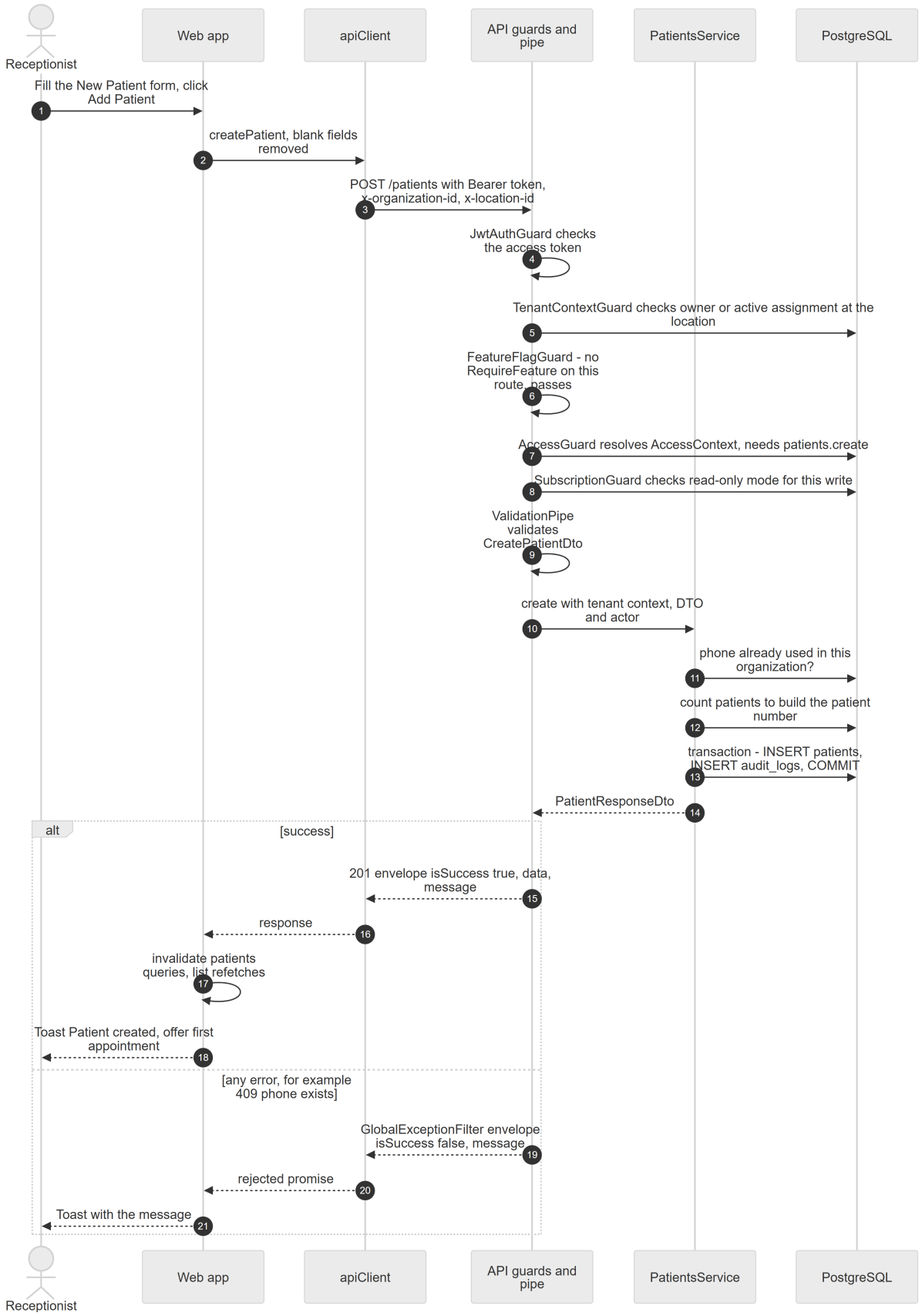


Figure 5.1 — Sequence of one write request, `POST /patients`, from the receptionist's click to the toast.

The diagram has six participants from left to right: the Receptionist, the Web app, the `apiClient` axios instance, the API's guards and validation pipe, `PatientsService`, and PostgreSQL. (1) The receptionist fills the New Patient form and clicks Add Patient. (2) The web app calls `createPatient`, which first removes blank optional fields. (3) The axios instance sends `POST /patients` with the Bearer token and the `x-organization-id` and `x-location-id`

headers. (4) `JwtAuthGuard` checks the access token. (5) `TenantContextGuard` checks in the database that the user owns the organization or has an active assignment at that location. (6) `FeatureFlagGuard` finds no `@RequireFeature` on this route and lets it pass. (7) `AccessGuard` resolves the user's access context and requires `patients.create`. (8) `SubscriptionGuard` checks that the clinic is not in read-only mode, because this is a write. (9) The `ValidationPipe` validates the body against `CreatePatientDto`. (10) The controller calls `PatientsService.create` with the tenant context, the DTO and the actor. (11) The service checks that the phone number is not already used in the organization, (12) counts existing patients to build the next patient number, and (13) inserts the patient and an `audit_logs` row in one transaction and commits. (14) It returns a `PatientResponseDto`. An "alt" box then shows two outcomes. On success the API returns a 201 response whose body is the envelope with `isSuccess: true`, `data` and `message`; the web app invalidates its cached patient lists, which refetch, and shows the toast "Patient created" with the option to book a first appointment. On any error, for example 409 because the phone exists, the `GlobalExceptionFilter` returns the envelope with `isSuccess: false` and a message; the axios promise rejects and the web app shows the message in a toast.

Technical references: `web-app/src/features/patients/components/CreatePatientModal.tsx` · `useCreatePatient` · `backend/src/patients/patients.controller.ts` · `PatientsService.create`

5.2 In the browser: hook, service and the axios instance

Status: [Implemented]

In simple terms. The screen never talks to the server directly. It asks a small "mutation hook" to do the work; the hook calls a service function; the service function uses one shared HTTP client that adds the user's credentials and the clinic and branch the user is working in.

Technically.

- The form component calls `useCreatePatient().mutateAsync(data)`. This is a TanStack Query mutation whose `mutationFn` is `createPatient` from `features/patients/services/patients.api.ts`.
- `createPatient` drops fields that are empty strings (the API would reject an empty `dob` or `email`) and calls `api.post('/patients', body)`. It returns `data.data`, the payload inside the response envelope.
- `api` is the axios instance `apiClient` in `web-app/src/lib/api.ts`:
  - Base URL: the `VITE_API_BASE_URL` build variable, default `http://localhost:5000`.
  - Timeout: 15 seconds. `withCredentials: true`, so the browser sends the refresh-token cookie.
- **Request interceptor.** Before every request it reads the Zustand auth store (`useAuthStore`), which lives in memory only and is never written to browser storage:
  - adds `Authorization: Bearer <access token>` when a token exists;
  - for lab portal URLs (`/lab-portal/...`) adds only `x-lab-id` and no clinic headers;
  - otherwise adds `x-organization-id` (the active organization) and `x-location-id` (the active location).
- **Response interceptor.** Described with the error path in section 5.7.

TanStack Query is configured once in `web-app/src/lib/query-client.ts`: queries are fresh for 5 minutes, are kept for 10 minutes, retry twice and do not refetch when the window regains focus; mutations retry once. Individual hooks may override these values (the access query, for example, uses 30 seconds and refetches on focus).

Technical references: `web-app/src/lib/api.ts` · `apiClient` · `web-app/src/lib/query-client.ts` · `web-app/src/store/useAuthStore.ts`

5.3 Arriving at the API: process start-up and Express middleware

Status: [Implemented]

An HTTP process is started by `bootstrapHttp(role)` in `backend/src/bootstrap/http.ts` (Chapter 6 explains the roles). It configures, in this order:

1. The NestJS application with Nest's own body parser switched off.
2. JSON and URL-encoded body parsers with a **5 MB** limit (consent signatures are sent as inline images).
3. The global `ValidationPipe` (section 5.5).
4. The global exception filter `GlobalExceptionFilter`.
5. CORS for one origin, `CORS_ORIGIN` (default `http://localhost:3000`), with credentials allowed.
6. `cookie-parser`, so the refresh cookie can be read.
7. A small middleware that opens a per-request context (`RequestContextService`, based on Node's AsyncLocalStorage).
8. Listening on `PORT` (default 5000).

Two middleware classes exist in `common/middleware/` (`ContextMiddleware` and `TenantContextMiddleware`) but **no module registers them** [Not implemented]. Tenant checks are done by a guard instead.

For every request NestJS then runs its fixed pipeline: Express middleware → global guards → interceptors (on the way in) → pipes → controller method → interceptors (on the way out) → exception filter if anything threw. The next sections walk this pipeline.

Technical references: `backend/src/bootstrap/http.ts` · `bootstrapHttp` · `backend/src/common/services/request-context.service.ts`

5.4 The five global guards

Status: [Implemented]

Five guards are registered as global guards (`APP_GUARD`) in `backend/src/app.module.ts`. NestJS runs them in registration order, and the first one that refuses stops the request.

#	Guard	Question it answers	Skips when	Refusal
1	`JwtAuthGuard`	Is there a valid, unexpired access token?	The method and path match an entry in `PUBLIC_ROUTES` (login, register, refresh, password reset, email verification, logout, the public theme, the TV display state, lab magic links).	401 with a message such as "Token has expired. Please refresh your token."
2	`TenantContextGuard`	Does this user have access to the organization and location in the headers?	`@Public()`; `@ContextFree()`; super admins. `@RequireSuperAdmin()` routes are checked here instead (non-super-admins get 403 "Super admin access required").	400 "Missing tenant context. Please select an organization and location." or 403 "You do not have access to this organization/location".
3	`FeatureFlagGuard`	Is every feature named by `@RequireFeature` active for this organization and location?	No `@RequireFeature` on the route; `@Public()`; super admins.	403 'The "" feature is not enabled for this location'.

#	Guard	Question it answers	Skips when	Refusal
4	`AccessGuard`	Does the user hold every permission in `@Can(...)`, or is the user a member for `@AnyMember()`?	`@Public()`; routes with neither marker pass (but still get the access context attached when an organization is known).	403 with `{ code: 'NO_PERMISSION', missing: [...] }`.
5	`SubscriptionGuard`	If this is a write, is the clinic out of read-only mode?	Reads (`GET`, `HEAD`, `OPTIONS`); `@Public()`; super admins; paths starting `/auth/`, `/account`, `/me/`, `/admin/subscriptions`; requests with no organization.	402 with `code: 'SUBSCRIPTION_EXPIRED'`.

The table lists the guards in the order they run, what each checks, when it steps aside, and what the caller receives when it refuses. In the patient example all five apply: the route is not public, it carries `@Can('patients.create')`, and it is a write.

Details worth knowing:

- **Access check for the tenant.** `TenantContextGuard` calls `AuthorizationService.validateAccess`. An owner passes for any active location of the organization. Anyone else needs an active assignment at that exact location with portal access switched on. When it passes, the guard stores `request.tenantContext = { organizationId, locationId }`.
- **`@Public()` and the JWT guard.** `JwtAuthGuard` only reads the `PUBLIC_ROUTES` list (`auth/constants/public-routes.ts`). The `@Public()` decorator alone does not let an unauthenticated request through; a route without a token must also be listed there.
- **No path-prefix skips.** `TenantContextGuard` deliberately has no list of path prefixes that bypass it; it relies only on the decorators.
- **Caching.** The access context is cached per user, organization, location and access version (Chapter 11); the enabled-feature set for 30 seconds (Chapter 10); read-only status for 30 seconds per process.

Technical references: `backend/src/app.module.ts` · `JwtAuthGuard` · `TenantContextGuard` · `FeatureFlagGuard` · `AccessGuard` · `SubscriptionGuard` · `backend/src/auth/constants/public-routes.ts`

5.5 Validation pipe and the controller

Status: [Implemented]

Validation. One global `ValidationPipe` validates every request body, query and parameter that is typed as a DTO class with `class-validator` rules. Its settings:

Setting	Effect
`transform: true`	The plain JSON body becomes an instance of the DTO class.
`whitelist: true`	Properties without a validation decorator are removed.
`forbidNonWhitelisted: true`	Instead of silently removing unknown properties, the request is refused with 400 (for example "property id should not exist").
`enableImplicitConversion: true`	Query strings such as `page=2` become numbers when the DTO says so.

The table lists the four options and their effect. The strict `forbidNonWhitelisted` rule matters for owners: a screen that sends a whole database row back (with `id` or timestamps) is refused, which is why some older edit screens fail (the Organizations and Locations chapter, Chapter 9, lists them).

Controller. In the example, `PatientsController.create` receives:

- `@Body() dto: CreatePatientDto` — the validated body;
- `@ActiveTenant() ctx` — `{ organizationId, locationId }` from `request.tenantContext` (or the headers);
- `@RequestActor(ActorSide.CLINIC) actor` — who is acting, for the audit trail.

Other handlers also take `@Access() access`, the resolved access context, when they need scope rules (Chapter 11). Controllers contain no business rules; they call a service and wrap the result.

Technical references: `backend/src/bootstrap/http.ts` · `backend/src/common/decorators/active-tenant.decorator.ts` · `backend/src/common/audit/request-actor.decorator.ts`

5.6 Service, transaction and database

Status: [Implemented]

`PatientsService.create` shows the usual service pattern:

1. **Checks that read.** The phone must be unique inside the organization (409 "A patient with this phone number already exists in this organization"). The next patient number is built from a count of the organization's patients, including deleted ones (`P-00001` , `P-00002` , …).
2. **One transaction for the write.** `dataSource.transaction(...)` saves the `patients` row and writes the audit record through `recordPatientChange` with the same entity manager. Both rows commit together or not at all.
3. **A response DTO.** The saved entity is mapped to `PatientResponseDto` .

Other modules follow the same shape with stronger guarantees where money or queues are involved: row locks (`pessimistic_write`), optimistic version checks, idempotency keys for queue actions, and outbox rows written in the same transaction (section 5.8). All tenant data is filtered by `organization_id` in the service; data that has a scoped permission is filtered further with `applyScope` (Chapter 11).

The database is PostgreSQL. The API connects with the `DATABASE_URL` connection string and always requests an SSL connection.

Technical references: `backend/src/patients/patients.service.ts` · `PatientsService.create` · `recordPatientChange`

5.7 The response envelope and the error path

Status: [Implemented]

Success. Every controller returns `ResponseWrapper.success(data, message)` . The JSON body is always the same envelope:

Field	Meaning
`data`	The payload (here the new patient).
`message`	A human-readable message, for example "Patient created successfully".
`isSuccess`	`true` .
`httpStatusCode`	Always `200` inside a success envelope, even when the HTTP status is 201.
`error` , `devMessage`	Empty on success.

The table describes the five fields of the envelope. The web app reads `res.data.data` for the payload. On the way out, the global `VisibleIfInterceptor` removes fields marked `@VisibleIf` that the caller may not see (for example treatment prices without `patients.price.view`); patient records have no such fields.

Back in the browser, the mutation's `onSuccess` invalidates every query whose key starts with `patients`, so open patient lists refetch, and shows the toast "Patient created". The modal then switches to an optional "Book First Appointment" step.

Errors. Any exception, anywhere after the middleware, reaches `GlobalExceptionFilter`. It logs the error and answers with the HTTP status of the exception (500 for unexpected errors) and an envelope with `isSuccess: false`, `message` (the first message when there are several, such as validation errors; "Something went wrong" for unexpected errors), `error` (the exception body, for example `{ code: 'NO_PERMISSION', missing: [...] }`) and `devMessage`.

Status	Typical cause
400	Validation failed, or tenant headers missing.
401	No token, invalid token or expired token.
402	`SUBSCRIPTION_EXPIRED`: the clinic is read-only.
403	No access to the organization or location, feature not active, `NO_PERMISSION`, or super admin required.
404	Record not found in this organization.
409	Business conflict, such as a duplicate phone number.
500	Unexpected error.

The table maps the common status codes to their causes. Screens show `err.response.data.message` in a toast, or a fallback text such as "Failed to create patient".

Automatic token refresh. The access token lives 15 minutes. When any request gets **401**, the axios response interceptor:

1. ignores 401s from `/auth/login` and `/auth/refresh` themselves;
2. if a refresh is already running, parks the request in a queue;
3. otherwise calls `POST /auth/refresh` (the httpOnly cookie carries the session id and refresh token), stores the new token and context in the auth store, replays the queued requests and retries the original request once;
4. if the refresh fails, or the retried request gets 401 again, clears the auth store and sends the browser to `/login`.

Technical references: `backend/src/common/dtos/ResponseWrapper.ts` · `backend/src/common/exceptions/HttpExceptionHandler.ts` · `GlobalExceptionFilter` · `VisibleIfInterceptor` · `web-app/src/lib/api.ts`

5.8 The realtime path: from a commit to every open screen

Status: [Implemented]

In simple terms. When something changes in a doctor's queue, every screen showing that queue (the doctor screen, the reception board, the waiting-room TV) must update within moments, even when the Platform runs several API servers. The Platform does this with a "transactional outbox": the change and a "please announce this" note are saved together; a separate worker reads the notes and broadcasts them; every API server pushes them to the browsers that are listening.

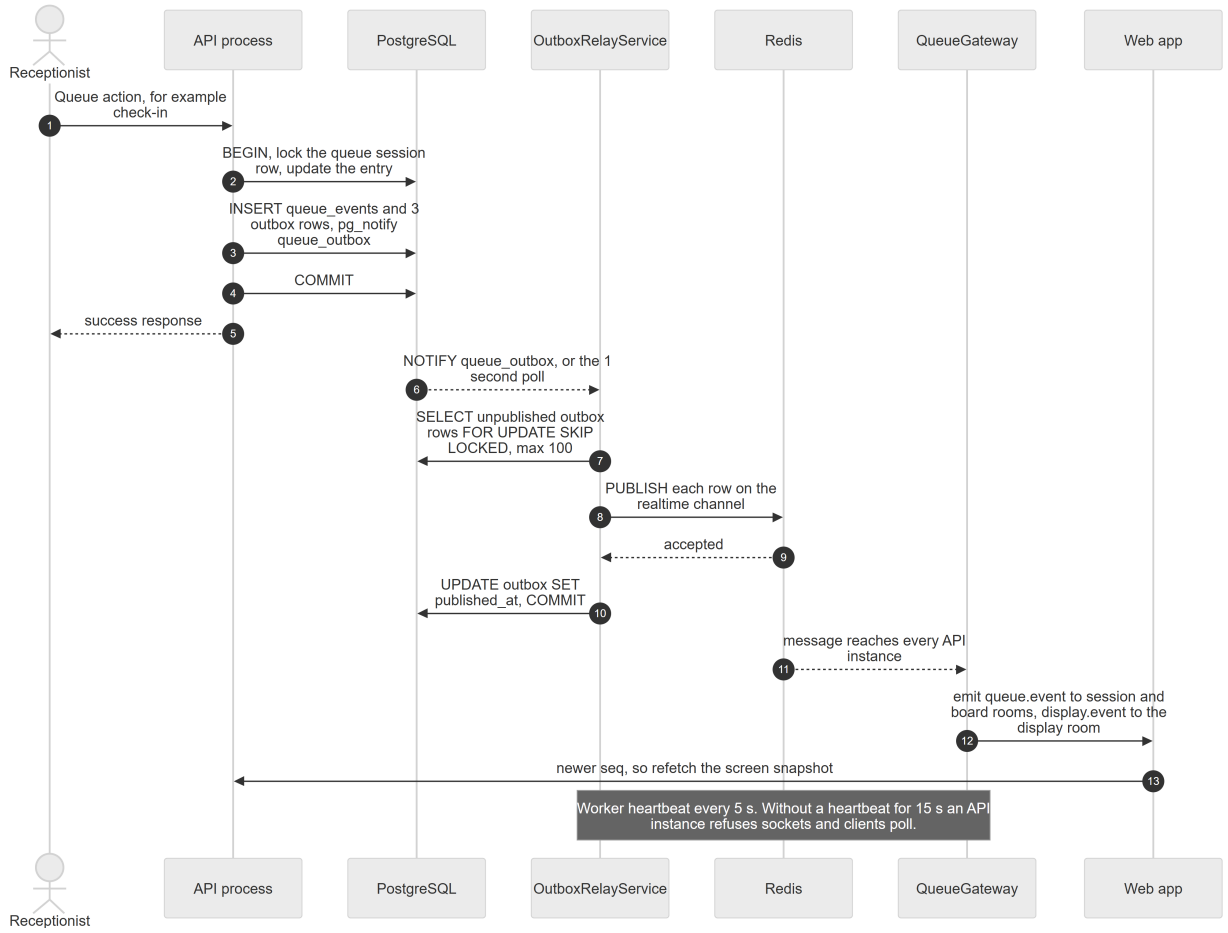


Figure 5.2 — Sequence of a queue change travelling from the database transaction, through the outbox relay and Redis, to the web app.

The diagram has seven participants: the Receptionist, the API process, PostgreSQL, the `OutboxRelayService` (in the worker), Redis, the `QueueGateway` (the Socket.IO gateway in every API process) and the Web app. (1) The receptionist performs a queue action, for example a check-in. (2) The API begins a transaction, locks the queue session row and updates the queue entry. (3) In the same transaction it inserts a `queue_events` audit row and three `outbox` rows, and calls `pg_notify('queue_outbox')`. (4) It commits. (5) The receptionist gets the success response. (6) PostgreSQL delivers the `queue_outbox` notification to the relay (if the notification is missed, the relay also polls every second). (7) The relay selects up to 100 unpublished outbox rows with `FOR UPDATE SKIP LOCKED`. (8) It publishes each row to Redis on the realtime channel, and (9) Redis confirms. (10) The relay stamps `published_at` on those rows and commits. (11) Redis delivers the message to every API instance. (12) Each gateway emits `queue.event` to the session room and the location board room, and `display.event` to the location's display room. (13) The web app sees a newer sequence number and refetches the screen's snapshot over HTTP. A note across the relay and the gateway says that the worker sends a heartbeat every 5 seconds, and that an API instance that has heard no heartbeat for 15 seconds refuses socket connections, so clients fall back to polling.

Facts behind the figure:

- **Three outbox rows per change**, one per room: `session:<sessionId>` (one doctor's queue), `location:<locationId>:board` (the reception board) and `location:<locationId>:display` (the TV). Payloads carry ids, statuses, tokens and a sequence number, never patient names.
- **At-least-once delivery.** If Redis does not accept the batch, the relay's transaction rolls back and the rows are retried. Clients drop duplicates by sequence number.
- **Rooms and access.** Clients connect to the `/realtime` Socket.IO namespace with their access token and `subscribe` to rooms. The gateway checks access per room: display rooms are open without login; session

and board rooms need access to that location; `org:<organizationId>` needs access to the organization.

- **Access changes use the same sockets.** When an organization's access version changes (a role edit, a feature switch), every process hears it over a separate Redis channel and the gateway emits `access.changed` to the `org:<id>` room. The web app then refetches `GET /me/access` (Chapter 11).
- **Single-process mode.** In the `all` role (development), the relay hands rows straight to the local gateway and Redis is not needed for realtime.
- **Notifications** use the same pattern with their own outbox table (`notification_outbox`) and relay loop (Chapter 6).

Technical references: `backend/src/queue/services/queue-orchestrator.service.ts` · `QueueOrchestrator.emit` · `backend/src/queue/realtime/outbox-relay.service.ts` · `RedisRealtimePublisher` · `RedisRealtimeSubscriber` · `QueueGateway` · `web-app/src/features/access/hooks/useAccessSocket.ts`

5.9 Requests that take a different path

Status: [Implemented]

- **Public requests** (login, register, refresh, password reset, verify email, logout, the login-page theme, the TV display state, lab magic links) skip the token check because they are listed in `PUBLIC_ROUTES` , and skip the other guards because they carry `@Public()` .
- **Context-free requests** (`@ContextFree()`) skip the tenant and access checks and check access themselves in the handler. Examples: `/auth/select-location` , `/auth/switch-location` , `GET /me/access` , `GET /feature-flags/my-features` , every `/locations` endpoint.
- **Super-admin requests** (`@RequireSuperAdmin()`) need no tenant headers; the tenant guard checks the super-admin role instead.
- **Lab portal requests** (`/lab-portal/...`) carry `x-lab-id` instead of clinic headers and are protected by their own lab portal guard.

Technical references: `backend/src/common/decorators/` · `backend/src/auth/auth.controller.ts` · `backend/src/labs/`

Chapter 6 — Processes, deployment and environments

This chapter explains how the backend of the Platform runs as operating-system processes, what each process does, what Redis is used for, who owns the database schema, and which environment variables the code reads. It is written for the product owner who has to run or deploy the Platform. Everything comes from the backend code and the current migrations document; no deployment configuration exists in the repository, and that gap is stated where it matters.

6.1 One codebase, three process roles

Status: [Implemented]

The backend is one NestJS application, but it can start in one of three **process roles**. The role decides which parts are switched on. The roles are defined in `backend/src/common/runtime/process-role.ts`.

Role	Entry file	Start command	What it runs	How many
`all`	`main.ts`	`npm run start:dev` (watch mode)	Everything: HTTP API, Socket.IO gateway, outbox relay, notification relay, scheduled jobs. The relay hands realtime events straight to the local gateway, so Redis is not needed for realtime.	One. The development default.
`api`	`main.api.ts`	`npm run start:api` (after a build)	HTTP API and the Socket.IO gateway. Receives realtime events from the worker through Redis. Runs no background jobs.	Two or more, each with its own port.
`worker`	`main.worker.ts`	`npm run start:worker` (after a build)	Outbox relay, notification relay and every scheduled job. No HTTP server at all.	**Exactly one.**

The table lists each role with its entry file, its npm script, what it runs and how many copies should run. The split exists so that the API can be scaled out without running the background jobs more than once: scheduled jobs are only loaded where jobs run (`runsJobs(role)` is true for `all` and `worker`), and HTTP is only started where it is needed (`runsHttp(role)` is true for `all` and `api`).

How the role reaches the code: each entry file calls `AppModule.forRole(role)`. That method adds, per role, the TypeORM connection (with schema ownership only for HTTP roles), the `ScheduleModule` (only where jobs run), `QueueRuntimeModule.forRole(role)` (gateway, Redis subscriber, relay and queue jobs as needed) and `NotificationsRuntimeModule.forRole(role)` (the notification sending loop only where jobs run).

Two oddities in the npm scripts:

- `npm run start:prod` runs `node dist/main`, but the build writes `dist/src/main.js`. Use `start:api` and `start:worker` for built code [Partial].
- The `start:api` and `start:worker` scripts run the compiled `dist/src/...` files, so `npm run build` must run first.

Technical references: `backend/src/common/runtime/process-role.ts` · `ProcessRole` · `runsHttp` · `runsJobs` · `backend/src/main.ts` · `backend/src/main.api.ts` · `backend/src/main.worker.ts` · `AppModule.forRole`

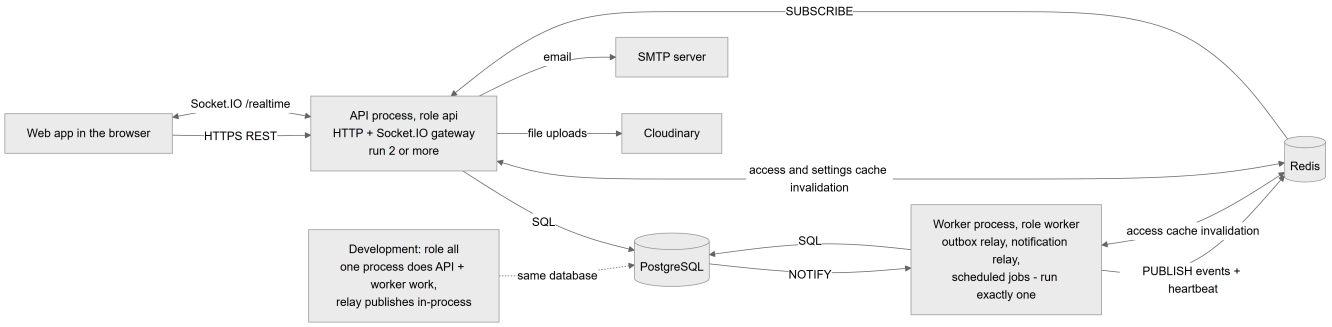


Figure 6.1 — The production process layout: API processes, one worker, PostgreSQL, Redis and the outside services, plus the single development process.

The figure shows the web app in the browser on the left. It sends HTTPS REST requests to the "API process, role api", which runs HTTP and the Socket.IO gateway and is meant to run two or more times; a two-way arrow labelled "Socket.IO /realtime" connects the same two boxes. The API process sends SQL to PostgreSQL, uploads files to Cloudinary and sends email through an SMTP server. The "Worker process, role worker" runs the outbox relay, the notification relay and the scheduled jobs, and must run exactly once; it also sends SQL to PostgreSQL, and PostgreSQL sends it NOTIFY messages. The worker publishes realtime events and a heartbeat to Redis, and Redis delivers them to the API process through a subscription. Two more two-way arrows connect Redis with the API process (access and settings cache invalidation) and with the worker (access cache invalidation). A separate box at the bottom, "Development: role all", shows that in development one process does the work of both API and worker and publishes realtime events in-process; a dotted line shows that it uses the same database.

6.2 What runs where

Status: [Implemented]

Component	`all`	`api`	`worker`
HTTP API (Express, port `PORT`)	yes	yes	no
Socket.IO gateway `/realtime` (`QueueGateway`)	yes	yes	no
Redis realtime subscriber (`RedisRealtimeSubscriber`)	no	yes	no
Queue outbox relay (`OutboxRelayService`)	yes, publishes in-process	no	yes, publishes to Redis
Notification relay (`NotificationRelayService`)	yes	no	yes
Scheduled jobs (`@Cron`, `@Interval`)	yes	no	yes
Postgres extensions, schema synchronize (dev), startup migrations	yes	yes	no
Feature catalog seeding and role template seeding at boot	yes	yes	yes
Access and settings cache invalidation bus (Redis)	yes	yes	yes

The table shows, for each running component, in which role it is active. Seeding runs in every process because every process loads every domain module; both seeders insert only missing rows and ignore conflicts, so concurrent processes are safe.

The relays. Both relays are woken by PostgreSQL `LISTEN` / `NOTIFY` on a dedicated database connection and also poll as a fallback: the queue relay every second, the notification relay every two seconds (which also picks up retries and messages held back by quiet hours). If the `LISTEN` connection fails, they poll only.

Scheduled jobs. There are eight. None of them sets a timezone, so every cron time in the next table is in the **server's** timezone, not the location's.

Job	Schedule	What it does
`queue-no-show-marker`	every 60 seconds	Marks booked patients who never checked in, past start time plus the allowed lateness, as no-show.
`queue-call-timeout`	every 15 seconds	Skips a called patient who never came, when that queue setting is on.
`queue-session-day-end`	every 5 minutes	Closes empty queue sessions from a past clinic day; sessions with waiting patients are left for an admin and logged.
`queue-cleanup`	every day at 03:15 server time	Deletes idempotency keys older than 24 hours and published outbox rows older than 7 days.
Patient credits expiry	every day at 01:00 server time	Marks active patient credits whose expiry date is before today (server-local midnight) as expired.
`billing-invariants`	every day at 02:30 server time	Checks the money invariants for every patient with billing rows and logs violations. Read-only.
Inventory alerts	every day at 08:00 server time	Finds low-stock and expiring items where inventory and inventory alerts are on, and only writes a log line; no notification is created [Partial].
`month-end-payroll`	on the 1st of each month at 05:00 server time	Generates payroll for the previous month for every active location.

The table lists each job, how often it runs and its effect. The queue jobs re-check state under the session lock through `QueueOrchestrator`, so running one twice is harmless. The queue jobs decide "which clinic day" with the location's timezone through `ClockService`; the cron trigger times themselves follow the server clock. The payroll job's code comment says 00:05, but the cron expression means 05:00.

Technical references: `backend/src/queue/queue-runtime.module.ts` · `backend/src/notifications/notifications-runtime.module.ts` · `backend/src/queue/jobs/queue-jobs.service.ts` · `BillingInvariantJob` · `InventoryAlertService` · `PayrollService.runMonthEndPayroll` · `PatientCreditsService.checkExpiredCredits`

6.3 What Redis is used for

Status: [Implemented]

Redis is used only for messages between processes. No business data is stored in it.

Use	Channel or name	Publishers → listeners	Without Redis
Realtime queue events	the queue realtime channel	Worker relay → every API instance; worker heartbeat every 5 s	In `all` mode not needed. In split mode the relay keeps rows unpublished and retries; API instances refuse sockets after 15 s without a heartbeat and clients poll.
Access changes	the access-changed channel	Any process that bumps an organization's access version → every process (access cache, feature cache, socket `access.changed`)	Only the process that made the change hears it; others catch up through their cache lifetimes (15 s for the access version, 30 s for features).
Settings cache invalidation	`settings-registry:invalidate`	Any process → every process	Other processes catch up through the settings cache lifetime.

Use	Channel or name	Publishers → listeners	Without Redis
Bull queue `submissions`	Bull connection	Registered, but no producer or consumer exists	Nothing depends on it.

The table lists the four uses of Redis, the channel names, who talks to whom and what happens when Redis is unavailable. Redis connections are created to fail fast (no offline queue) and reconnect with back-off. The connection string is `REDIS_URL`; the default is a Redis on the local machine. The exact channel names are defined in `realtime-bus.ts` and `access-change-bus.ts`.

Login rate limiting does not use Redis: attempts are counted in each process's memory, so with several API instances each instance counts on its own.

Technical references: `backend/src/common/redis/redis-connection.ts` · `backend/src/queue/realtime/realtime-bus.ts` · `AccessChangeBus` · `backend/src/settings-registry/settings-cache-bus.ts` · `backend/src/auth/guards/rate-limiter.guard.ts`

6.4 Who owns the database schema

Status: [Implemented]

In simple terms. The table structure is defined by the entity classes in the code. In development the API adjusts the database to the entities automatically on start. In production nothing changes the tables automatically: an operator first reviews the planned changes and then applies them. Hand-written migrations add what entities cannot express, such as triggers.

Technically.

- **Entities own table shapes.** Only processes that serve HTTP own the schema (`api` and `all`); the worker never touches it.
- **Start-up order in a schema-owning process:**
  1. `ensurePostgresExtensions` installs the `btree_gist` extension (needed by the doctor double-booking constraint) and a `uuid_generate_v7()` function (default primary key of the lab tables).
  2. The TypeORM connection starts. `synchronize` is **on** unless `NODE_ENV` is `production`.
  3. `runPendingMigrations` records the legacy baseline as applied, then runs pending migrations, each in its own transaction. If one fails, start-up fails.
- **Migrations.** `backend/src/migrations/` holds three files. `1776356529788-create-table.ts` is a legacy baseline that is recorded as applied and never executed. The other two (`doctor-compensation`, `lab-management-foundation`) are hand-written and idempotent: they skip tables that already exist and add triggers and backfills.
- **Production.** `synchronize` is off. The documented procedure is: build; review the SQL with `npm run schema:log:prod`; apply it with `npm run schema:sync:prod` after a backup; start **one** API instance so it runs the migrations; only then start further API instances and the worker (concurrent start-ups take no migration lock). Alternatively run `npm run migration:run:prod` as a release step.
- **CLI data source.** The TypeORM command-line tools use `backend/src/data-sourse.ts`, which loads entities by file pattern. It therefore also sees entity files that no module registers (`chairs` and the old single `inventory` entity), which can show up in `schema:log` output.

Technical references: `backend/src/app.module.ts` · `backend/src/common/database/ensure-extensions.ts` · `backend/src/common/database/migrations.ts` · `runPendingMigrations` · `backend/src/data-sourse.ts` · `docs/database-migrations.md`

6.5 Environments

Status: [Partial]

The code knows only one environment switch: `NODE_ENV`. When it equals `production`:

- TypeORM `synchronize` is off;
- the refresh-token cookie is marked `secure` (sent over HTTPS only).

Everything else is the same in every environment. There is no separate staging configuration, no environment-specific config file and no feature that behaves differently per environment. The README and the migrations document describe a production database hosted on Neon (managed PostgreSQL); the code's SSL settings are written to work with Neon.

Not in the repository [Not implemented]:

- no Dockerfile or container compose file;
- no CI pipeline (no `.github/` folder or other CI configuration);
- no hosting, process-manager or load-balancer configuration.

Whoever deploys the Platform must therefore provide, outside the repository: a PostgreSQL database, a Redis server (required as soon as `api` and `worker` run as separate processes), a way to run two or more `api` processes behind a load balancer and exactly one `worker`, HTTPS, and the web app build. The Socket.IO gateway allows both WebSocket and long-polling transports; the web app connects with WebSocket only, and a code comment notes that long-polling behind several API instances would need sticky sessions.

Technical references: `backend/src/app.module.ts` · `backend/src/auth/auth.service.ts` (`REFRESH_TOKEN_COOKIE_OPTIONS`) · `backend/src/queue/realtime/queue.gateway.ts`

6.6 Environment variables

Status: [Implemented]

Values are never shown in this handbook. The backend reads its variables through `@nestjs/config` (which loads `backend/.env`) or `process.env`. The template file `backend/.env.example` lists some of them.

Variable	Purpose	In `.env.example`
`DATABASE_URL`	PostgreSQL connection string for the API, the worker, the relays' `LISTEN` connections and the CLI.	yes
`PORT`	HTTP port of an API process (default 5000).	yes
`NODE_ENV`	`production` turns `synchronize` off and makes the refresh cookie secure.	yes
`JWT_ACCESS_SECRET`	Signs and checks access tokens (also used by the Socket.IO gateway).	yes
`JWT_REFRESH_SECRET`	Listed in the template, but no code reads it: refresh tokens are random values stored with the session, not JWTs.	yes

Variable	Purpose	In `.env.example`
`EMAIL_TOKEN_SECRET`, `RESET_TOKEN_SECRET`	Listed in the template, but no code reads them: verification and reset tokens are random values.	yes
`REDIS_URL`	Redis for the realtime bus, the access and settings buses and Bull (default: local Redis).	no
`CORS_ORIGIN`	The one web origin allowed by CORS and by the Socket.IO gateway (default `http://localhost:3000`).	no
`FRONTEND_URL`	Base URL of the web app used in email links (default `http://localhost:3000`).	no
`SMTP_USER`, `SMTP_PASS`	Credentials of the outgoing mail account.	yes
`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`	File uploads to Cloudinary.	yes
`MAGIC_LINK_SECRET`	Signs lab magic links; falls back to `JWT_ACCESS_SECRET`, then `JWT_SECRET`.	no
`JWT_SECRET`	Last fallback for the magic-link secret only.	no
`LAB_MAGIC_LINK_TTL_HOURS`	Lifetime of a lab magic link (default 24 hours, at most 7 days).	no
`LAB_LINK_RATE_LIMIT`	Request limit for magic-link pages (default 120).	no
`RATE_LIMIT_LOGIN_MAX`, `RATE_LIMIT_LOGIN_WINDOW_SECONDS`	Login rate limit (default 5 attempts per 900 seconds).	no
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `GITHUB_CLIENT_ID`, `GITHUB_CLIENT_SECRET`	OAuth sign-in. OAuth is not registered, so these have no effect [Not implemented].	yes
`VITE_API_BASE_URL` (web app)	Base URL of the API for the browser (default `http://localhost:5000`).	no web template exists

The table lists every environment variable the code reads, with its purpose and whether the template file mentions it. Variables only used by the integration test suites (for example `BILLING_IT_DATABASE_URL` and the `*_KEEP_SCHEMA` switches) are described in Chapter 7.

Technical references: `backend/.env.example` (names only) · `backend/src/bootstrap/http.ts` · `backend/src/lab-access/magic-links/magic-link-token.ts` · `backend/src/common/services/mail.service.ts` · `web-app/src/lib/api.ts`

Chapter 7 — Running the Platform locally

This chapter lists what a developer needs to run the Platform on one machine: prerequisites, installation, configuration, start commands, the first super-admin step and the test suites. It is written for the product owner who wants to see the system running or hand it to a new engineer. The steps come from the two `package.json` files, the backend's configuration code and the root README; nothing was executed while writing this chapter.

7.1 Prerequisites

Status: [Implemented]

Need	Why	Notes
Node.js and npm	Both applications are Node projects.	Neither `package.json` pins a Node version. The README recommends Node 20 or newer.
PostgreSQL	The only data store.	The account must be allowed to create the `btree_gist` extension and a function on first start. The API always opens its connection **with SSL** (`ssl: { rejectUnauthorized: false }`), so the server must accept SSL connections; Neon does. Whether a plain local PostgreSQL without SSL works was not verified [Unverified].
Redis	Messages between processes.	Optional when running the single development process (`all` role): realtime then works in-process. The access, settings and Bull connections still try to reach Redis and log warnings while it is missing. Required when `api` and `worker` run as separate processes.
A Cloudinary account	File uploads (logos, patient documents, attachments).	Uploads fail without it; the rest of the Platform runs.
An SMTP mail account	Verification and password-reset emails.	Optional for local work.

The table lists each external dependency, why the Platform needs it and what to know when setting it up locally.

Technical references: `backend/src/app.module.ts` · `backend/src/common/database/ensure-extensions.ts` · `README.md` (root)

7.2 Install

Status: [Implemented]

The two applications are installed separately; there is no root-level install.

1. In `backend/` , run `npm install` .
2. In `web-app/` , run `npm install` .

Technical references: `backend/package.json` · `web-app/package.json`

7.3 Configure

Status: [Implemented]

1. Copy `backend/.env.example` to `backend/.env`. The template names: `DATABASE_URL`, `PORT`, `NODE_ENV`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, the Google and GitHub OAuth pairs, `EMAIL_TOKEN_SECRET`, `RESET_TOKEN_SECRET`, `SMTP_USER`, `SMTP_PASS` and the three `CLOUDINARY_*` variables.
2. Fill in at least `DATABASE_URL` and `JWT_ACCESS_SECRET`. Leave `NODE_ENV` unset or not `production`, so the schema is created from the entities.
3. Add the variables the template does not list when the defaults do not fit: `REDIS_URL` (default: Redis on the local machine), `CORS_ORIGIN` and `FRONTEND_URL` (default `http://localhost:3000`), and optionally the magic-link and rate-limit variables. Chapter 6 lists every variable with its purpose.
4. The web app reads one variable, `VITE_API_BASE_URL` (default `http://localhost:5000`). Put it in `web-app/.env` only if the API runs elsewhere.

Several template entries have no effect: `JWT_REFRESH_SECRET`, `EMAIL_TOKEN_SECRET` and `RESET_TOKEN_SECRET` are not read by any code, and the OAuth variables do nothing because OAuth is not registered.

Technical references: `backend/.env.example` (names only) · `web-app/src/lib/api.ts`

7.4 Run

Status: [Implemented]

Terminal	Command	Result
1	in `backend/`: `npm run start:dev`	One `all`-role process in watch mode on port 5000: HTTP API, Socket.IO, relays and scheduled jobs.
2	in `web-app/`: `npm run dev`	The web app's Vite dev server on port 3000.

The table shows the two commands needed for a working local system. Then open `http://localhost:3000`.

What happens on the first backend start:

1. The `btree_gist` extension and the `uuid_generate_v7()` function are created.
2. TypeORM `synchronize` creates every table from the entities.
3. The legacy migration baseline is recorded and the two hand-written migrations run (triggers and backfills).
4. The feature catalog is seeded with the 24 active features, and the 5 system role templates are seeded into every existing organization.

To try the production split locally: `npm run build`, then `npm run start:api` and `npm run start:worker` in two terminals, with Redis running. Do not start a second worker.

Technical references: `backend/src/main.ts` · `backend/src/bootstrap/http.ts` · `AppModule.forRole` · `FeatureFlagsService.seedCatalog` · `AccessTemplatesService`

7.5 The first super admin

Status: [Partial] — there is no screen, command or seed that creates a super admin; it is a manual database step.

The super admin is the platform operator who creates organizations, entitles features, records subscription payments and runs Config Studio. A user is a super admin when the `users.global_role` column holds `super_admin`.

1. Open `/register` and sign up. Self-signup creates **both** a user and a new clinic: an organization on a 14-day trial with Professional limits, its `MAIN` location and the ownership link. That trial clinic is harmless for local work.

2. Promote the user with one SQL statement against the database:

```
UPDATE users SET global_role = 'super_admin' WHERE email = '<your sign-up email>';
```

3. Sign out and sign in again, so the login response reports the new role.
4. As super admin you choose an organization at sign-in, and you can open the setup pages (Organizations, Locations, Lab Directory, Config Studio, Subscriptions) even before choosing a location. Create an organization and its locations, entitle features, then add employees and give them logins and roles (Chapter 11).

Email verification does not block sign-in.

Technical references: `backend/src/common/enums/global-role.enum.ts` · `backend/src/subscriptions/clinic-setup.service.ts` · `web-app/src/routes/__root.tsx` (`SUPER_ADMIN_SETUP_PATHS`)

7.6 Test suites

Status: [Implemented]

Suite	Command	What it runs	Needs a database
Backend unit	in `backend/` : `npm test`	Jest over the 67 `*.spec.ts` files in `backend/src` , including `access-coverage.spec.ts` , which fails if any endpoint lacks an access decorator.	no
Backend e2e	`npm run test:e2e`	One starter test that boots the full application and calls `GET /` .	yes (it starts the real module)
All DB integration specs	`npm run test:int`	Every `*.int-spec.ts` file under `backend/test/` with the billing suite's setup.	yes
Labs	`npm run test:int:labs`	Lab module integration tests.	yes
Tenancy	`npm run test:int:tenancy`	Tenant isolation tests.	yes
Access core	`npm run test:int:access`	Access resolution, patient scope, sidebar layouts.	yes
Access acceptance	`npm run test:int:acceptance`	The 166-case access-control acceptance plan in 11 spec files.	yes
Web	in `web-app/` : `npm test`	Vitest with Testing Library and jsdom over 34 test files.	no

The table lists each test command, what it covers and whether it needs PostgreSQL. There are 35 integration spec files in total.

How the database suites protect data: each suite creates its **own PostgreSQL schema** (for example `billingit` , `tenancy_it` , `acceptit`), drops and recreates it at the start and drops it at the end, and never touches the `public` schema. They read `BILLING_IT_DATABASE_URL` when set, otherwise `DATABASE_URL` ; for Neon they strip the connection-pooler part of the host name, because schema isolation needs a direct connection. Some suites accept a `*_KEEP_SCHEMA` variable to keep the schema for inspection. The suites share one database, so run them one at a time.

Other useful scripts: `npm run lint` and `npm run format` (backend), `npm run check` (web: Prettier and ESLint fixes), and `npm run access:export` (backend), which regenerates the web app's copy of the permission catalog after `permissions.ts` changes.

Technical references: `backend/package.json` · `backend/test/*.json` · `backend/test/billing/db.config.ts` · `web-app/vitest.config.ts` · `backend/scripts/export-access-catalog.ts`

Chapter 8 — The multi-tenant model

The Platform serves many clinic businesses from one installation and one database, and each business can have several branches. This chapter explains how the data of one clinic is kept apart from another, how a person can work for clinics, how a signed-in user ends up "inside" one clinic and one branch, and how the Platform decides what "today" means for a clinic. It is written for the product owner, who needs this model to reason about every other chapter.

8.1 Organizations, locations and the tenant boundary

Status: [Implemented]

In simple terms. An **organization** is one clinic business; it is the tenant. A **location** is one branch of that business. A dental group with three branches is one organization with three locations. Almost every record in the Platform (patients, appointments, invoices, staff assignments, queue sessions) carries the organization it belongs to, and most also carry a location.

Technically.

- `organizations` is the tenant table. `locations` belongs to exactly one organization (`organization_id` , deleted with it by `ON DELETE CASCADE`). Each location has one `location_settings` row.
- Tenant tables carry an `organization_id` column, and services always filter by the active organization. Data that differs per branch also carries `location_id` .
- All tenants share one database and one schema. Isolation is done in application code (guards plus `organization_id` filters in every query), not by separate databases or database row-level security.
- The **super admin** stands outside all tenants (`users.global_role = 'super_admin'`) and can enter any active organization.

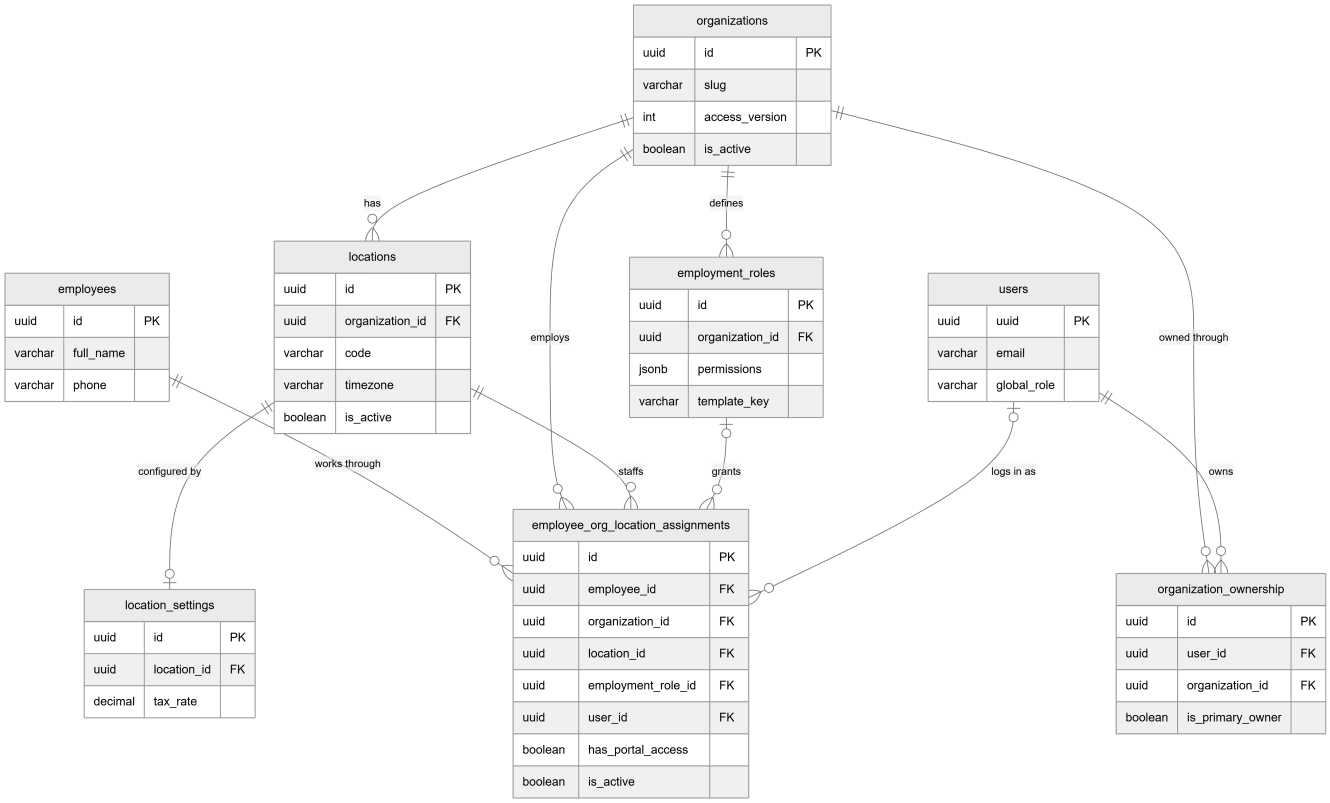


Figure 8.1 — The tables that define who belongs to which organization and location.

The diagram shows eight tables and their relationships. `organizations` (with `id`, `slug`, `access_version`, `is_active`) "has" zero or more `locations` (with `organization_id`, `code`, `timezone`, `is_active`). Each location is "configured by" at most one `location_settings` row (with `location_id` and `tax_rate`). `users` (with `uuid`, `email`, `global_role`) "owns" organizations through `organization_ownership` rows (with `user_id`, `organization_id`, `is_primary_owner`); an organization can have several owners and a user can own several organizations. An organization "defines" zero or more `employment_roles` (with `organization_id`, the `permissions` map and `template_key`). The `employees` table (with `id`, `full_name`, `phone`) has no organization column; a person "works through" zero or more `employee_org_location_assignments`. Each assignment links one employee to one organization ("employs") and one location ("staffs"), optionally to one employment role ("grants"), and optionally to one user account ("logs in as"); it also holds `has_portal_access` and `is_active`.

Technical references: `backend/src/organizations/entities/organization.entity.ts` · `backend/src/locations/entities/location.entity.ts` · `backend/src/auth/services/authorization.service.ts`

8.2 Ownership

Status: [Implemented]

An **owner** is a user linked to an organization through an `organization_ownership` row. Owners are not employees and have no role: the access resolver gives an owner **every permission at its widest scope at every location** of the organization (Chapter 11).

- **How owners are created.** Self-signup creates the clinic and makes the new user its primary owner (`is_primary_owner = true`). Further owners are created from user administration with the "owner" account type and are stored with `is_primary_owner = false`.
- **Several owners, several clinics.** A clinic can have several owners and one user can own several clinics; the pair (`user_id`, `organization_id`) is unique.
- **What an owner can enter.** Every active location of every active organization the owner owns.

Technical references: `backend/src/organizations/entities/organization-ownership.entity.ts` · `backend/src/user/user.service.ts` · `AuthorizationService.isOwner`

8.3 The employee person record and assignments

Status: [Partial] — the data model supports one person at several locations and clinics, but the API only ever creates one assignment per employee.

In simple terms. The Platform separates the **person** from the **job**. The **employee** record holds who the person is (name, phone, national ID number, gender, date of birth, photo). Each **assignment** says where and how that person works: in which organization, at which location, in which role, with which salary, and whether the person may sign in there.

Technically.

- `employees` has **no** `organization_id`. Phone is unique across the whole Platform, and so is the national ID number when given.
- `employee_org_location_assignments` holds, per job: `organization_id`, `location_id`, `employment_role_id` (the **role**, may be empty), `designation`, `employee_code` (unique per organization), an optional queue `token_prefix`, salary fields, `has_portal_access`, `user_id` (the **staff login**, may be empty), `is_primary` and `is_active`. One person can have at most one assignment per location (unique on `employee_id`, `location_id`).
- Permissions are tied to the assignment, not to the person: the same person can be a Doctor at one location and have no role at another.
- **What the API does today.** `POST /employees` creates the person and exactly one assignment, at the location the creator is working in. It refuses a phone number that already exists anywhere on the Platform ("An employee with this phone number already exists"). No endpoint adds a second location or a second clinic to an existing employee; the multi-location model can only be filled outside the API [Partial].
- **Staff logins.** User administration can create a login for an existing employee; that login is linked to all of the person's active assignments in the organization. A login only grants entry where the assignment is active and `has_portal_access` is on.

Technical references: `backend/src/employees/entities/employee.entity.ts` · `backend/src/employees/entities/employee-org-location-assignment.entity.ts` · `EmployeesService.create` · `backend/src/user/user.service.ts`

8.4 Who can enter which clinic and branch

Status: [Implemented]

At sign-in, and whenever a location is selected, the Platform works out the list of places a user may enter (`AuthorizationService.resolveUserAccess`):

Kind of user	Organizations offered	Locations offered
Super admin	Every active organization	Every active location of each
Owner	Every active organization the user owns	Every active location of each
Employee with a staff login	Organizations where the user has an active assignment with portal access, if the organization is active	Exactly those assigned locations that are active

The table shows the three ways to gain entry. When one user is both owner and employee in the same organization, the owner entry wins. Deactivated organizations and locations disappear from the list. The same rules are checked again on every request by `validateAccess` (section 8.5): an owner passes for any active location of the organization, an employee needs an active, portal-enabled assignment at exactly that location.

Technical references: `AuthorizationService.resolveUserAccess` · `AuthorizationService.validateAccess`

8.5 The tenant context on every request

Status: [Implemented]

In simple terms. The web app always tells the API which clinic and which branch the user is working in. The API checks that the user is allowed there before doing anything else.

Technically.

- The access token (a JWT, valid 15 minutes) carries only the user id, email, global role and session id. **It does not carry the organization or location.**
- The web app's axios instance adds two headers to every request from the in-memory auth store: `x-organization-id` and `x-location-id` (lab portal requests carry `x-lab-id` instead).
- `TenantContextGuard`, the second of the five global guards, reads the headers and calls `validateAccess`. Missing headers give 400 "Missing tenant context. Please select an organization and location."; no access gives 403. On success it stores `request.tenantContext`.
- Controllers read the context with `@ActiveTenant()` and pass the organization id (and, where relevant, the location id) to the service, which filters every query by it.
- The guard steps aside for `@Public()` and `@ContextFree()` routes and for super admins, and checks the super-admin role for `@RequireSuperAdmin()` routes. It has **no** path-prefix exceptions.
- The organization and location in the headers also select the access context (Chapter 11) and the active features (Chapter 10).

Technical references: `backend/src/auth/guards/tenant-context.guard.ts` · `TenantContextGuard` · `backend/src/common/decorators/active-tenant.decorator.ts` · `web-app/src/lib/api.ts`

8.6 Sign-in, choosing a location and switching

Status: [Implemented]

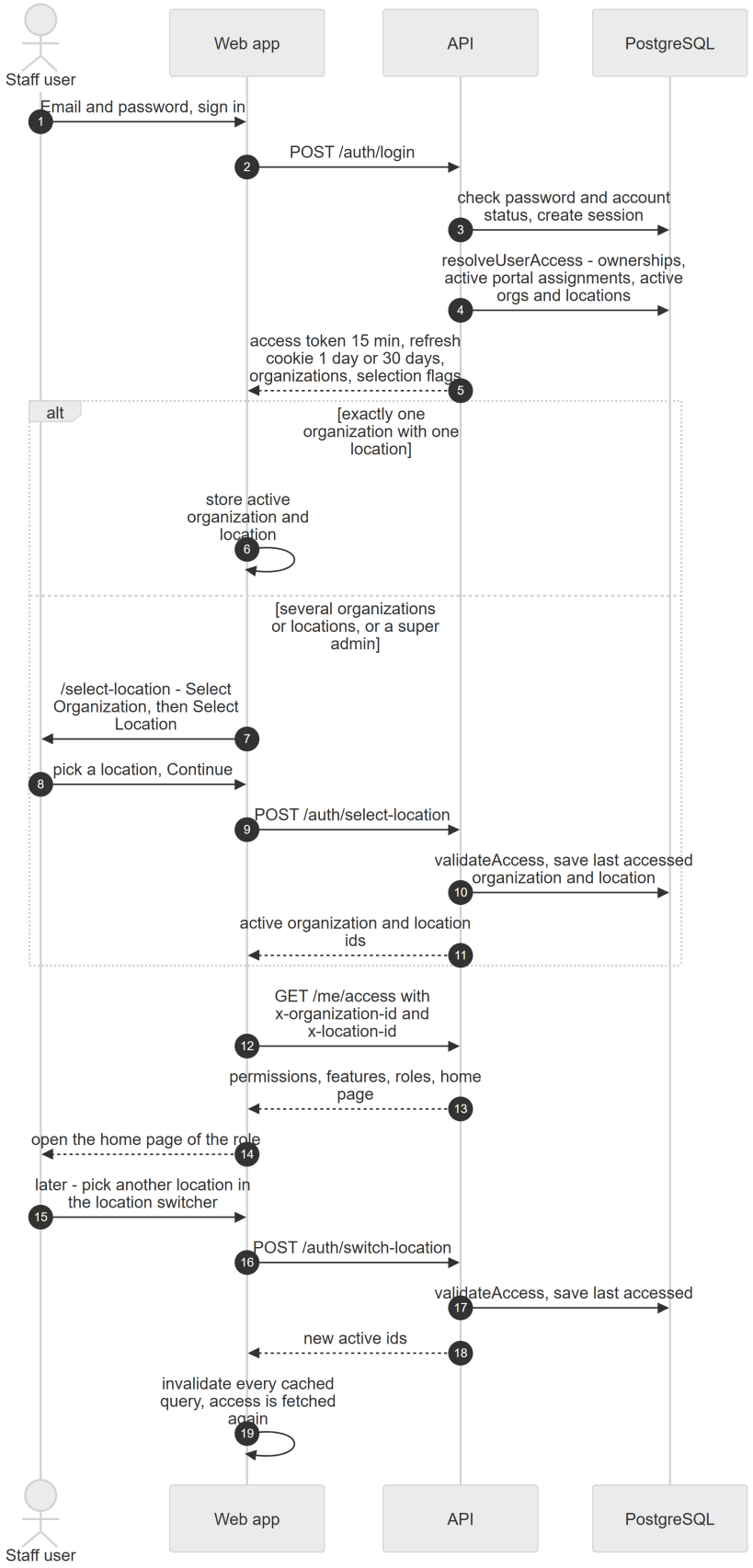


Figure 8.2 — Sequence from sign-in to the first page, and a later location switch.

The diagram has four participants: the Staff user, the Web app, the API and PostgreSQL. (1) The user enters email and password and signs in. (2) The web app calls `POST /auth/login`. (3) The API checks the password and account status and creates a session. (4) It resolves the user's access: ownerships, active portal assignments, active organizations and locations. (5) It returns an access token valid for 15 minutes, sets a refresh cookie valid for 1 day (30 days when the user ticks "keep me signed in"), and returns the list of organizations with two flags

saying whether an organization or a location must still be chosen. An "alt" box shows two paths. If the user has exactly one organization with one location, the web app stores them as active. Otherwise, or for any super admin, the web app shows `/select-location` with "Select Organization" and then "Select Location"; the user picks a location and clicks Continue; the web app calls `POST /auth/select-location`; the API runs `validateAccess` and saves the last accessed organization and location; it returns the active ids. After the box, the web app calls `GET /me/access` with the two tenant headers and receives the permissions, features, roles and home page, then opens the role's home page. The last part shows a later switch: the user picks another location in the location switcher, the web app calls `POST /auth/switch-location`, the API checks access and saves the choice, returns the new ids, and the web app invalidates every cached query so all data and the access context are fetched again.

Rules behind the figure:

- **Automatic choice.** One organization with one location is selected automatically. A super admin always chooses an organization first. When a choice is needed, the API pre-fills the last organization and location the user worked in (`users.last_accessed_organization_id` / `last_accessed_location_id`), if they are still allowed.
- **Where the browser goes.** The root route (`__root.tsx`) redirects: not signed in → `/login`; signed in but no location chosen → `/select-location`. Super admins may open the setup pages (Organizations, Locations, Lab Directory, Config Studio, Subscriptions) without a location. Users who only belong to a lab go to the lab portal.
- **Session restore.** The auth store is in memory only. On page load the web app calls `POST /auth/refresh`; the refresh cookie returns a new token and the same selection data.
- **Switching.** The location switcher in the top bar lists every organization and location the user may enter, grouped by organization. With only one location it shows the name without a menu.

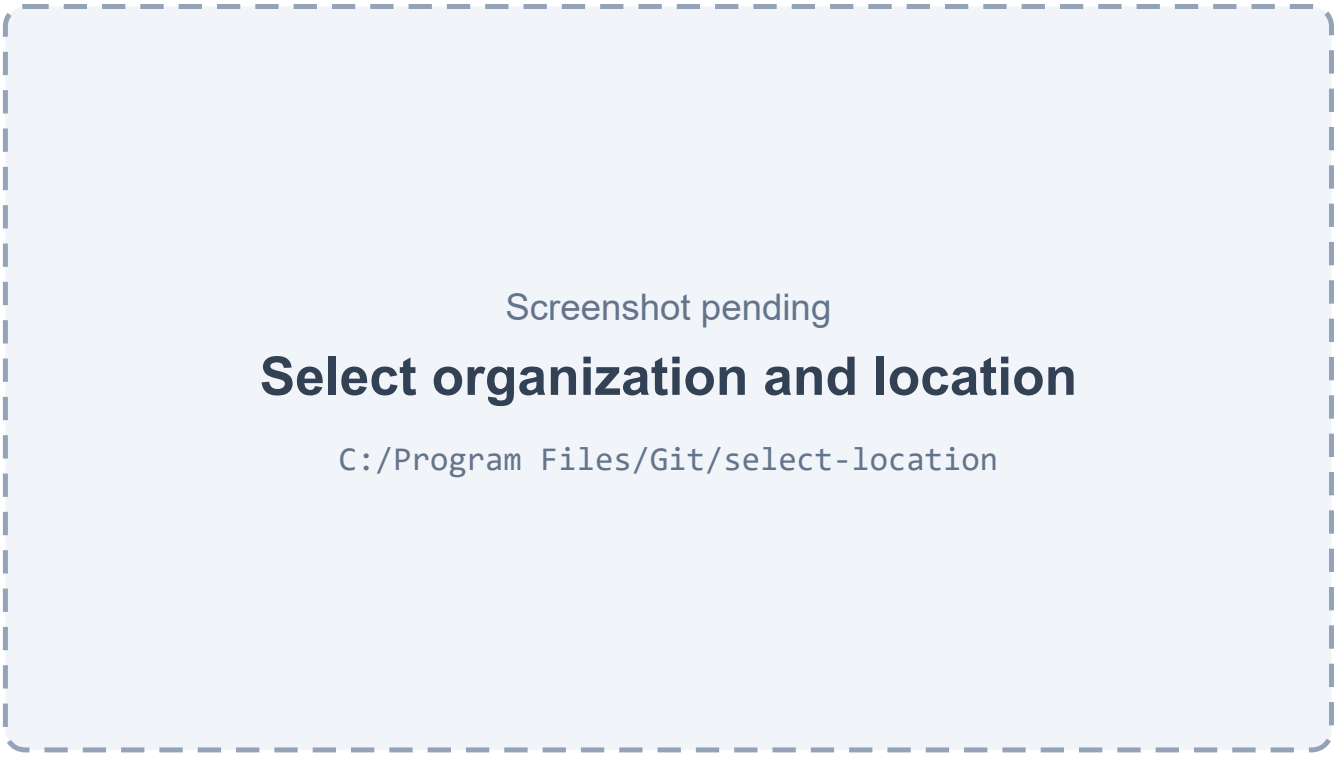


Figure 8.3 — Select organization and location (screenshot pending).

The page appears after sign-in when a choice is needed. Its header reads "Select Organization" or "Select Location", with a welcome line using the user's name ("Choose an organization to continue" or "Choose where you'd like to work today"). The organization step shows one card per organization; picking an organization with a single location selects it at once. The location step shows one card per location of the chosen organization and a **Continue** button. If the organization has no locations yet, the page says "No locations yet" and offers **Create**

First Location, which opens the Locations page. On success a toast says "Location selected" and the user lands on the home page. (The only existing screenshot of this page is excluded from the handbook because it shows compliance badges that the current code no longer displays.)

Technical references: `backend/src/auth/auth.service.ts` (`login` , `selectLocation` , `switchLocation`) · `web-app/src/routes/__root.tsx` · `web-app/src/features/auth/components/LocationSelector.tsx` · `web-app/src/components/layout/LocationSwitcher.tsx`

8.7 Clinic time: the location's timezone

Status: [Implemented]

In simple terms. A clinic's day starts at midnight where the branch is, not where the server is. Each location stores its own timezone, and every "today", "now" and opening-hours check uses it.

Technically.

- `locations.timezone` holds an IANA timezone name. New locations created through `POST /locations` default to `UTC` ; the setup wizard copies the organization's timezone (default `Asia/Karachi`) to every location. The value is not validated; an unknown name falls back to UTC.
- `ClockService` (`common/clock/`) is the single source of clinic time. `timezoneFor(locationId)` reads the timezone (cached for 60 seconds); `today(tz)` gives the local date; `dayRange(tz, date)` gives the UTC start and end of a local day; `at(tz, date, time)` converts a local wall-clock time; `wallClock(tz, instant)` gives the local weekday and time.
- It is used by appointments (working hours, availability, same-day checks), the queue (the queue day and token day), billing, payroll, reports, notifications, employees and the lab modules.
- Scheduled jobs are triggered by the **server** clock (Chapter 6); jobs that need a clinic day compute it through `ClockService` .

Technical references: `backend/src/common/clock/clock.service.ts` · `ClockService` · `backend/src/common/clock/tz.util.ts`

8.8 Limitations and known gaps

- One person cannot be given a second location or clinic through the API (section 8.3) [Partial].
- Deactivating an organization or a location hides it at the next sign-in or selection, but does not end sessions that are already working there; requests keep passing the tenant guard for an owner of a deactivated organization, because `validateAccess` checks the location but not the organization's status for owners [Partial].
- Locations created from the Locations screen get timezone `UTC` and currency `USD` unless changed, because that form has no timezone or currency field (Chapter 9).
- Isolation relies on every service filtering by organization; there is no database-level row security.

Chapter 9 — Organizations and locations

This chapter describes the two records that frame every clinic on the Platform: the **organization** (the clinic business, the tenant) and its **locations** (the branches). It covers their fields, who can create and change them, their settings, and how they are switched off. It is written for the product owner; subscriptions, trials and plan limits get their own chapter (Chapter 45), and only read-only mode is summarised here.

9.1 What these records are for

Status: [Implemented]

- An **organization** answers "who is the clinic?": name, contact details, default timezone and currency, specialties, plan and subscription dates.
- A **location** answers "where does it operate?": address, a short branch code, its own timezone (which defines "today" for queues and reports), its own currency, and per-branch settings such as the tax rate.

The **super admin** manages organizations. Locations are managed by anyone holding the permission `admin.locations.manage` in that organization: owners and super admins always hold it, the system **Admin** role holds it, and a custom role can be given it. The owner's one-time setup wizard (`/setup` , Chapter 45) also edits the organization and its main location.

Sidebar: "Organizations" (super admin only, `/organizations`) and "Locations" (`/locations` , needs `admin.locations.manage`).

Technical references: `backend/src/organizations/` · `backend/src/locations/` · `web-app/src/features/organizations/` · `web-app/src/features/locations/` · `backend/docs/modules/01-organizations-locations.md`

9.2 Organization fields

Status: [Implemented]

Field	Meaning
`name` , `slug`	Display name; `slug` is unique, generated from the name on creation and never regenerated.
`email` , `phone` , `website`	Contact details.
`subscription_plan`	`free` , `starter` , `professional` or `enterprise` . Default `free` . The plan limits are defined in code (`subscriptions/plans.ts`).
`clinic_type`	Free text. No API writes it.
`specialty_tags`	List of `general` , `dental` , `skin` , `ortho` , `physio` , `eye` . Default `general` . Written by the setup wizard; the web app uses it to show specialty-specific screens.
`timezone` , `currency`	Organization defaults: `Asia/Karachi` and `PKR` . The setup wizard copies them to every location.
`max_locations` , `max_users`	Per-organization overrides of the plan limits; empty means "use the plan's limit".
`trial_ends_at` , `paid_until`	Subscription period. Both empty means an **unmanaged** clinic: no expiry and no limits.

Field	Meaning
`onboarding_pending`	True for self-signup clinics until the owner finishes `/setup`.
`access_version`	Counter raised on every access change (roles, assignments, features, ownership); used to invalidate access caches (Chapter 11). Never written by normal saves.
`is_active`	Soft switch-off.

The table explains each column of `organizations` in business terms. Three related tables hang off an organization: `organization_ownership` (its owners), `organization_settings` (legacy settings, section 9.4) and `subscription_payments` (Chapter 45).

Technical references: `backend/src/organizations/entities/organization.entity.ts` · `Organization`

9.3 Managing organizations (super admin)

All endpoints are in `OrganizationsController` and carry `@RequireSuperAdmin()`, so only the super admin can call them, for any organization.

9.3.1 Create an organization

Status: [Partial] — the endpoint works; the screen's form has fields the API refuses.

What it does. Creates a clinic with a name and optional contact details and plan.

How the user does it. On the Organizations page, click **Create**, fill the form and save.

Behind the scenes. `POST /organizations` saves the row, then seeds the organization's feature flags from the catalog defaults (Chapter 10) and the 5 system roles (Chapter 11). The two steps are not one transaction. No owner, no location and no subscription dates are created: the clinic is "unmanaged" until the super admin sets dates or records a payment.

Rules. Name is required (at most 255 characters); email must be valid; phone at most 20, website at most 500 characters; plan must be one of the four enum values. Duplicate slug gives 409.

Known problems (by reading the code, not executed). The form also offers Clinic Type, Timezone and Currency, which the create DTO does not accept, so filling them makes the request fail with 400. The plan selector offers "FREE", "BASIC", "PREMIUM" and "ENTERPRISE", which do not match the API's lower-case values, so choosing a plan also fails with 400. No audit row is written.

Technical references: `OrganizationsController.create` · `OrganizationsService.create` · `CreateOrganizationDto` · `web-app/src/features/organizations/OrganizationManagementView.tsx`

9.3.2 List and view organizations

Status: [Implemented]

`GET /organizations` lists active organizations by name; `GET /organizations/:id` returns one. Deactivated organizations do not appear in the list.

Technical references: `OrganizationsController.findAll` · `OrganizationsController.findOne`

9.3.3 Edit an organization

Status: [Partial] — the endpoint works; the screen sends fields the API refuses.

`PATCH /organizations/:id` accepts name, email, phone, website, plan and `isActive`. The edit dialog sends the whole organization row (including `id`, `slug` and timestamps), which the strict validation pipe refuses with 400 (by reading the code, not executed). A plan change made here is not audited and does not touch the subscription dates.

Technical references: `OrganizationsController.update` · `UpdateOrganizationDto`

9.3.4 Deactivate an organization

Status: [Implemented]

The trash action asks "Deactivate organization?" and calls `DELETE /organizations/:id`, which sets `is_active = false`. Data is kept. The organization disappears from sign-in and selection lists. Sessions that are already working in it are not ended. There is no screen to reactivate it; only `PATCH /organizations/:id` with `isActive: true` does that.

Technical references: `OrganizationsController.deactivate` · `OrganizationsService.deactivate`



Figure 9.1 — Organizations (screenshot pending).

The page is titled "Organizations" with the subtitle "Manage clinic organizations and their settings" and a **Create** button at the top right. A table lists active organizations with the columns Name, Slug, Plan and Status, and an actions column with four icon buttons, each with a tooltip: "Manage features" (opens the feature entitlement page, Chapter 10), "Organization settings", "Edit organization" and "Deactivate organization". The create and edit dialog contains Name, a read-only Slug, Clinic Type, Timezone, Currency and Subscription Plan. Non-super-admins who open the URL are sent to the dashboard.

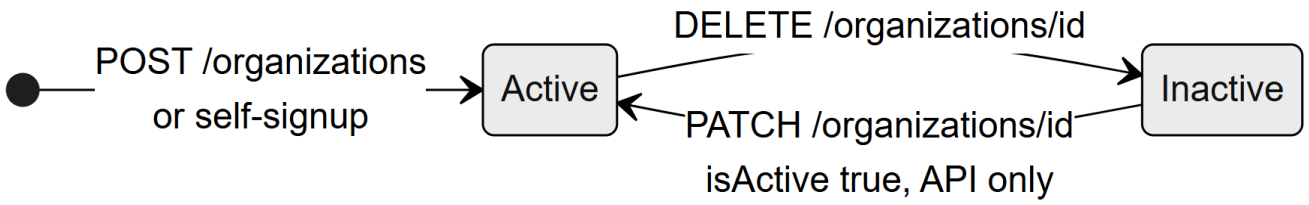


Figure 9.2 — The two states of an organization's `is_active` flag.

The diagram reads left to right. From the start point an organization becomes **Active**, either by `POST /organizations` or by self-signup. `DELETE /organizations/id` moves it to **Inactive**. `PATCH /organizations/id` with `isActive true`, which only the API offers, moves it back to Active. There is no final state: the API never hard-deletes an organization.

9.4 Organization settings (legacy)

Status: [Not implemented] — the settings are stored and editable, but no backend code reads them.

`organization_settings` has one row per organization, created on first read. It holds old feature switches (`feature_appointments`, `feature_inventory`, `feature_finance`, `feature_reminders`, `feature_whatsapp`, `feature_patient_portal`), `allow_location_override`, and booking defaults (`default_slot_duration`, `advance_booking_days`, `cancellation_notice_hours`).

- `GET /organizations/:id/settings` is allowed to the super admin and to owners of that organization.
- `PATCH /organizations/:id/settings` is super-admin only. Limits: slot duration 5–480 minutes, advance booking 1–365 days, notice 0–168 hours.
- **No backend code reads any of these values.** Real feature gating is the feature system (Chapter 10). The web app uses `allow_location_override` only to grey out three inputs on the location settings page.
- The settings screen sends the whole row, which the API should refuse with 400 (by reading the code).

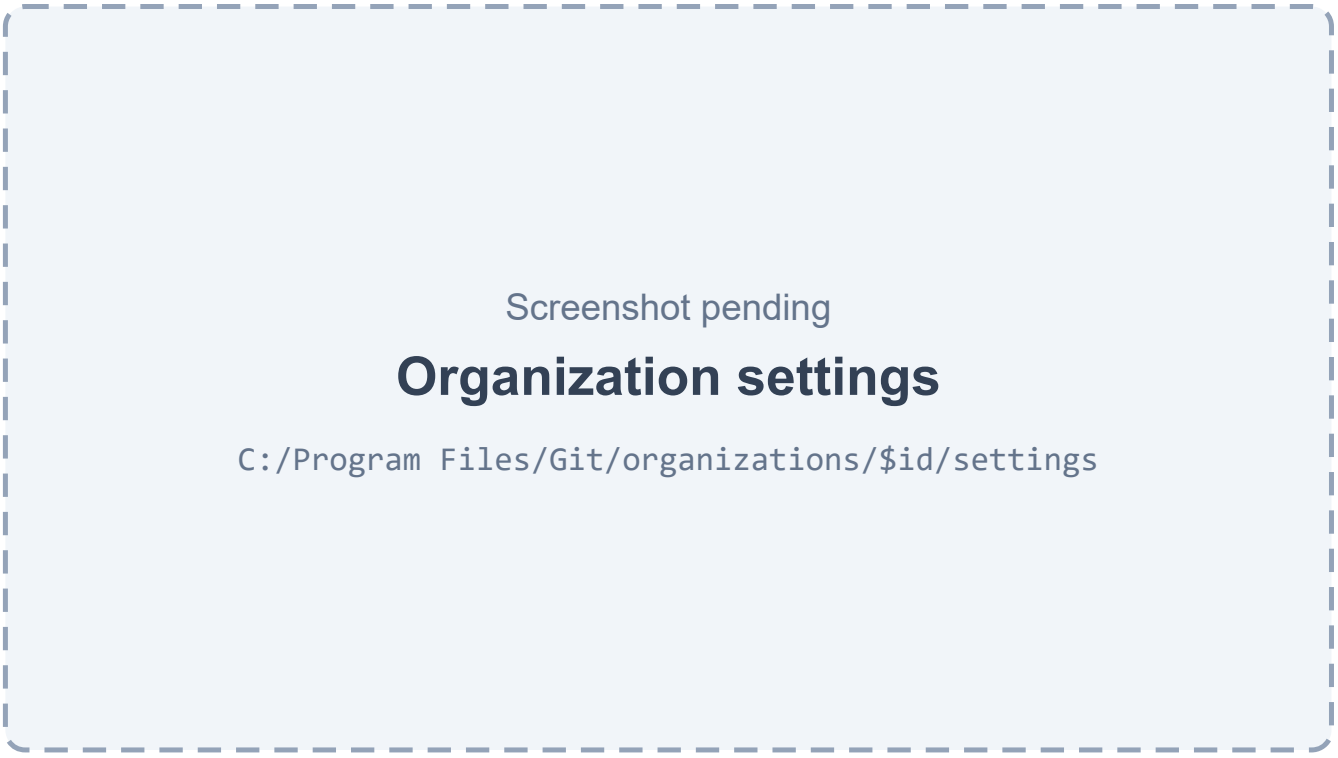


Figure 9.3 — Organization settings (screenshot pending).

The page has a back button to Organizations and two cards. "Global Overrides" holds one switch, **Allow Location Override**. "Booking Defaults" holds three number inputs: Default Slot Duration (mins), Advance Booking Days and Cancellation Notice (hours). A Save button stores the card values. None of these values changes how bookings behave.

Technical references: `backend/src/organizations/entities/organization-settings.entity.ts` · `OrganizationsController.getSettings` · `OrganizationsController.updateSettings` · `web-app/src/features/organizations/OrganizationSettingsFeature.tsx`

9.5 Location fields

Status: [Implemented]

Field	Meaning
`name`	Branch name, for example "Main location".
`code`	Short branch code, unique within the organization. Cannot be changed after creation.
`token_prefix`	Prefix of the daily queue tokens; falls back to `code`. Read by the queue's token engine; no API writes it.
`address`, `city`, `state`, `postal_code`, `country`, `phone`, `email`	Contact details.
`timezone`, `currency`	Defaults `UTC` and `USD` (different from the organization defaults). `ClockService` uses `timezone` for every clinic "today".
`is_main_branch`	A flag; several locations can carry it. The setup wizard treats the first active main branch as "the" main location.
`total_chairs`, `capacity_per_day`, `operating_hours`, `manager_id`, `business_model`	Stored, but no backend code reads them.
`is_active`	Soft switch-off.

The table explains each column of `locations`. Note the timezone default: a location created from the Locations screen gets `UTC`, because that form has no timezone or currency field; only the setup wizard sets the clinic's timezone and currency on locations.

Technical references: `backend/src/locations/entities/location.entity.ts` · `Location`

9.6 Managing locations

All location endpoints are in `LocationsController`. The controller is `@ContextFree()`, because an owner manages locations before choosing one; each handler therefore checks `admin.locations.manage` itself, in the organization of the location being acted on (or, for create and list, the organization in the body or the `x-organization-id` header). An invalid organization gives 400 "Select an organization first"; a missing permission gives 403 `NO_PERMISSION`.

9.6.1 Add a location

Status: [Implemented]

How the user does it. On the Locations page, click **Create Location** and fill Name, Code, Address, City, Country, Phone and Email. A super admin also picks the organization ("Organization (Required)").

Behind the scenes. `POST /locations` first checks the plan's location limit (`SubscriptionsService.assertCanAddLocation`, skipped for super admins), then saves the location and a default settings row as two separate saves (not one transaction). No audit row is written.

Rules and errors. Name and code are required; code at most 50 characters and unique in the organization (409 on duplicates). Plan limit reached gives 403 `PLAN_LIMIT`; an expired subscription gives 402 `SUBSCRIPTION_EXPIRED`.

Technical references: `LocationsController.create` · `LocationsService.create` · `CreateLocationDto`

9.6.2 List, view and edit locations

Status: [Partial] — list and view work; the edit screen sends fields the API refuses.

`GET /locations` returns, for a super admin, every active location of every organization; otherwise the active locations of the active organization (or of every organization where the user holds the permission organization-wide). `GET /locations/:id` returns one. `PATCH /locations/:id` accepts the create fields except `code` and the organization, plus `isActive`. The edit dialog sends the whole row, which the API should refuse with 400 (by reading the code). Setting `isActive: true` reactivates a location **without** the plan-limit check.

Technical references: `LocationsController.findAll` · `LocationsController.findOne` · `LocationsController.update` · `UpdateLocationDto`

9.6.3 Deactivate a location

Status: [Implemented]

The trash action asks "Deactivate location?" and calls `DELETE /locations/:id` (`is_active = false`). Nothing stops deactivating the last location or the main one. Staff assignments, appointments and queue sessions at that location are left as they are; the location stops being offered at sign-in and in the switcher. The list hides inactive locations, so the screen cannot reactivate one.

Technical references: `LocationsController.deactivate` · `LocationsService.deactivate`

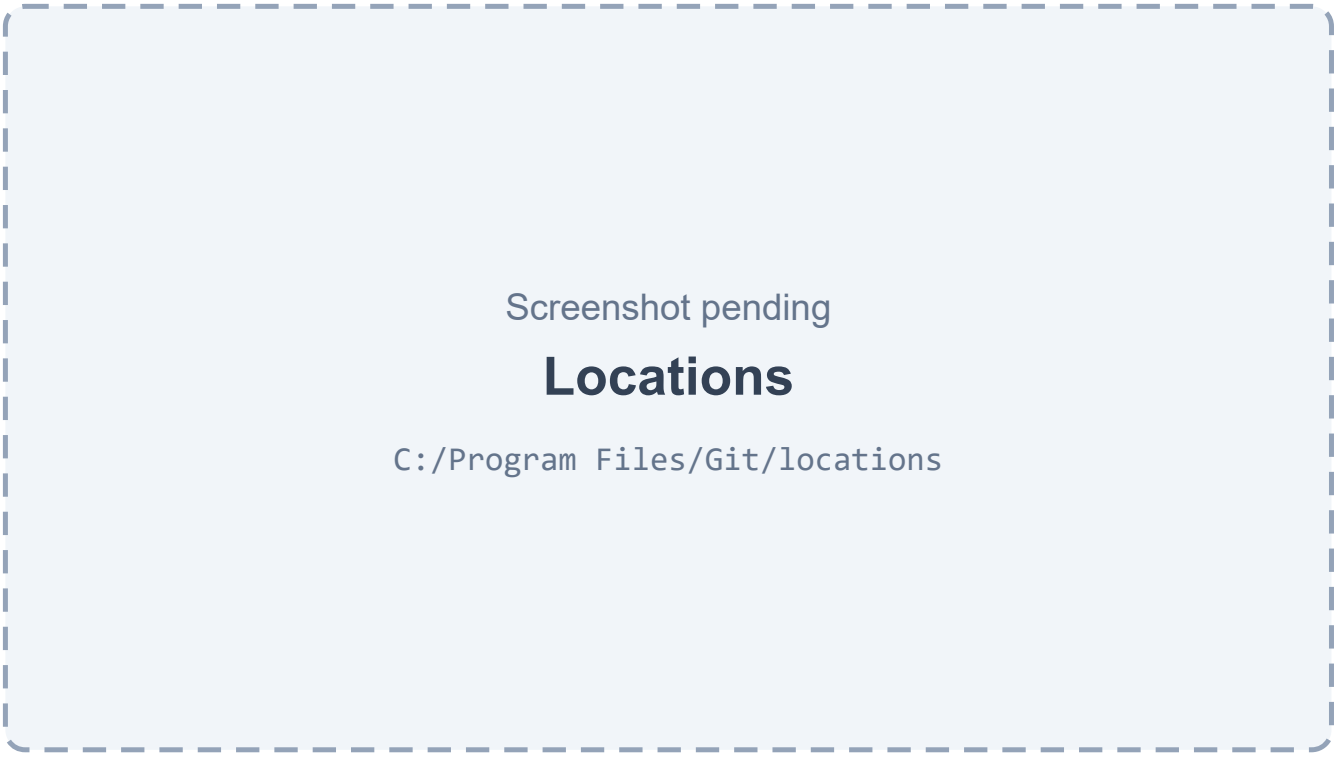


Figure 9.4 — Locations (screenshot pending).

The page is titled "Locations" with the subtitle "Manage clinic locations and their settings" and a **Create Location** button. A table lists active locations with the columns Name, Organization, City, Phone and Status, and three icon actions with tooltips: "Location settings", "Edit location" and "Deactivate location". The create and edit dialog holds the fields listed in section 9.6.1; its code field shows the example "e.g. NYC-01". Toasts confirm "Location created successfully", "Location updated successfully" and "Location deactivated".

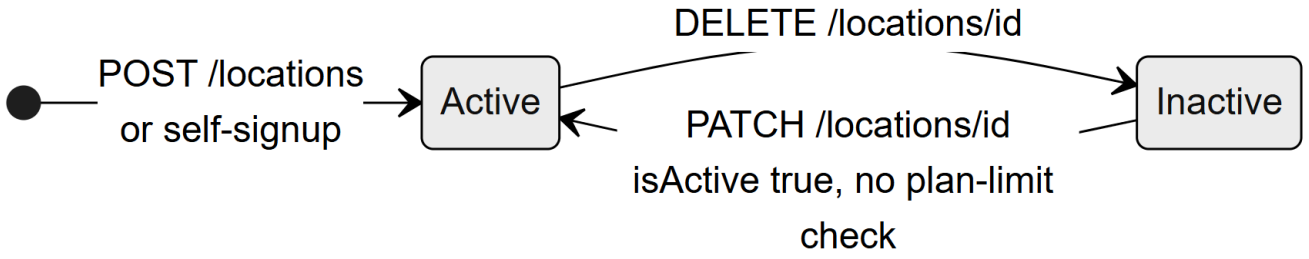


Figure 9.5 — The two states of a location's `is_active` flag.

The diagram reads left to right. A location becomes **Active** through `POST /locations` or self-signup. `DELETE /locations/id` moves it to **Inactive**. `PATCH /locations/id` with `isActive true` moves it back to Active, and the label notes that this path does not check the plan's location limit. There is no final state.

9.7 Location settings

Status: [Partial] — the tax rate and a few other values are used; the edit screen's save is expected to fail, and several stored values do nothing.

Each location has one `location_settings` row, created with the location or on first read.

Setting	Used by
`tax_rate` (percent, two decimals, default 0)	**Used.** Sales tax applied when an invoice is issued (tax is added on top of prices), and by lab billing and purchase orders.
`max_appointments_per_day` (default 50; 0 means no cap)	**Used.** Daily booking cap for the location.
`logo_url`	**Used.** Fallback logo for printed invoices when the clinic theme has none.
`appointment_slot_duration` (default 30)	Read only by the web app, as the default length in the booking form.
`advance_booking_days`, `cancellation_notice_hours`, `reminder_enabled`, `reminder_days_before`, `reminder_hours_before`, `reminder_channels`, `payment_gateway`, `late_fee_percentage`, `primary_color`, `theme`	Stored; no code reads them.

The table lists each setting and whether anything uses it.

- `GET /locations/:id/settings` and `PATCH /locations/:id/settings` both need `admin.locations.manage`. Staff without it (receptionists, doctors) cannot read the slot duration, so their booking form falls back to 30 minutes.
- **Field-name mismatch.** The update DTO names several fields differently from the stored columns (for example `defaultSlotDuration` against `appointmentSlotDuration`, `maxDailyAppointments` against `maxAppointmentsPerDay`). The screen sends the stored names plus `id` and timestamps, so its save should fail with 400; a client sending the DTO names would get success but nothing stored for those fields. In practice, the daily cap and slot duration cannot be changed through the API (by reading the code, not executed). The tax rate has the same name on both sides.
- The tax rate can also be set in the setup wizard's location step (0–100).

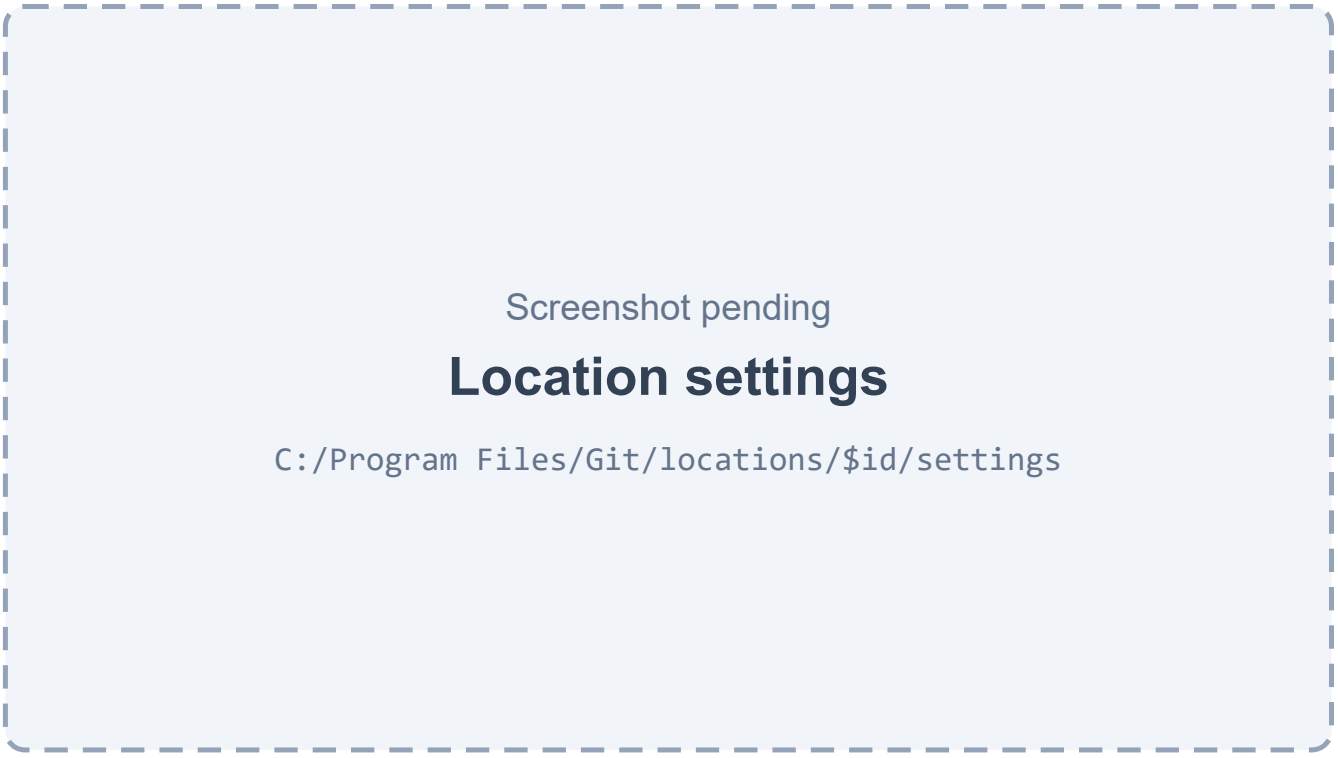


Figure 9.6 — Location settings (screenshot pending).

The page has a back button to Locations. An "Appointment Settings" card holds Slot Duration (mins), Max Appointments / Day, Advance Booking Days and Cancellation Notice (hours); the organization's legacy "Allow Location Override" switch can grey out three of these. A "Payment & Billing" card holds Tax Rate (%) and Late Fee

(%). The page also embeds the queue settings section for this location (the Queue chapter, Chapter 30, covers it). A "Reminder Settings" card exists in the code but is hidden until message delivery exists. Users without the permission see "No access — You don't have permission to manage this location's settings." On success a toast says "Location settings updated successfully".

Technical references: `backend/src/locations/entities/location-settings.entity.ts` · `UpdateLocationSettingsDto` · `LocationsController.getSettings` · `LocationsController.updateSettings` · `backend/src/billing/tax/location-tax-rate.ts` · `web-app/src/features/locations/LocationSettingsFeature.tsx`

9.8 Subscriptions and read-only mode (summary)

Status: [Implemented]

An organization with subscription dates is "managed": it is on a trial (self-signup gives 14 days with Professional limits) or paid until a date, and after that date it gets 3 grace days. When the grace days have passed, the clinic is in **read-only mode**: the fifth global guard, `SubscriptionGuard`, refuses every write with 402 `SUBSCRIPTION_EXPIRED`, while reads, sign-in, the account and setup pages and the super admin keep working. Recording a subscription payment (a manual super-admin action) lifts read-only mode at once; the status is computed on each request and cached for 30 seconds per process. Chapter 45 covers plans, limits, payments and the setup wizard in full.

Technical references: `backend/src/subscriptions/subscription.guard.ts` · `backend/src/subscriptions/plans.ts` · `SubscriptionsService.isReadOnly`

9.9 Limitations and known gaps

- Several edit screens (organization edit, organization settings, location edit, location settings) send whole rows and are expected to fail with 400 under the strict validation pipe; this was found by reading the code and not executed.
- Organizations and locations cannot be reactivated from any screen.
- New locations default to `UTC` and `USD`; timezone and currency are not validated.
- `is_main_branch` is not exclusive, and the last or main location can be deactivated.
- Organization and location create, edit and deactivate write no audit rows; only subscription changes are audited.
- `organization_settings` and most location settings are stored but unused.

9.10 Owner's FAQ

Can an owner create a new organization? No. Only the super admin can, or a new owner through self-signup, which creates a separate clinic.

Can an owner add a branch? Yes, from Locations, as long as the plan's location limit allows it and the subscription has not expired.

Can a branch code be changed? No. The code is fixed at creation; the token prefix falls back to it.

Where is the sales tax rate set? In the location's settings (Tax Rate %) or in the setup wizard's location step. Each branch has its own rate.

What happens to patients and staff when a branch is deactivated? Nothing is deleted or moved. The branch is no longer offered at sign-in and in the location switcher.

What does "unmanaged" mean? An organization with neither trial end nor paid-until date. It never expires and has no plan limits. Organizations created by the super admin start this way.

Chapter 10 — Features: entitlement, enablement and location overrides

A **feature** is a named part of the Platform, such as "Invoices" or "Lab Management", that can be on for one clinic and off for another. This chapter explains the three switches that decide whether a feature is active (the super admin's entitlement, the owner's switch and a per-location override), the 24 features that exist, how the backend enforces them and which screens control them. It is written for the product owner, who sells and configures features per clinic.

10.1 The three switches

Status: [Implemented]

In simple terms. The platform operator decides what a clinic is allowed to have; the clinic owner decides what the clinic actually uses; and the owner can switch a feature off at one branch. A feature works at a branch only when all three agree.

Switch	Who sets it	Stored in	Meaning
Entitled	Super admin	`org_feature_flags.is_entitled`, optional `entitled_until`	The feature is part of this clinic's plan. With `entitled_until`, it is a trial that ends at that moment.
Enabled	Owner (or anyone with `admin.features.manage`); the super admin can also set it	`org_feature_flags.is_enabled`	The clinic has switched the feature on. Only possible while entitled.
Location override	Owner (same permission)	`location_feature_overrides.is_disabled`	Switches the feature off at one location. It can never switch on something the organization has off.

The table lists the three switches, who controls each, where it is stored and what it means. A feature is **active** for an organization when its catalog entry is active, it is entitled (and the trial has not ended), and it is enabled. It is active at a location when, in addition, that location has not switched it off.

Defaults and missing rows.

- When an organization is created (by the super admin or by self-signup), the Platform writes one `org_feature_flags` row per active catalog feature: features that are "on by default" in the catalog are both entitled and enabled; all others are neither.
- If an organization has **no** row for a feature, it counts as entitled, and enabled according to the catalog default. This keeps organizations that predate entitlement working.
- A trial end date must be in the future when set. When it passes, the feature becomes "not entitled" with the reason `trial_expired`; the owner sees it locked.

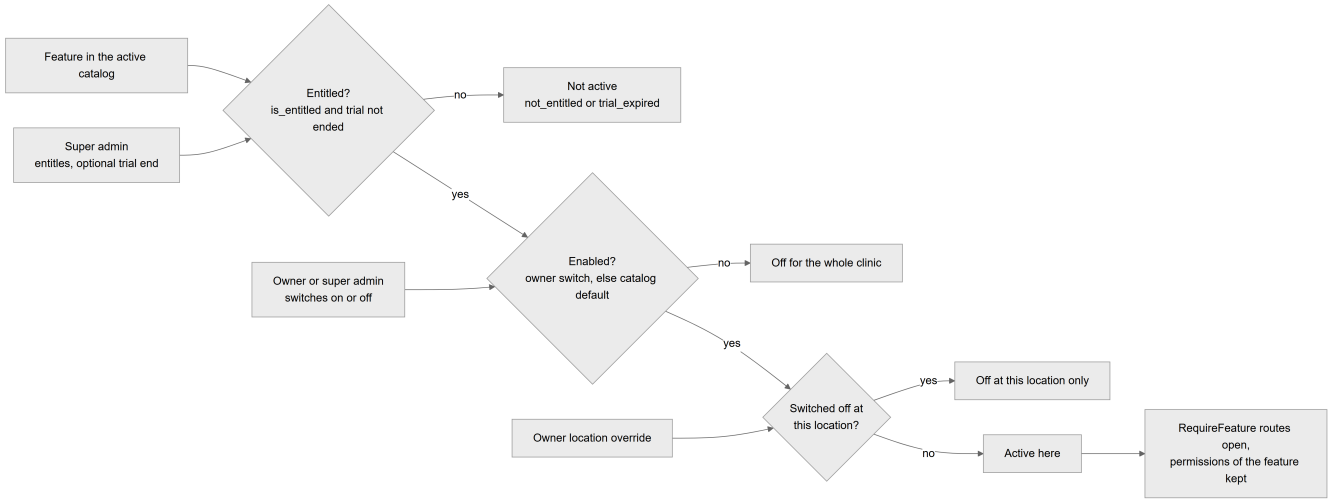


Figure 10.1 — The decision path from a catalog feature to "active at this location".

The flowchart reads left to right. It starts from "Feature in the active catalog" and reaches the first decision, "Entitled? is_entitled and trial not ended", which the super admin controls by entitling with an optional trial end. If the answer is no, the feature is "Not active" with the reason `not_entitled` or `trial_expired`. If yes, the second decision asks "Enabled? owner switch, else catalog default"; the owner (or the super admin) controls this switch. If no, the feature is "Off for the whole clinic". If yes, the third decision asks "Switched off at this location?", controlled by the owner's location override. If yes, the feature is "Off at this location only". If no, the feature is "Active here", and the final box says what that means: routes marked `@RequireFeature` open, and the permissions that belong to the feature are kept.

Technical references: `backend/src/feature-flags/feature-state.ts` · `featureState` · `resolveEnabledFeatures` · `narrowByLocation` · `FeatureFlagsService.seedFlagsForOrg`

10.2 The feature catalog

Status: [Implemented]

The catalog (`feature_catalog`) lists every feature key with a name, a module, a description, specialty tags, a default and a sort order. It is seeded from code on every start: keys that are missing are inserted, existing rows are never changed (so super-admin edits stay), and keys that are not built are deactivated.

There are **24 active features**:

Key	Name	On by default	Specialty tags	What it controls
`appointments.core`	Appointments	yes	general	12 permissions: booking and the whole queue.
`patients.core`	Patient Records	yes	general	9 permissions, including viewing, registering, clinical records and conducting consultations.
`patients.teeth_chart`	Teeth Chart	no	dental, ortho	`patients.chart.edit` ; one endpoint marked `@RequireFeature` .
`patients.treatment_plans`	Treatment Plans	yes	general	`plans.manage` , `plans.templates.manage` ; three endpoints marked `@RequireFeature` .
`patients.prescriptions`	Prescriptions	yes	general	`prescriptions.create` , `prescriptions.library.manage` ; six endpoints marked `@RequireFeature` .

Key	Name	On by default	Specialty tags	What it controls
`patients.documents`	Documents	yes	general	`patients.documents.manage`.
`patients.insurance`	Insurance	no	general	No permission; the three patient insurance endpoints are marked `@RequireFeature`.
`billing.invoices`	Invoices	no	general	9 billing permissions (desk, invoices, payments, refunds, cash close).
`billing.discounts`	Discounts & Coupons	no	general	`billing.discount.apply`, `billing.discount.custom`. Coupons no longer exist; discounts work through discount types.
`billing.expenses`	Expense Tracking	no	general	`expenses.view`, `expenses.manage`.
`billing.financial_summary`	Financial Dashboard	no	general	The financial summary endpoints (`@RequireFeature`) and the dashboard's finance widgets.
`billing.credit_packages`	Credit Packages	no	general	The credit package endpoints (`@RequireFeature`); no page uses them.
`billing.commissions`	Doctor Commissions	no	general	`earnings.own.view`, `earnings.all.view`, `compensation.manage`.
`comms.whatsapp`	WhatsApp	no	general	Nothing. No WhatsApp sending exists.
`comms.in_app`	In-App Notifications	yes	general	Nothing reads this key; in-app notifications work whether it is on or off.
`inventory.core`	Inventory	no	general	`inventory.view`, `inventory.stock.adjust`, `inventory.po.manage`.
`inventory.alerts`	Inventory Alerts	no	general	Read by the daily inventory alert job, which only writes a log line.
`hr.payroll`	Payroll	yes	general	`payroll.manage`.
`hr.leave`	Leave Management	no	general	`leave.approve`.
`hr.commissions`	Commissions	no	general	Nothing. Doctor commissions use `billing.commissions`.
`reports.financial`	Financial Reports	no	general	`reports.financial.view`, `finance.overview.view`.
`reports.clinical`	Clinical Reports	no	general	`reports.clinical.view`.
`reports.exports`	Data Exports	no	general	`reports.export`.
`dental.lab_orders`	Lab Management	no	dental, ortho	7 lab permissions (orders, billing, catalog, payables, links, templates).

The table lists each active key, its display name, whether new clinics get it entitled and enabled, its specialty tags and what it actually controls in the code. Most features work through permissions: when a feature is not active, every permission tied to it is dropped from every role at resolution time (Chapter 11). Eleven permissions belong to no feature ("core") and are always available: staff management and the nine Admin permissions.

Status of individual keys:

- [Implemented]: every key in the table whose "What it controls" names permissions or `@RequireFeature` endpoints.
- [Partial]: `billing.credit_packages` (backend only, no page), `inventory.alerts` (the job only logs), `billing.discounts` (the name still says "Coupons").
- [Not implemented]: `comms.whatsapp` and `hr.commissions` (switching them on changes nothing); `comms.in_app` (the switch has no effect).

Twelve keys are retired [Not implemented]. Their seed entries are commented out, and on every start the Platform deactivates any catalog row with these keys so nobody can switch on a feature that does nothing: `appointments.reminders`, `appointments.recurring`, `appointments.online_booking`, `appointments.waiting_room`, `patients.portal`, `patients.crm`, `billing.pdf_receipts`, `billing.insurance_claims`, `billing.online_payments`, `comms.sms`, `comms.email`, `dental.before_after`. (Some of the underlying ideas exist in other forms, such as the doctor-wise queue and the patient CRM tab, but these keys gate nothing.)

Catalog editing. `GET`, `POST /feature-flags/catalog` and `PATCH /feature-flags/catalog/:id` let the super admin read, add and edit catalog rows [Partial: backend only, no screen calls them].

Technical references: `backend/src/feature-flags/feature-catalog.seed.ts` · `FEATURE_CATALOG_SEED` · `RETIRED_FEATURE_KEYS` · `FeatureFlagsService.seedCatalog` · `backend/src/access/catalog/permissions.ts`

10.3 How the backend enforces features

Status: [Implemented]

There are two mechanisms.

1. **Through permissions (the common case).** Every permission names one feature. `AccessResolver` keeps a permission only if its feature is active at the request's organization and location (`filterByFeatures`). Turning a feature off therefore takes the permission away from every role **without changing the roles**; turning it back on restores access exactly as it was.
2. **Through `@RequireFeature(key)`.** Used where no permission expresses the feature, mainly on `@AnyMember()` routes. `FeatureFlagGuard`, the third global guard, collects every `@RequireFeature` key on the handler and its controller (both apply) and refuses with 403 'The "" feature is not enabled for this location' if any is not active. Super admins and `@Public()` routes skip the check. The location header narrows the set; the guard trusts it without a membership check, which is safe because a location can only remove features.

Caching. `FeatureFlagsService` keeps the active set per organization, and per organization and location, in process memory for 30 seconds. Every change clears the cache in the process that made it. The change also raises the organization's **access version** (automatically for `org_feature_flags` writes, explicitly for location overrides), which is broadcast over Redis so every other process drops its cache at once, and the web app gets `access.changed` and refetches. If Redis is down, other processes catch up within the 30-second lifetime.

Every change is audited in `audit_logs` (entity types `org_feature_flag` and `location_feature_override`), in the same transaction as the change, with the before and after values.

Technical references: `backend/src/feature-flags/guards/feature-flag.guard.ts` · `FeatureFlagGuard` · `requiredFeaturesOf` · `FeatureFlagsService.getEnabledFeatures` · `FEATURE_CACHE_TTL_MS` · `AccessVersionSubscriber` · `FeatureAdminService`

10.4 Super admin: entitle features for a clinic

Status: [Implemented]

What it does. Decides which features a clinic may use, optionally as a trial, and can also flip the clinic's own switch.

How the user does it. On the Organizations page, click the "Manage features" action of a clinic. The page lists features by module. Per feature, the **Entitled** switch grants or withdraws it, a date picker ("No trial end") sets a trial end, and the **Enabled** switch flips the owner's switch. Per module, **Entitle module** and **Withdraw module** act on all features of that module at once.

Behind the scenes — technically.

Action	Endpoint
Load the page	`GET /feature-flags/org/:organizationId`
Entitle or withdraw one feature, with optional trial end	`PATCH /feature-flags/org/:organizationId/:featureKey/entitlement`
Entitle or withdraw a module or a list of keys	`POST /feature-flags/org/:organizationId/entitlement/bulk`
Flip the enabled switch (no entitlement check)	`PATCH /feature-flags/org/:organizationId/:featureKey`
Flip many enabled switches	`POST /feature-flags/org/:organizationId/bulk`
Create missing rows from catalog defaults	`POST /feature-flags/org/:organizationId/seed`

The table maps each action on the screen to its endpoint. All carry `@RequireSuperAdmin()`. Rows are locked (`pessimistic_write`) while being changed, and each changed row writes an audit entry recorded as a platform action.

Rules. The trial end must be a valid date in the future; it is ignored when withdrawing. Re-entitling keeps a running trial; an ended trial is dropped. Unknown keys give 404.

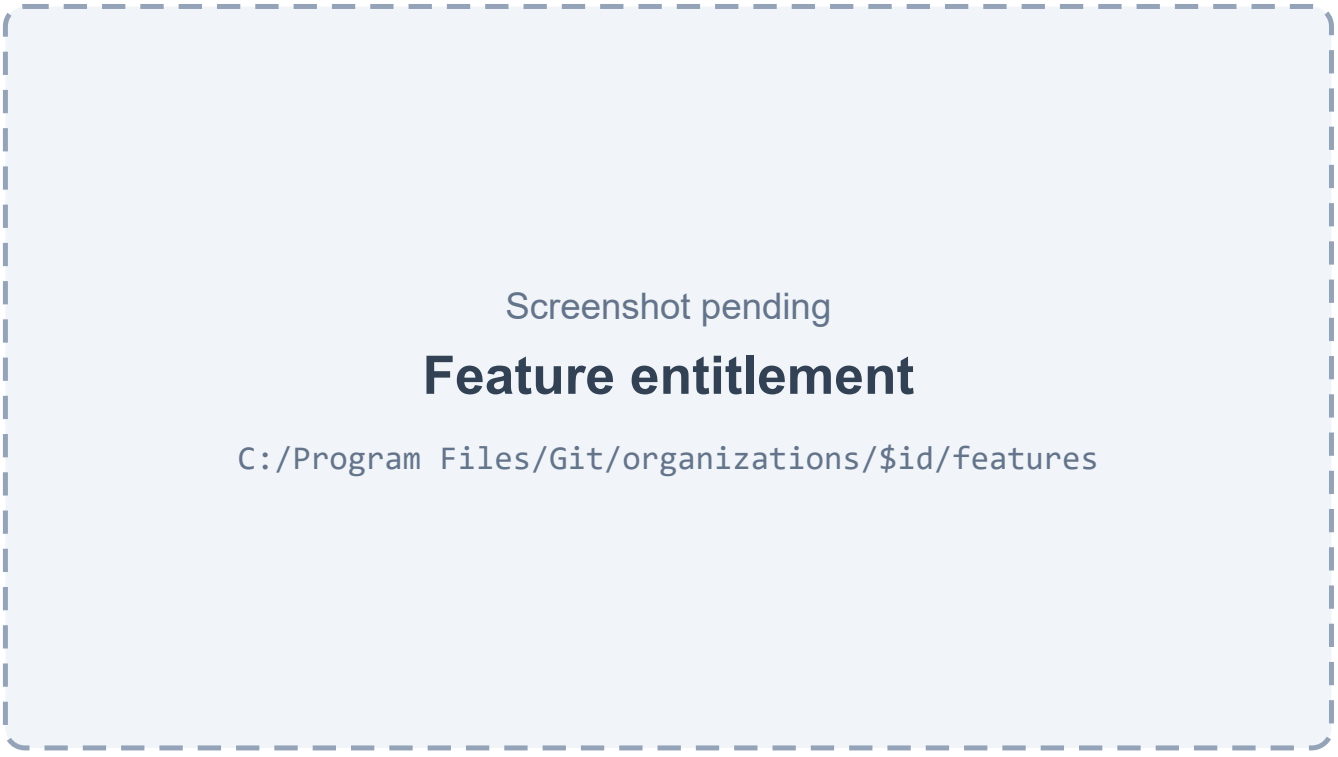


Figure 10.2 — Feature entitlement (screenshot pending).

The page is titled "Feature entitlement" with a back button to Organizations and the subtitle "Entitled = in this clinic's plan (optionally a trial). Enabled = the clinic owner's switch." A search box ("Search features") filters the list. Each module is a card with **Entitle module** and **Withdraw module** buttons. Each feature row shows its name, specialty tags as small badges, a state badge ("Not entitled", "Trial ended" or "Off by clinic" when it

applies), the description, a trial-end date picker, and two labelled switches, **Entitled** and **Enabled**, each with a tooltip. Empty and error states read "No features in the catalog" and "Could not load this organization's features."

Technical references: `backend/src/feature-flags/feature-flags.controller.ts` · `FeatureAdminService.setEntitlement` · `FeatureAdminService.bulkSetEntitlement` · `web-app/src/features/feature-flags/components/OrgFeatureEntitlements.tsx` · `EntitlementRow`

10.5 Owner: switch features on or off

Status: [Implemented]

What it does. Lets the clinic turn the features in its plan on or off for the whole clinic, or off at single locations.

How the user does it. Open **Features** in the clinic setup part of the sidebar (`/admin/features`). Each feature has a switch. Locked features (not entitled, or trial ended) show a lock, a "Locked" badge and a Roman-Urdu note asking the user to contact the platform admin, and their switch is disabled. Turning a feature off opens a confirmation, "Turn off ?", that lists the permissions that stop working and the roles that lose them ("No role uses this feature." when none). A location icon opens "Per-location settings for ", where each location can be switched off or back on.

Behind the scenes — technically.

Action	Endpoint	Rule
Load the page	`GET /admin/features`	Returns modules, features with entitled, enabled, active, trial end, locked reason, permission count, and the locations where each is off; plus the active locations.
Show the impact before turning off	`GET /admin/features/:key/impact`	Lists the feature's permissions and the active roles that grant any of them.
Turn on or off for the clinic	`PATCH /admin/features/:key`	Refused with 403 `NOT_ENTITLED` (with the reason `not_entitled` or `trial_expired`) when the clinic is not entitled.
Switch off or back on at one location	`PUT /admin/features/:key/locations/:locationId`	The location must belong to the clinic. "Back on" only removes the override; it never turns on a feature the clinic has off.

The table maps each owner action to its endpoint and rule. The whole controller requires `admin.features.manage` , a core permission that owners, super admins and the system Admin role hold. The organization always comes from the active tenant, never from the URL. A super admin acting here is recorded in the audit log as a platform action.

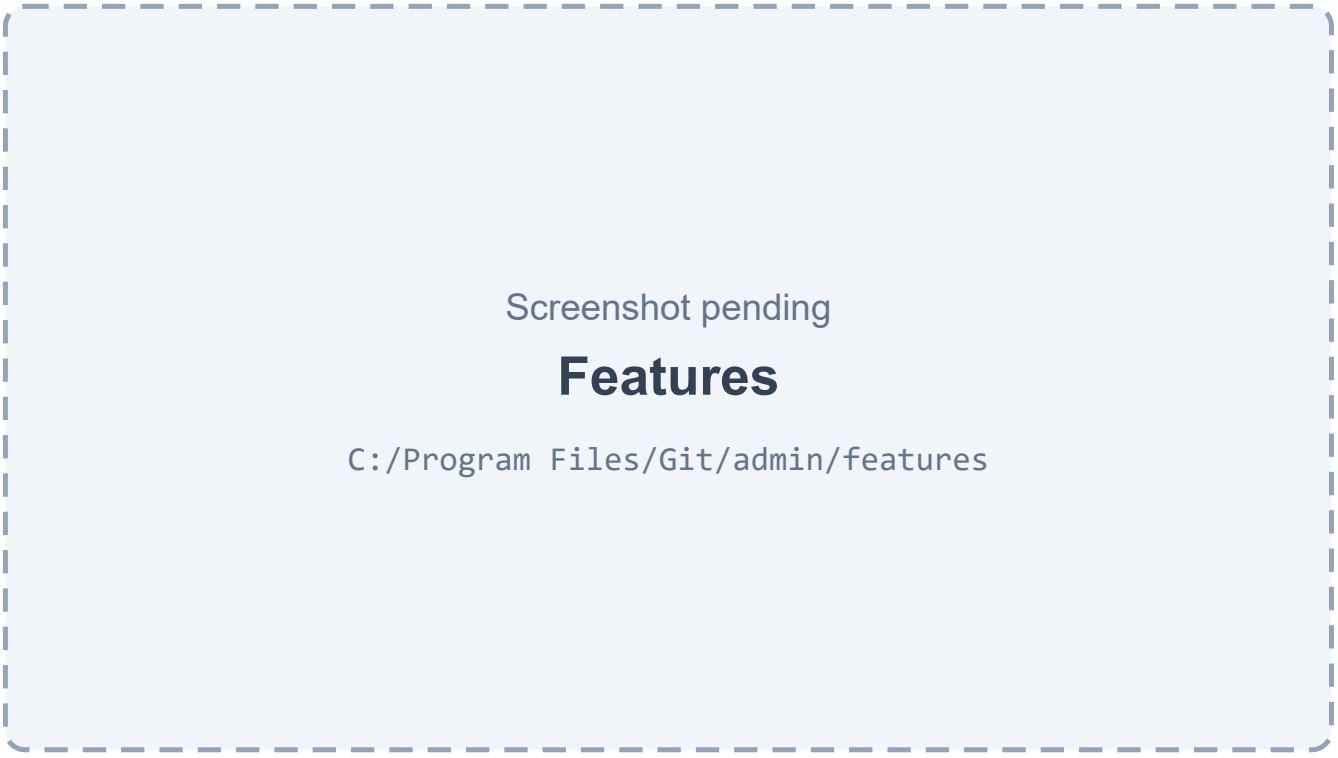


Figure 10.3 — Features (screenshot pending).

The page is titled "Features" with the subtitle "Turn the features in your plan on or off for the clinic, or for one location." A filter row has a search box ("Search features") and two selectors, "All modules" and "All states". Features are grouped by module. Each row shows the name, an "On", "Off" or "Locked" badge, a "Trial until " badge during a trial, an "Off at N locations" badge when overrides exist, the description, a per-location icon button and the main switch with a tooltip such as "Turn off for the clinic". Locked rows show "Plan mein nahi — super admin se rabta karein" or "Trial khatam ho gaya — super admin se rabta karein". The per-location sheet lists each location as "Follows the clinic (on)" or "Off at this location" with a switch. Empty states read "No features yet — The platform has not set up any features for this clinic." and "No locations".

Technical references: `backend/src/feature-flags/admin-features.controller.ts` · `AdminFeaturesController` · `FeatureAdminService.setEnabled` · `FeatureAdminService.setLocationOverride` · `FeatureAdminService.getImpact` · `web-app/src/features/feature-settings/`

10.6 How the web app uses features

Status: [Implemented]

- `GET /me/access` returns the active feature keys for the active organization and location together with the permissions (Chapter 11). Because permissions are already filtered by feature, most screens only ask `can(...)`.
- An older endpoint, `GET /feature-flags/my-features`, returns the same active set; `FeatureFlagsContext` loads it and offers `useFeature(key)` and a `RequireFeaturePage` wrapper, used by six pages and some widgets (for example the dashboard's finance widgets check `billing.financial_summary`).
- When features change, the `access.changed` socket event makes the web app refetch both.

Technical references: `web-app/src/context/FeatureFlagsContext.tsx` · `web-app/src/components/feature/RequireFeaturePage.tsx` · `FeatureFlagsController.getMyFeatures`

10.7 Limitations and known gaps

- Three active keys (`comms.whatsapp` , `comms.in_app` , `hr.commissions`) have no effect; an owner can switch them without any change in behaviour.
- `inventory.alerts` only produces a server log line, no notification.
- Catalog editing has no screen.
- The legacy `organization_settings.feature_*` switches (Chapter 9) are unrelated to this system and do nothing.

10.8 Owner's FAQ

What happens to roles when I turn a feature off? Nothing is deleted. The feature's permissions stop working for everyone; turning the feature back on restores them exactly.

Can I turn a feature on at one branch only? Not directly. Turn it on for the clinic and switch it off at the other branches.

Why can't I switch a feature on? It is not in your plan, or its trial has ended. Only the super admin can entitle it.

What happens when a trial ends? The feature becomes inactive at once (within the 30-second cache), and the owner's switch shows it locked with "trial ended".

Does turning a feature off log people out? No. Open screens receive an access update and lose the related menu items and buttons.

Chapter 11 — Roles and permissions

This chapter explains who may do what inside a clinic on the Platform. It covers the fixed list of 70 permissions, how they are grouped into roles, the five system roles every clinic starts with, how a clinic builds its own roles safely, and how the backend and the web app check permissions on every request and every screen. It is written for the product owner; the deeper customization screens (sidebar builder, theme, the owner's Features page) are covered in the Customization chapter, Chapter 58.

11.1 The model in one paragraph

Status: [Implemented]

The Platform never checks role names. It checks **permissions**, such as `billing.invoice.void`. A **role** is a clinic-owned bundle of permissions, stored as a map `{ permission: scope or true }` in `employment_roles.permissions`. An employee gets a role through an **assignment** at one location (Chapter 8), so the same person can have different roles at different branches. **Owners and super admins** have no role: they always get every permission at its widest scope. A permission whose **feature** is not active (Chapter 10) is dropped at resolution time. Anything not granted is denied (**fail-closed**).

Technical references: `backend/src/access/` · `backend/src/access/catalog/permissions.ts` · `docs/access-control/README.md`

11.2 The 70 permissions

Status: [Implemented]

The catalog is fixed in code; clinics combine its keys into roles but can never invent new ones. Each permission has a label, a module group, a feature, optionally a list of permissions it **implies**, optionally a set of **scopes**, and optionally a **high-risk** mark.

Module (group)	Permissions	Feature
Patients (12)	`patients.view` *, `patients.create`, `patients.edit`, `patients.delete` !, `patients.export` !, `patients.price.view`, `patients.clinical.view`, `patients.clinical.edit`, `patients.documents.manage`, `patients.chart.edit`, `plans.manage`, `plans.templates.manage`	`patients.core`; documents → `patients.documents`; chart → `patients.teeth_chart`; plans → `patients.treatment_plans`
Appointments (5)	`appointments.view` *, `appointments.book`, `appointments.reschedule`, `appointments.cancel`, `appointments.override_rules` !	`appointments.core`
Queue (7)	`queue.board.view`, `queue.checkin`, `queue.entry.manage`, `queue.entry.reprioritize`, `queue.doctor.own`, `queue.doctor.any`, `queue.display.manage`	`appointments.core`
Consultation (3)	`consultation.conduct` *, `prescriptions.create`, `prescriptions.library.manage`	`patients.core`; prescriptions → `patients.prescriptions`

Module (group)	Permissions	Feature
Billing (9)	`billing.desk.use`, `billing.invoice.view` *, `billing.invoice.edit`, `billing.invoice.void` !, `billing.payment.collect`, `billing.refund.issue` !, `billing.price.override` !, `billing.cash_close`, `billing.activity.view`	`billing.invoices`
Discounts (2)	`billing.discount.apply`, `billing.discount.custom` !	`billing.discounts`
Expenses (2)	`expenses.view`, `expenses.manage`	`billing.expenses`
Commissions (3)	`earnings.own.view`, `earnings.all.view`, `compensation.manage` !	`billing.commissions`
Lab (7)	`lab.orders.view` *, `lab.orders.create`, `lab.billing.manage`, `lab.catalog.manage`, `lab.payables.manage`, `lab.links.manage`, `lab.templates.share`	`dental.lab_orders`
Inventory (3)	`inventory.view`, `inventory.stock.adjust`, `inventory.po.manage`	`inventory.core`
Team (4)	`employees.view`, `employees.manage`, `payroll.manage` !, `leave.approve`	core; payroll → `hr.payroll`; leave → `hr.leave`
Reports (4)	`reports.clinical.view`, *`reports.financial.view`*, `reports.export` !, `finance.overview.view`	`reports.clinical`, `reports.financial`, `reports.exports`
Admin (9)	`admin.features.manage` !, `admin.roles.manage` !, `admin.users.manage` !, `admin.nav.manage`, `admin.settings.manage`, `admin.locations.manage`, `admin.audit.view`, `admin.theme.manage`, `notifications.settings.manage`	core (always on)

The table lists every permission by module, with its feature. An asterisk (*) marks the 7 **scoped** permissions; an exclamation mark (!) marks the 13 **high-risk** permissions. "Core" permissions belong to no feature and can never be switched off.

Scopes. A scoped permission is granted with how far it reaches:

- `own` — only records tied to the user as an employee (for example patients the doctor has seen through an appointment or the queue);
- `location` — records of the active location;
- `org` — every record of the organization.

A wider scope includes the narrower ones. Unscoped permissions are stored as `true` and reach the whole organization.

Implies. Granting a permission also grants what it implies, transitively. Examples: `patients.create` implies `patients.view`; `consultation.conduct` implies `patients.clinical.edit`, which implies `patients.clinical.view`; `queue.doctor.any` implies `queue.doctor.own`; `compensation.manage` implies `earnings.all.view`, which implies `earnings.own.view`; `appointments.override_rules` implies `appointments.book`. An implied scoped permission takes the granting permission's scope when it supports it, otherwise its narrowest scope.

Technical references: `backend/src/access/catalog/permissions.ts` · `PERMISSIONS` · `expandImplies` · `normalizeGrants` · `filterByFeatures`

11.3 Bundles: None, Basic and Full, and Advanced mode

Status: [Implemented]

So that owners do not tick 70 boxes, the roles screen offers three ready levels per module:

- **None** — nothing from the module.
- **Basic** — a sensible everyday set. For example: Patients Basic = view at location, register, edit, manage documents; Appointments Basic = view at location, book, reschedule, cancel; Queue Basic = board, check-in, manage entries; Consultation Basic = conduct at `own`, write prescriptions; Billing Basic = desk, view invoices at location, collect payments; Reports Basic = clinical reports at location; Admin Basic = clinic settings and audit log.
- **Full** — every permission of the module at its widest scope.

Advanced opens the module's full list of permissions, each with a checkbox and, for scoped ones, a scope selector. When a role's grants in a module match no level exactly, the module shows **Custom**.

Technical references: `backend/src/access/catalog/bundles.ts` · `BUNDLES` · `matchBundle` · `web-app/src/features/roles/components/ModuleBundleRow.tsx`

11.4 The five system roles

Status: [Implemented]

Every organization gets five **system roles** (templates). They are inserted when the organization is created and checked again on every start, insert-missing only: an existing system role is never changed, and one that is missing is recreated. System roles are **read-only**; to customise one, clone it.

Role	Home page	What it can do by default
Admin	`/dashboard`	Runs the clinic: the Full bundle of every module, minus the clinical acts `consultation.conduct`, `prescriptions.create` and `queue.doctor.own` and minus `earnings.own.view` (it still gets `queue.doctor.own` and `earnings.own.view` back through `queue.doctor.any` and `earnings.all.view`). It can run and force-complete any doctor's queue, manage roles, users, features, locations, the sidebar and the theme.
Doctor	`/queue/doctor`	Own patients (`patients.view` own), clinical records view and edit, dental chart, documents, treatment plans; own appointments; own queue; conduct consultations (own); write prescriptions; own earnings; lab orders (view own, create); clinical reports (own).
Receptionist	`/appointments`	Patients at the location (view, register, edit, see prices, documents); appointments at the location (view, book, reschedule, cancel); queue board, check-in, manage entries, TV display; billing desk, invoices at the location, collect payments, apply preset discounts; view lab orders at the location.
Assistant	`/appointments`	Patients at the location with clinical view and dental chart editing; appointments view and booking; queue board and check-in; inventory view; lab orders view at the location.
Accountant	`/billing`	Treatment prices; billing desk; invoices at the location (view, edit, void); payments, refunds, cash close, billing activity; expenses view and manage; every doctor's earnings; financial reports at the location, finance overview, report export; lab billing and lab payables.

The table lists the five templates with their home page and default powers. The **home page** is where a member of that role lands after sign-in; the web app only uses it if the user may open that page, otherwise it falls back to `/dashboard`. Owners and super admins are not shown here because they have no role.

Technical references: `backend/src/access/catalog/templates.ts` · `ROLE_TEMPLATES` · `AccessTemplatesService`

11.5 Custom roles and the roles screen

Status: [Implemented]

What it does. Lets the clinic create its own roles (for example "Hygienist"), copy a system role and change it, set its home page and see who has it.

How the user does it. Open **Roles & permissions** (`/admin/roles`). The list shows system roles first, then custom ones, each with a "System" or "Custom" badge and its member count; a search box ("Search roles…") and a type filter narrow it. **New role** asks for a name ("e.g. Hygienist") and a description and starts empty. Opening a role shows three tabs:

- **Permissions** — one row per module with None / Basic / Full and **Advanced**;
- **Home page** — the page members land on ("Default (dashboard)" when empty);
- **Members** — everyone with this role, per location ("Nobody has this role yet" when empty; roles are assigned on each employee's page).

The header offers **Delete**, **Clone**, **Discard** and **Save changes**. Leaving with unsaved changes asks "Leave without saving?". Granting a high-risk permission opens a "Grant high-risk" dialog that requires a **Reason**.

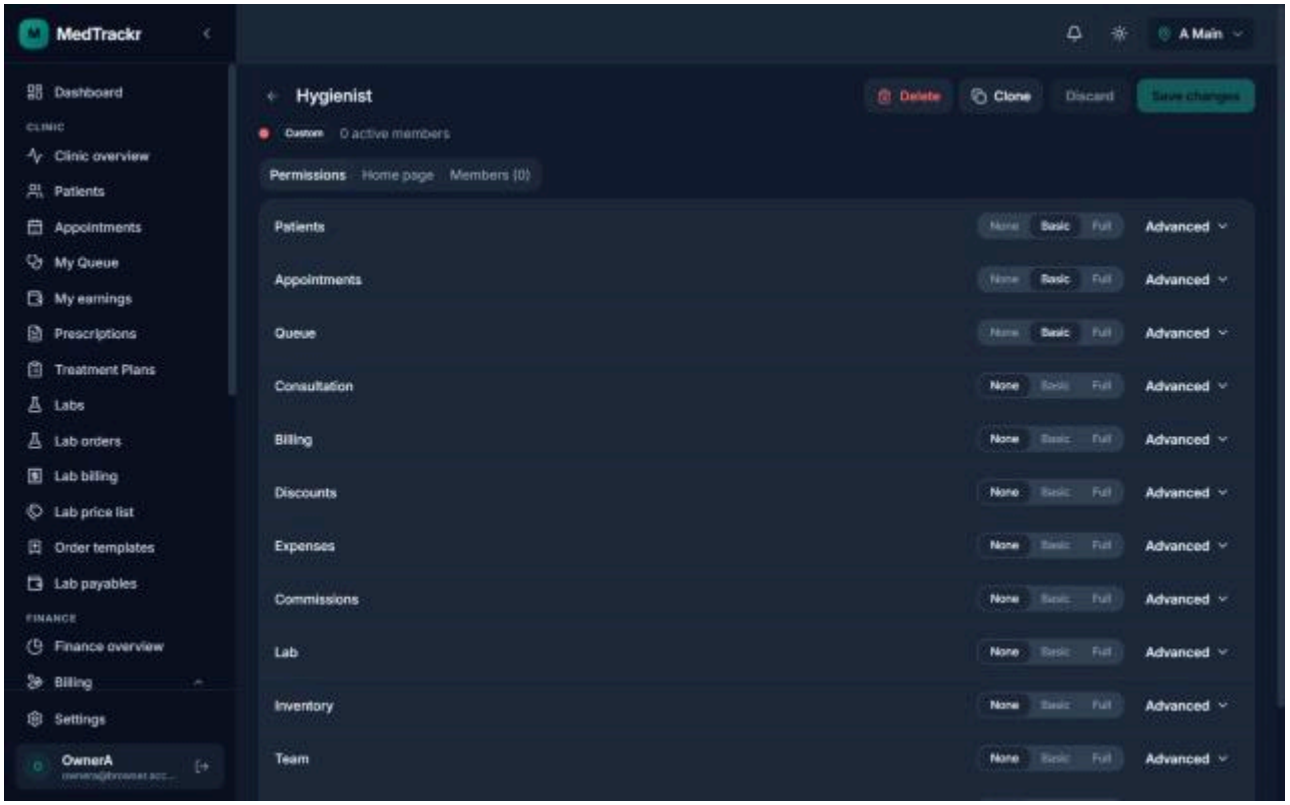


Figure 11.1 — The role editor for a custom role, showing None / Basic / Full per module.

The screenshot was taken on 2026-10-23 in dark theme and shows the former working name "MedTrackr" in the corner; the product is now called the Platform and each clinic sets its own name. The left sidebar shows the full owner menu (Dashboard, the Clinic section with Clinic overview, Patients, Appointments, My Queue, My earnings, Prescriptions, Treatment Plans, Labs, Lab orders, Lab billing, Lab price list, Order templates, Lab payables, and the Finance section with Finance overview and Billing), with Settings and the signed-in test owner at the bottom. The top bar shows the notification bell, the theme toggle and the active location. The main area shows a custom role named "Hygienist" with a back arrow, a "Custom" badge and "0 active members", and the buttons Delete, Clone, Discard and Save changes. Under the header are the tabs Permissions (selected), Home page and Members (0).

Each module (Patients, Appointments, Queue, Consultation, Billing, Discounts, Expenses, Commissions, Lab, Inventory, Team) has a None / Basic / Full selector and an "Advanced" expander; in this role Patients, Appointments and Queue are set to Basic and the other modules to None.



Figure 11.2 — Roles (screenshot pending).

The page is titled "Roles" with the subtitle "What each role can see and do. System roles are templates — clone one to customise it." A **New role** button sits at the top right, and a filter row holds the search box and the "All roles / System / Custom" type filter. Roles appear as cards or table rows with name, badge, description and member count, and row actions for opening, cloning and deleting. Empty state: "No roles yet — Create a role and pick what it can do, module by module."

Behind the scenes — technically.

Action	Endpoint
List roles with bundle summary and member counts	`GET /admin/roles`
One role with its members	`GET /admin/roles/:id`
Create	`POST /admin/roles`
Clone (any role, including system roles)	`POST /admin/roles/:id/clone`
Edit name, description, permissions, home page, active flag	`PATCH /admin/roles/:id`
Delete	`DELETE /admin/roles/:id`
Catalog, bundles and templates for the editor	`GET /access/catalog`

The table maps each action to its endpoint. All need `admin.roles.manage`. Each write runs in one transaction with an `audit_logs` row (entity type `employment_role`) holding before and after values and the reason. A legacy `/employment-roles` controller serves the employee form's role picker (`employees.view`) and applies the same rules to writes.

Rules and validations.

- **Escalation protection.** A user who is not an owner or super admin cannot grant a permission they do not hold, or a wider scope than they hold. Only newly added or widened grants are checked, implies included. Refusal: 403 `ESCALATION` listing the permissions.
- **Reason for high-risk grants.** Newly granted high-risk permissions need a reason, or 400 `REASON_REQUIRED`.
- **Valid maps only.** Unknown keys, a missing scope on a scoped permission, or a scope on an unscoped one give 400 `INVALID_PERMISSIONS`.
- **System roles are read-only** (403 `SYSTEM_ROLE_READ_ONLY`) and cannot be deleted.
- **Unique names** per organization (409 `ROLE_NAME_TAKEN`). A clone without a name becomes " (copy)", "(copy 2)" and so on.
- **No delete while in use.** A role with active members cannot be deleted (409 `ROLE_HAS_MEMBERS`). Deletion is permanent.
- **Home page** must be an in-app path starting with a single `/`.

Technical references: `backend/src/employees/employment-roles.admin.controller.ts` · `EmploymentRolesService` · `backend/src/employees/employment-roles.rules.ts` · `findEscalations` · `highRiskAdded` · `AccessCatalogController` · `web-app/src/features/roles/`

11.6 Resolving access: AccessResolver, cache and access version

Status: [Implemented]

In simple terms. For each user, clinic and branch, the Platform works out once what the user may do, remembers the answer, and throws the answer away as soon as anything relevant changes.

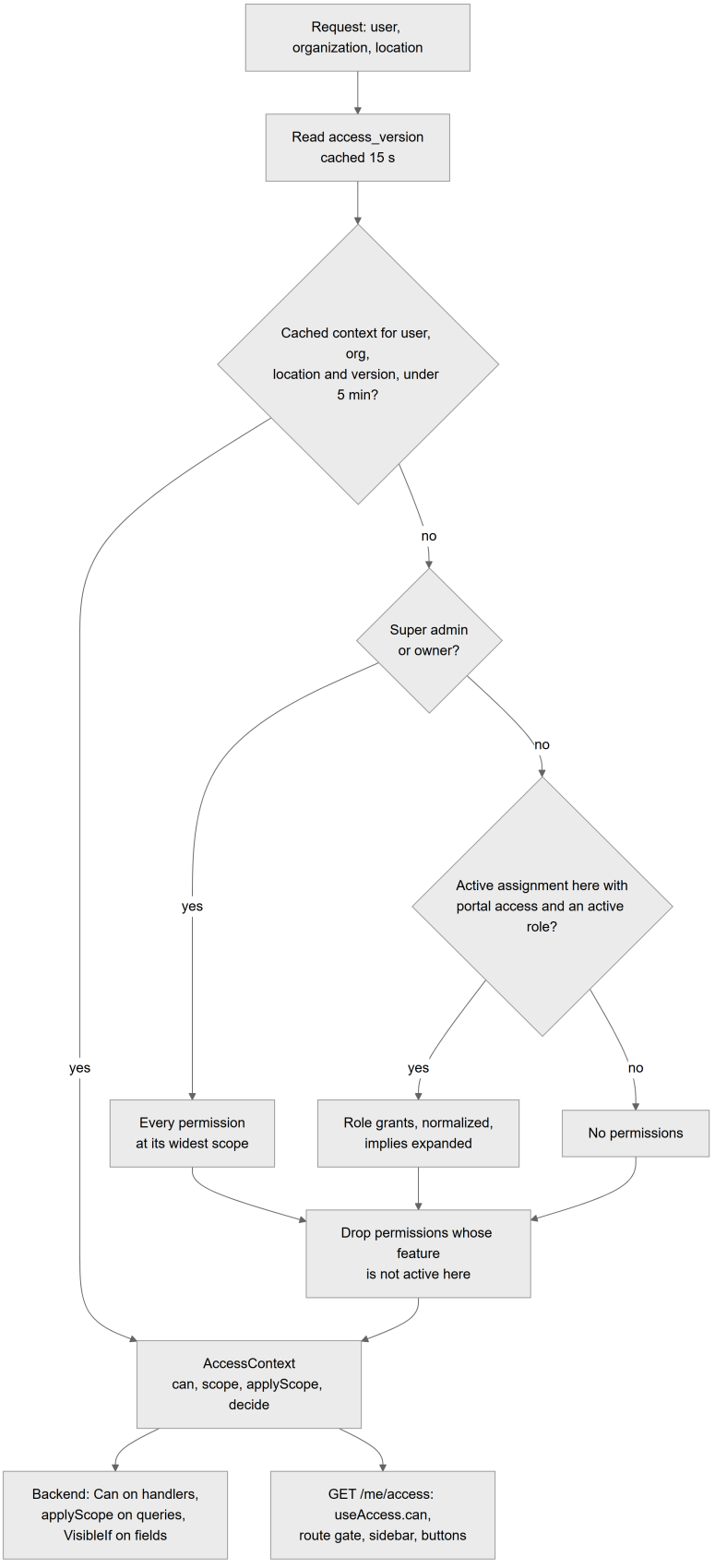


Figure 11.3 — The access resolution path from a request to the permission checks in the backend and the web app.

The flowchart reads top to bottom. A request brings a user, an organization and a location. The resolver first reads the organization's `access_version` (cached for 15 seconds). It then asks whether a cached context exists for this user, organization, location and version that is less than 5 minutes old; if yes, it goes straight to the AccessContext. If not, it asks "Super admin or owner?". If yes, the user gets every permission at its widest scope. If no, it asks whether the user has an active assignment at this location with portal access and an active role. If no,

the user gets no permissions; if yes, the role's grants are normalized and expanded with everything they imply. All three branches meet at "Drop permissions whose feature is not active here", which produces the AccessContext, offering `can`, `scope`, `applyScope` and `decide`. The context feeds two final boxes: the backend (`@Can` on handlers, `applyScope` on queries, `@VisibleIf` on fields) and the web app (`GET /me/access`, `useAccess().can`, the route gate, the sidebar and the buttons).

Technically.

- `AccessResolver.resolve(user, organization, location)` builds an `AccessContext` holding: the user, organization, location, version, `isSuperAdmin`, `isOwner`, `isMember` (super admin, owner or portal-enabled assignment), the employee id, active features, granted permissions, the role and its home page.
- The cache lives in process memory: at most 10,000 entries, 5 minutes each, keyed on user, organization, location and **access version**.
- `organizations.access_version` is raised by one after any committed change to employment roles, assignments (role, location, organization, active flag, portal access, login), organization feature flags or ownership. An entity subscriber does this automatically for TypeORM writes, once per transaction after commit; location overrides and raw SQL bump it explicitly.
- A bump is broadcast over Redis to every process, which drop their cached contexts and feature sets, and the Socket.IO gateway emits `access.changed` to the room `org:<id>`; open browser tabs refetch their access. If Redis is down, other processes catch up within 15 seconds (the version cache).
- **Fail-closed.** A missing key is no access. Roles with only old-style boolean flags give nothing. A role that is inactive, or belongs to another organization, gives nothing. A location header that is not a valid id means no location, so staff get no role permissions.

Technical references: `backend/src/access/access.resolver.ts` · `AccessResolver` · `backend/src/access/access-context.ts` · `AccessContext` · `backend/src/access/version/` · `AccessVersionService` · `AccessVersionSubscriber` · `AccessChangeBus`

11.7 Checking access in the backend

Status: [Implemented]

Every HTTP handler must say who may call it, with exactly one of these markers (on the method or its controller):

Marker	Meaning	Checked by
`@Can('perm', ...)`	All listed permissions are required; class and method lists merge.	`AccessGuard` (403 `NO_PERMISSION` with the missing keys)
`@AnyMember()`	Any active member of the location (owners and super admins too).	`AccessGuard`
`@Public()`	No sign-in (the route must also be in `PUBLIC_ROUTES`).	`JwtAuthGuard` list, all guards skip
`@RequireSuperAdmin()`	Super admin only.	`TenantContextGuard`
`@ContextFree()`	The handler checks access itself (for example `/locations`, `GET /me/access`).	the handler

The table lists the five markers, what they mean and which component enforces them. A controller protected by its own `@UseGuards(...)` (lab portal, magic links) also counts.

- **The coverage test.** `access-coverage.spec.ts` loads every controller and fails if any handler outside the auth and session controllers has none of the markers, or still uses the removed `@Roles()`. At runtime, a handler without a marker would pass the access guard, so this test is the safety net.

- **Row scope.** Handlers that list data take `@Access() access` and call `access.applyScope(queryBuilder, 'patients.view', columns)`. For `own` it adds the caller's employee condition, for `location` the active location condition, for `org` nothing extra; with no grant it adds a condition that matches no rows. Patients, for example, are "own" when the doctor has an appointment or queue entry with them, and "location" when their primary location or an appointment or queue entry is at the active location.
- **Field hiding.** A response property marked `@VisibleIf('perm')` is removed by a global interceptor when the caller lacks the permission; with no access context every guarded field is removed. It is used today for treatment and treatment-plan prices (`patients.price.view`).
- **Range decisions.** `access.decide(perm, { value, min, max })` returns allow, deny or "needs approval". It is used for discounts: changing a preset discount outside its range needs `billing.discount.custom`.
- **Rule for developers.** Code never checks role names; it asks `can(...)`.

Technical references: `backend/src/access/access.guard.ts` · `AccessGuard` · `backend/src/access/decorators/access.decorators.ts` · `backend/src/access/access-coverage.spec.ts` · `backend/src/access/visible-if/visible-if.ts` · `backend/src/patients/access/patient-scope.ts`

11.8 Checking access in the web app

Status: [Implemented]

- `GET /me/access` returns, for the active organization and location: the version, owner and super-admin flags, active features, granted permissions with scopes, the role, the employee id, the home page and the clinic's sidebar layout. `AccessProvider` loads it (fresh for 30 seconds, refetched on window focus and on every `access.changed` event) and serves it through `useAccess()`.
- **Buttons and widgets** ask `useAccess().can('perm')` or wrap content in `<Can perm="...">`.
- **Routes** declare `staticData: { can: ... }` (one permission, or an array meaning any of them). One gate in the root route, `RouteAccessGate`, checks every matched route and shows the **No access** page instead of the screen when the check fails.
- **The sidebar** is built from a developer-owned registry of 39 items, each with its own requirement (a permission, super admin only, owner or super admin, or lab member). Items the user may not open are hidden. The clinic's saved sidebar layout only arranges items; it never grants access.
- **The permission list in the web app** is a generated copy of the backend catalog (`features/access/generated/catalog.ts`), refreshed with `npm run access:export`.

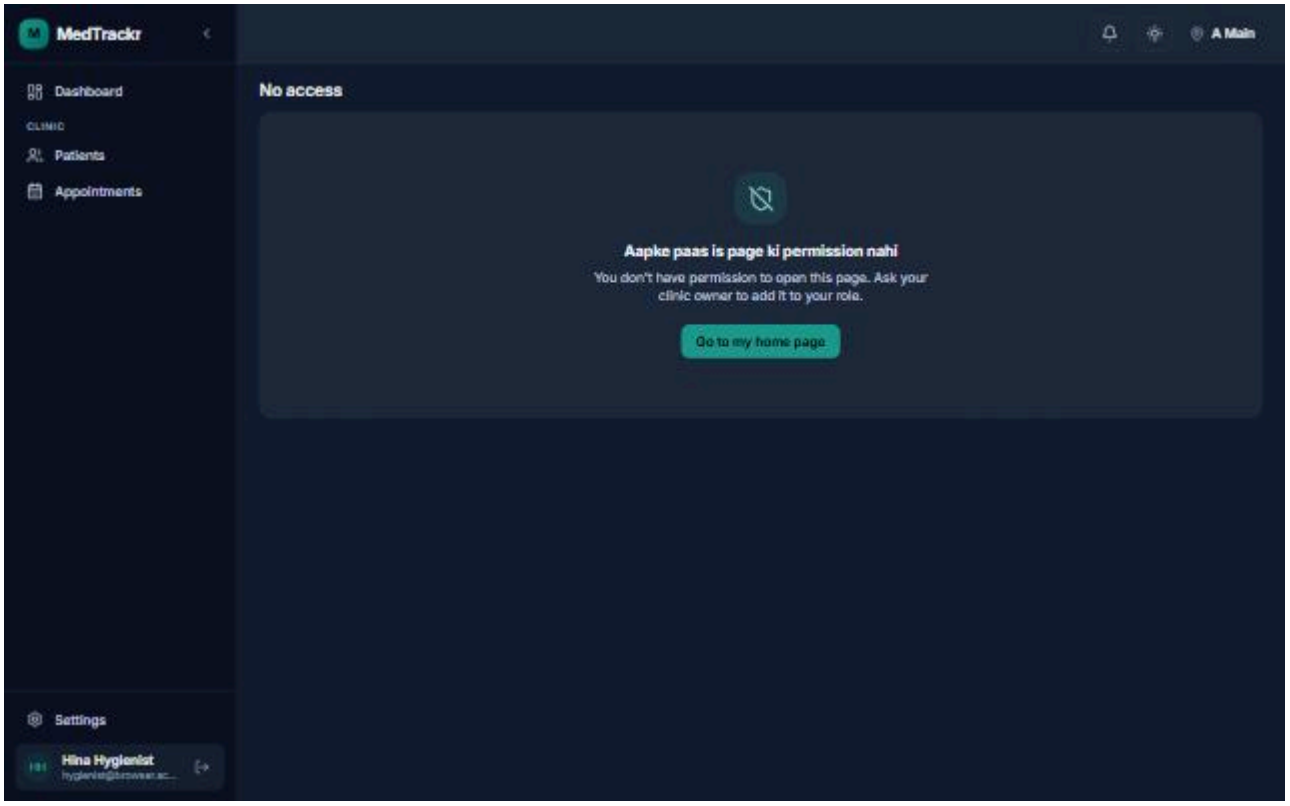


Figure 11.4 — The page shown when a role lacks a page's permission.

The screenshot was taken on 2026-10-23 in dark theme and shows the former working name of the product in the corner. A test user in a custom role opened a page their role does not allow. The sidebar is short: Dashboard, and under "Clinic" only Patients and Appointments, with Settings and the user at the bottom. The main area is titled "No access" and shows a shield icon, the Roman-Urdu headline "Aapke paas is page ki permission nahi" ("You do not have permission for this page"), the English text "You don't have permission to open this page. Ask your clinic owner to add it to your role." and a button "Go to my home page", which opens the role's home page, or the dashboard when none is set.

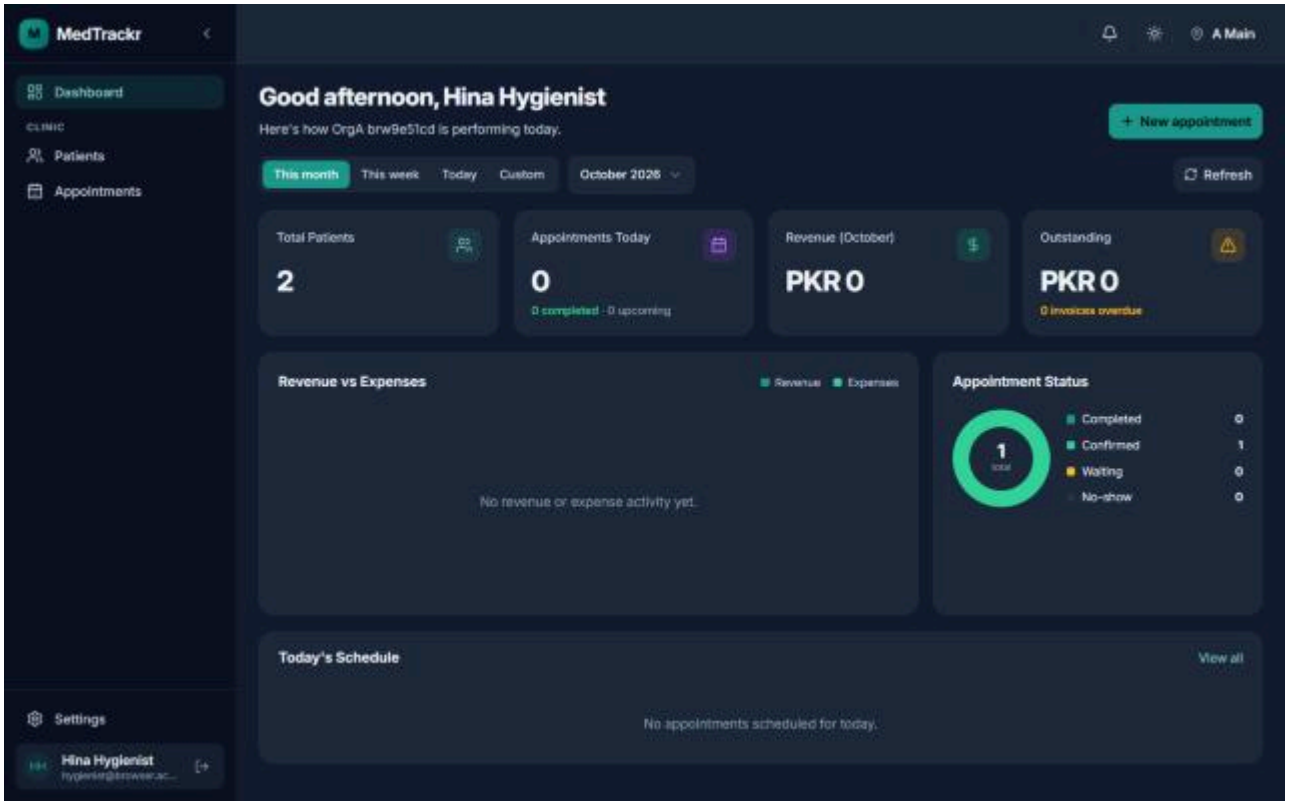


Figure 11.5 — The sidebar of a user in a restricted custom role.

This screenshot from 2026-10-23 (dark theme, former working name in the corner) is used only to show the sidebar; the dashboard content in it predates the dashboard redesign of 2026-10-23. The user holds a custom "Hygienist" role with the Basic level for Patients, Appointments and Queue. The sidebar therefore shows only Dashboard and, under "Clinic", Patients and Appointments, plus Settings at the bottom. Every other menu item is hidden because the role lacks its permission or the item needs an owner or super admin.

Technical references: `backend/src/access/access.controller.ts` · `MeAccessController` · `web-app/src/features/access/components/AccessProvider.tsx` · `RouteAccessGate` · `web-app/src/features/access/components/NoAccess.tsx` · `web-app/src/features/access/nav/nav.registry.ts` · `web-app/src/features/access/nav/resolveNav.ts` · `useAccess`

11.9 Limitations and known gaps

- A system role deleted directly in the database is recreated on the next start; a clinic cannot permanently remove a system role (the API refuses to delete one anyway).
- Missing markers are caught by a unit test, not at runtime.
- `POST /auth/switch-location` still returns eight old permission flags derived from the new permissions; the web app ignores them.
- Permissions follow the assignment's location: a staff member with no assignment at the active location gets no role permissions there.

11.10 Owner's FAQ

Can a receptionist see treatment prices? Yes with the default Receptionist role, which holds `patients.price.view`. Without it, prices are removed from the API responses, not only hidden on screen.

Can I give my manager the right to edit roles without making them all-powerful? Yes: grant `admin.roles.manage`. They still cannot grant anything they do not hold themselves.

Why can't I edit the Doctor role? System roles are read-only templates. Clone it and edit the copy, then assign the copy.

How quickly does a role change take effect? On the next request: the change raises the access version, every server drops its cached answer, and open screens refetch their access.

What does a doctor see with `patients.view` at scope own? Only patients they have an appointment or queue entry with.

Where is the history of role changes? In the audit log, as `employment_role` entries with before and after values and the reason given for high-risk grants.